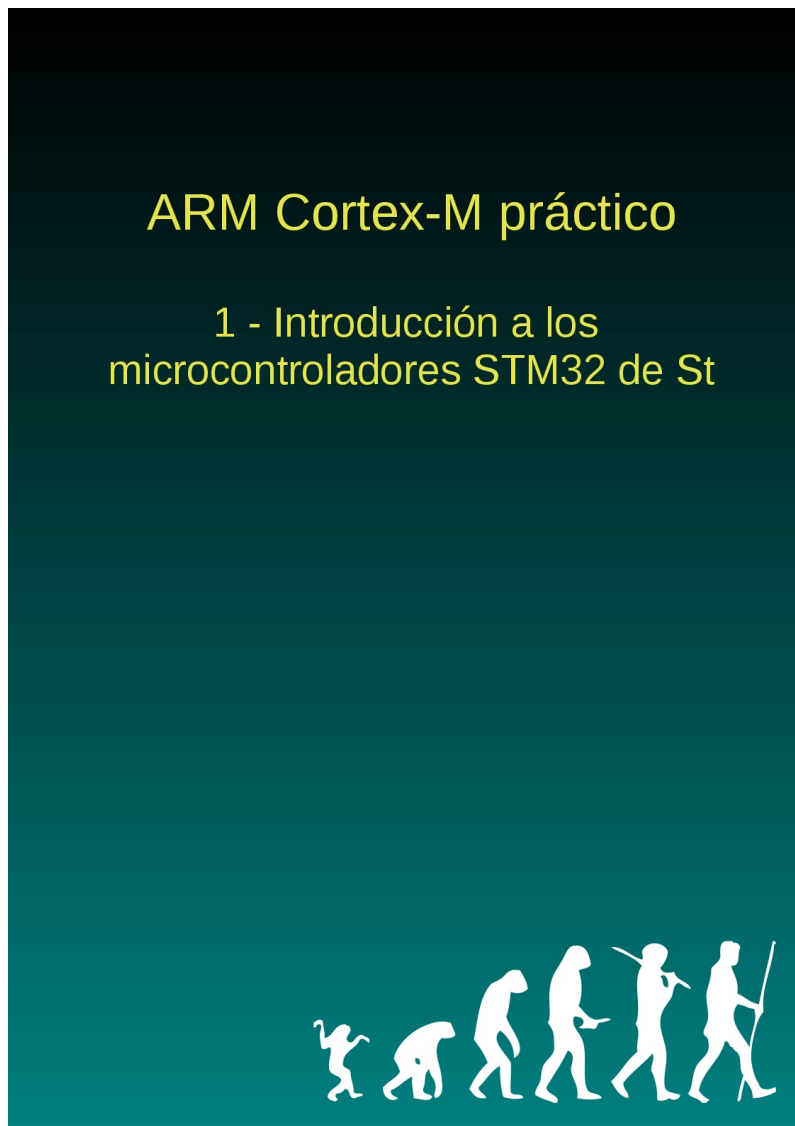

ARM Cortex-M práctico.

1 - Introducción a los microcontroladores STM32 de St



Àngel Perles

Departament d'Informàtica de Sistemes i Computadors

Universitat Politècnica de València



Licencia Reconocimiento – NoComercial – SinObraDerivada (by-nc-nd): No se permite un uso comercial de la obra original ni la generación de obras derivadas.

ARM Cortex-M práctico. 1 - Introducción a los microcontroladores STM32 de St

Àngel Perles. aperles@disca.upv.es

26 de octubre de 2019

Presentación

Cuidadín que esto no es más que un borrador y unas anotaciones.

Yo he trabajado siempre con microcontroladores de la familia 8051 porque me permitían elegir fabricante y herramientas en función del tipo de problema a resolver. Así fuí cambiando del fabricante Intel a Siemens (ahora Infineon), a Temic, a Atmel y, finalmente, a los formidables Silabs. Ni Microchip (PIC), ni Motorola (HC11) ni los AVR de Atmel, ni Renesas eran capaces de competir con este estándar industrial, aunque competían bien en otras ventajas.

En el año 200X decidí que era el momento de cambiar a una arquitectura de 32 que me facilitase la escritura de las aplicaciones en lenguaje C, proporcionase más rendimiento y mantuviese las ventajas del 8051. Tras analizar distintas arquitecturas, decidí apostar por la arquitectura ARM Cortex-M por el modelo de licencia seguido y porque ya había dos fabricantes que había apostado por ella: St y Luminary Micro (ahora Texas Instruments). Hubo suerte y ahora hay infinidad de fabricantes que los producen, herramientas libres y comerciales excelentes y una magnífica comunidad donde localizar información.

Más adelante decidí trasladar el cambio al ámbito educativo. Tras unas pruebas de concepto (asignaturas en la Universidad, cursos a profesionales, etc.) llego a la conclusión de que hace falta un libro adecuado al nivel de los alumnos y que no hay ninguno que se adapte al perfil de mis alumnos.

Con el fin de ir solucionando el problema, voy a ir anotando lo que voy haciendo con los alumnos, a ver si así es más fácil ir avanzando en la línea adecuada y se consiguen profesionales más preparados en este ámbito. En cualquier caso, es imposible lograr una obra que lo cubra todo, así que el enfoque está muuuuuuyyyyyy orientado al perfil de mis alumnos.

Explicar el objetivo de la obra: nada de cosas maravillosas conectadas por móvil a la nube y demás chorrads que son espectaculares pero no enseñan a sert independiente. Eso es básico y, si hay segunda parte, estará orientada a ser productivo para lograr hacer esas maravillas sabiendo lo que se está haciendo.

Ahora voy a explicar cómo usar esto. Partimos de que el aprendiz tiene conocimientos básicos de electrónica digital, electrónica analógica y de programación en lenguaje C. Si, además, se tienen conocimientos sobre otros microcontroladores (PIC, AVR, 8051, HC-11, etc.) entonces será fácil seguir esto (espero). Si no se cumplen estas condiciones, mejor no sigas y empieza con la fantástica plataforma Arduino. Eso no es para jugar, es para desarrollos serios para empresas.

Empieza a trabajar de manera lineal y, cuando no se entienda algo de programación, acude al apartado correspondiente para ver si te lo resuelve. Si no es así, deberás buscar ayuda fuera del libro.

Àngel Perles

Reconocimientos

A Jaume Planas, de St España, y a Ruben Carrillo, de Venco, por su apoyo en la puesta en marcha del curso con las placas Discovery.

A Ricardo Mercado, por la imagen del equipo comercial desarrollado por él.

A Antonio Sánchez, de Fermax, por su apoyo dándoles un baño de realidad a mis alumnos de Electrónica y Automática.

Índice general

Presentación	III
Reconocimientos	V
Contenido	VII
1 Los microcontroladores	1
1.1 Introducción	1
1.2 Qué es un microcontrolador	1
1.3 Aplicaciones del microcontrolador	3
1.4 Sistemas embebidos, embarcados o empotrados	4
1.5 Clasificaciones típicas de los microcontroladores	5
1.5.1 Por el tamaño de palabra	5
1.5.2 Por el enfoque en la ejecución de instrucciones: CISC o RISC	6
1.5.3 Por el camino usado para los datos y las instrucciones: Von Neuman o Harvard	6
1.6 Elijiendo el microcontrolador adecuado	7
1.6.1 El mercado de microcontroladores	8
1.6.2 Familias, fabricantes y licencias	9
1.6.3 Elección de la familia de microcontroladores ARM Cortex-M	10
1.6.4 Elección del microcontrolador St STM32F4xxx	13
1.7 Las herramientas	13
1.7.1 El lenguaje de programación	13
1.7.2 Sistemas operativos o microkernels	14
1.7.3 Simuladores	14
1.7.4 Sondas de depuración	14
1.7.5 Kits de desarrollo/evaluación	16
2 Arquitectura del microcontrolador St STM32F4xxx (ARM Cortex-M4F)	17
2.1 Introducción	17

2.2 Familia St STM32F4	17
2.3 Arquitectura del microcontrolador STM32F407.	19
2.4 Encapsulado y patillaje de un St STM32F4	20
2.5 Sistemas con un STM32F4	23
2.5.1 Alimentación, reloj y reset	23
2.5.2 Ejemplos	25
2.5.3 Mínimo, mínimo	25
2.5.4 Razonable	25
2.6 ¿Más adelante?	25
2.6.1 Mapa de memoria	25
2.6.2 Arranque	25
 3 Entrada/salida digital	 27
3.1 Introducción	28
3.2 Puertos y líneas.	29
3.3 La célula de cada pin	29
3.4 Salida digital	30
3.4.1 La célula en modo salida	30
3.5 Biblioteca HAL para gestionar la GPIO.	32
3.6 Entrada digital	36
3.6.1 Célula en modo entrada	36
3.6.2 Ejemplo: Un pulsador	37
3.7 Acceso directo a los registros de periférico	39
3.7.1 Ejemplo: dial selector	40
3.8 Ejemplo: un display de 7 segmentos	45
3.9 Teclados matriciales	49
3.10 Multiplexado temporal con varios display de 7 segmentos	53
3.11 E/S digital con el STM32CubeMX	59
 4 Interrupciones	 61
4.1 Introducción	61
4.2 Funcionamiento general y jerga del sistema de interrupciones	63
4.3 Interrupciones en los ARM Cortex-M	65
4.3.1 Cosas pendientes	69
4.4 El periférico EXTI y las interrupciones	69
4.4.1 Funcionalidad	69
4.4.2 Configuración de interrupciones en pines GPIO	70
4.4.3 El servicio de interrupción	72
4.4.4 El servicio de interrupción con “callback”	75

5	Contadores y temporizadores	77
5.1	Introducción	77
5.2	Los <i>timers</i> en genérico	78
5.3	SysTick, el contador común a los Cortex-M	78
5.3.1	Biblioteca HAL	80
5.3.2	Midiendo el paso del tiempo	81
5.3.3	Haciendo pausas de precisión.	82
5.3.4	Tareas periódicas	82
5.3.5	NO MIRAR: Cómo es	83
6	Programación en C para ARM Cortex-M	87
6.1	Introducción	88
6.2	El desarrollo es “cruzado”	88
6.3	Del código al ejecutable	89
6.4	Tratando con datos	91
6.4.1	Tipos enteros	91
6.4.2	Tipos enumerados	92
6.4.3	Tipos en coma flotante	93
6.4.4	Algunos atributos fundamentales	95
6.5	Tablas “ <i>look-up</i> ”	100
6.6	Tratamiento bit a bit. Máscaras	101
6.6.1	Representación externa. Representación interna	101
6.6.2	Representación hexadecimal	102
6.6.3	Operadores de bit	104
6.6.4	Máscaras	104
6.7	Bibliotecas	107
6.8	El <i>Cortex Microcontroller Software Interface Standard (CMSIS)</i>	111
6.9	El firmware STM32Cube HAL y LL de St	113
7	Entorno de trabajo	117
7.1	Introducción	118
7.2	Ordenador personal	119
7.3	Placas de evaluación.	120
7.3.1	STM32F429ZI Discovery	120
7.3.2	STM32L476 Nucleo-64.	120
7.4	Sistema de depuración St-Link.	122
7.4.1	Instalación y configuración	123
7.4.2	Comprobación de la placa.	124
7.4.3	Actualización de la sonda.	124

7.4.4	Volcado de ejecutables	124
7.5	Keil MDK-ARM 5	126
7.5.1	Obtener e instalar Keil	126
7.5.2	Construir un proyecto	127
7.5.3	Configurar Keil para usar St-Link	129
7.5.4	Volcado de un proyecto en un microcontrolador	129
7.5.5	Añadir archivos a un proyecto existente	131
7.5.6	Crear archivos nuevos	131
7.5.7	Añadir rutas de búsqueda de archivos de cabecera .h	132
7.6	STM32CubeMX: STM32Cube initialization code generator	133
7.7	Paquete de bibliotecas STM32Cube	135
7.8	Plantilla para la placa St Discovery 429i-Disc1	136
7.8.1	Unidad de disco virtual O:	136
7.8.2	Puente en la Discovery para habilitar	138
7.8.3	Pendiente para la L4	138
8	Prácticas	139
8.1	Práctica: Instalación de St-Link y volcado de ejecutable en la placa St Discovery	140
8.1.1	Objetivos	140
8.1.2	Material necesario	140
8.1.3	Instalación y comprobación de St-Link	140
8.1.4	Actualización de la sonda St-Link de la Discovery	141
8.1.5	Volcado de ejecutables en la Discovery	141
8.1.6	Volcado de la demo de la Discovery	141
8.2	Práctica: Instalación y prueba de ARM Keil MDK	143
8.2.1	Objetivos	143
8.2.2	Material necesario	143
8.2.3	Instalación del software	143
8.2.4	Construcción del proyecto “CubeLEDs”	143
8.2.5	Volcado del proyecto en el microcontrolador	144
8.2.6	Ampliación: cambio en el parpadeo	145
8.3	Práctica: Instalación y prueba de las STM32Cube F4 HAL	147
8.3.1	Objetivos	147
8.3.2	Material necesario	147
8.3.3	Instalación y prueba de las STM32F4Cube HAL	147
8.4	Práctica: Uso de la plantilla oficial	149
8.4.1	Objetivos	149
8.4.2	Material necesario	149
8.4.3	Introducción	149
8.4.4	Preparación inicial	150
8.4.5	Usando la plantilla	150

8.4.6 Salida estándar con la plantilla	150
8.4.7 Qué puñetas es eso de la salida estándar.	153
8.5 Práctica: Bibliotecas.	157
8.5.1 Objetivos.	157
8.5.2 Material necesario	157
8.5.3 Introducción	157
8.5.4 Preparación inicial	158
8.5.5 Incorporar la biblioteca led	158
8.5.6 Probar el módulo LED	158
8.5.7 Actividad: cambio de LED	159
8.6 Práctica: Salida digital con una válvula.	161
8.6.1 Objetivos.	161
8.6.2 Material necesario	161
8.6.3 Introducción	162
8.6.4 Preparación inicial	162
8.6.5 Actividad: desarrollar el módulo valve	162
8.6.6 Actividad: comprobar el funcionamiento del módulo	162
8.6.7 Actividad opcional: mostrar mensajes por la salida de depuración	163
8.6.8 Actividad opcional: mostrar información por el LCD	163
8.7 Práctica: Entrada digital con un sensor de rebose	165
8.7.1 Objetivos.	165
8.7.2 Material necesario	165
8.7.3 Introducción	165
8.7.4 Preparación inicial	166
8.7.5 Actividad: desarrollar la biblioteca overflow	166
8.7.6 Actividad: comprobar el módulo desarrollado.	167
8.7.7 Actividad opcional: desarrollar una aplicación de control.	167
8.7.8 Actividad opcional: conexión real de la válvula.	167
8.8 Práctica: Entrada digital con máscaras	169
8.8.1 Objetivos.	169
8.8.2 Material necesario	169
8.8.3 Introducción	169
8.8.4 Preparación inicial	170
8.8.5 Actividad: incorporar biblioteca STM32F4xx_AP_GPIO	170
8.8.6 Actividad: incorporar biblioteca de lectura del pulsador (button)	170
8.8.7 Actividad: comprobación de la biblioteca button	170
8.8.8 Actividad opcional: mostrar estado del pulsador en el LCD	171
8.9 Práctica: Optimizaciones de código para el display de 7 segmentos.	173
8.9.1 Objetivos.	173
8.9.2 Material necesario	173
8.9.3 Introducción	173
8.9.4 Preparación inicial	174

8.9.5	Modificar <code>main()</code>	174
8.9.6	Tarea: Incorporar las distintas implementaciones de <code>display7seg.c</code>	175
8.9.7	Tarea: Optimizaciones del compilador	175
8.9.8	NO HACER: Tarea: Probar el display	176
8.10	Práctica: Usando EXTI para contar vehículos en una carretera	177
8.10.1	Objetivos	177
8.10.2	Material necesario.	177
8.10.3	Introducción	177
8.10.4	Preparación inicial	178
8.10.5	Modificar <code>main()</code>	178
8.10.6	Tarea: Desarrollar la biblioteca <code>cars</code>	179
8.10.7	Tarea: Añadir el manejador	179
8.10.8	Probar la aplicación	180
8.10.9	Ampliación: Salida por la pantalla gráfica	180
8.10.10	Interrupción con método “callback”	181
8.10.11	Cambiar el nivel de prioridad de las interrupciones.	181
8.11	Haciendo pausas basadas en SysTick	183
8.11.1	Objetivos	183
8.11.2	Material necesario.	183
8.11.3	Introducción	183
8.11.4	Preparación inicial	184
8.11.5	Tarea: Implementar la biblioteca <code>delay</code>	184
8.11.6	Tarea: Efectos visuales con el LED	184
8.11.7	Tarea: Verificar la temporización.	184
8.11.8	Tarea avanzada: Tareas en segundo plano	184
8.12	MBED: Desarrollo ARM en la nube	185
8.12.1	Objetivos	185
8.12.2	Material necesario.	185
8.12.3	Introducción	185
8.12.4	Descripción del problema	186
8.12.5	Preparación inicial	186
8.12.6	Los proyectos.	188
8.12.7	El compilador	189
8.13	Manipulando osciladores y relojes (STM32L476)	193
8.13.1	Objetivos	193
8.13.2	Material necesario.	193
8.13.3	Introducción	193
8.13.4	Preparación inicial	194
8.13.5	Tarea: Implementar la biblioteca <code>clocks</code>	194
8.13.6	Tarea: Mejora de la biblioteca	195
8.14	Retardos de precisión (STM32L476)	197
8.14.1	Objetivos	197

8.14.2	Material necesario	197
8.14.3	Introducción	197
8.14.4	Preparación inicial	198
8.14.5	Tarea: Implementar la biblioteca delayus	198
8.14.6	Tarea: CutrePWM	198
8.15	Probando PWM sobre un LED (STM32L476)	199
8.15.1	Objetivos	199
8.15.2	Material necesario	199
8.15.3	Introducción	199
8.15.4	Preparación inicial	199
8.15.5	Tarea: Configurar una señal PWM para atacar el LED de la placa	199
8.15.6	Tarea: Cutre-efecto desvanecimiento	200
8.15.7	Reto	200
8.16	Práctica: Un módulo para manejar servos	201
8.16.1	Objetivos	201
8.16.2	Material necesario	201
8.16.3	Introducción	202
8.16.4	Preparación inicial	202
8.16.5	Actividad: desarrollar el módulo servo	202
8.16.6	Actividad: comprobar el funcionamiento del módulo	204
8.16.7	Actividad: comprobar funcionamiento con el servo	204
Soluciones		205
Glosario		205

Capítulo 1

Los microcontroladores

Àngel Perles (aperles@disca.upv.es)

1.1 Introducción

(Al terminar este capítulo, faltaría redactar intro con resumen y objetivos). (Pendent incorporar material llibre 8051).

1.2 Qué es un microcontrolador

Podemos definir un microcontrolador como un computador completo en un chip.

El microcontrolador es la evolución natural del microprocesador. Fue Intel quien en 1.971 diseña el microprocesador para un fabricante japonés de calculadoras, hecho que lleva a una verdadera revolución en la concepción de los dispositivos de uso general e industrial, pues se pasa de la idea de *lógica cableada*, donde se hace un diseño digital no modificable, a la de *lógica programada*, donde el diseño electrónico se combina con software.

La aparición del microprocesador impulsa el diseño de circuitos integrados específicos, consiguiéndose en 1.976 integrar el microprocesador y los chips periféricos, creándose el microcomputador monopaleta. A los microcomputadores monopaleta especializados en aplicaciones industriales se les denominará microcontroladores.

Los microcontroladores agilizan y hacen muy flexible el diseño de sistemas de control, lo cual es una baza fundamental en las características competitivas del mercado actual. Son económicos y sus características se mejoran continuamente.

Usar microcontroladores tiene importantes ventajas, por ejemplo:

- *Reducción del hardware*, en cuanto al tamaño y la cantidad de elementos que forman el circuito electrónico.
- *Disminución de coste* en material, mano de obra y mantenimiento.
- *Facilidad para introducir cambios* o variar características modificando el programa de control.
- *Reducción del tiempo de diseño* al basarse en software.

Y, como todo, tiene sus pequeños “inconvenientes”, pues se necesitan conocimientos de electrónica y de informática para emplearlos.

Volviendo a la idea de que el microcontrolador es un computador completo en un chip, dentro de este chip se tendrán todas las unidades funcionales del computador y una serie de periféricos especializados. La figura 1.1 representa estos bloques y su interconexión.

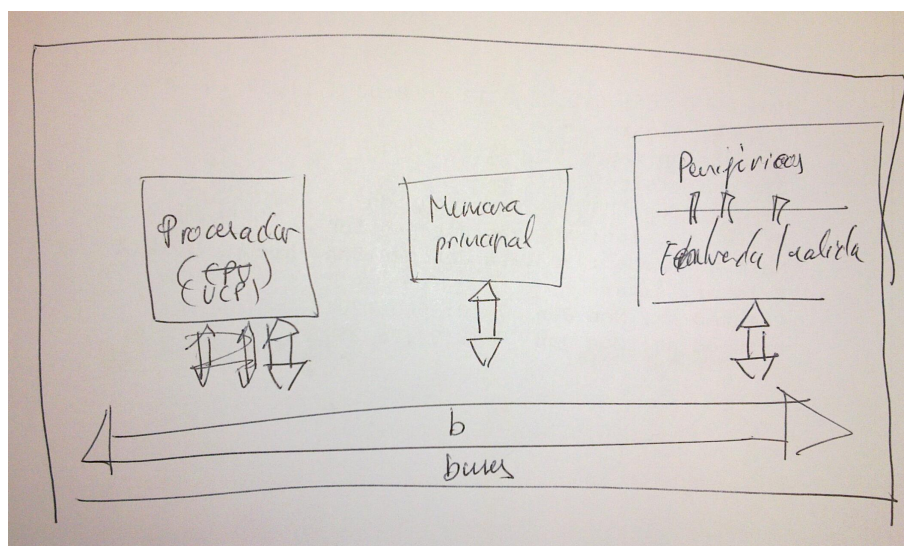


Figura 1.1: Esquema de bloques del microcontrolador

El *procesador*, también conocido como CPU (del inglés, *Central Processing Unit*) es la encargada de ejecutar los programas en forma de instrucciones máquina para procesar datos. Tanto las instrucciones como los datos estarán representados como números digitales binarios.

La *memoria principal* será la encargada de almacenar el programa y los datos. En general, convivirán distintos tipos de memoria dentro del encapsulado del microcontrolador. Por ejemplo, se tendrá memoria ROM (*Read-Only Memory*) para almacenar el programa y las constantes, y memoria RAM (*Random Access Memory*) para contener las variables de la aplicación, que se perderán al dejar de alimentar el chip.

En el subsistemas de *entrada/salida* estarán los distintos periféricos típicos de un microcontrolador: señales de entrada/salida digital, conversores analógico-digitales, temporizadores, etc.

Todos estos elementos se interconectan entre sí mediante líneas eléctricas denominadas *buses* y por las que circulan señales binarias.

1.3 Aplicaciones del microcontrolador

En general, un microcontrolador destaca por la siguientes características:

- Responde rápidamente a eventos. En muchos casos, la ejecución es determinista.
- Son relativamente económicos.
- La ejecución de instrucciones es lenta (pocos millones de instrucciones por segundo).
- Tiene un consumo energético muy bajo.

En contraposición, un microprocesador de propósito general, tiene estas características:

- Es muy lento respondiendo a eventos. Es muy difícil calcular cuánto se tardará en atender a un evento.
- Son caros en comparación con los microcontroladores.
- Ejecutan muchas instrucciones por segundo (cientos o miles de millones de instrucciones por segundo).
- Tiene un consumo energético alto.

Por tanto, el microcontrolador está radicalmente enfocado a resolver problemas muy específicos, pero que se dan en abundancia. Ejemplos típicos de aplicaciones son:

- En electrodoméstico, por ejemplo microondas, calefacciones, frigoríficos, lavadoras inteligentes, cafeteras, básculas, etc.
- En el automóvil, por ejemplo frenos ABS, airbag, control iluminación, cerraduras, climatización, etc.
- En control industrial, por ejemplo autómatas, reguladores, robots, empaquetadoras, etc.
- En informática, por ejemplo impresoras, plotters, teclados inalámbricos, lector código de barras, NFCs.

- En dispositivos móviles, por ejemplo, gestor de sensores, GPS, pulsómetros, relojes inteligentes, etc.
- IoT, wearables, etc., etc.

1.4 Sistemas embebidos, embarcados o empotrados

Todas las aplicaciones anteriores y cualquiera que se planteen tienen en común que el microcontrolador no es más que otro componente electrónico que forma parte de un circuito, y solo tiene sentido formando parte de un todo electrónico o electromecánico. A este tipo de unión se le denomina sistema embebido, embarcado o empotrado, según la manía de cada cual.

Para ilustrar la idea, la figura 1.2 es un ejemplo de sistema embebido. Si se abre este sistema, se verá una placa de circuito impreso con diversos componentes electrónicos, entre los cuales está el microcontrolador.



Figura 1.2: Un conocido dispositivo electrodoméstico y circuito interior.

1.5 Clasificaciones típicas de los microcontroladores

Hay infinidad de modelos de microcontrolador en el mercado, así que se tiende a clasificarlos por tres atributos principales que permiten, a grosso modo, tener una idea de su campo de aplicación. Como es seguro que aparecerán estos términos al analizar las características de un microcontrolador, entonces es el momento idóneo para introducirlas. Esta clasificación de microcontroladores es extensible a cualquier procesador digital.

Sin ser purista en los términos, las tres clasificaciones principales son por el *tamaño de palabra*, por el *tipo de juego de instrucciones máquina* y por el *modo de acceso a los programas y datos*. En cualquier caso, los microcontroladores actuales suelen mezclar internamente varios de estos conceptos, por lo que es muy difícil clasificarlo absolutamente.

1.5.1 Por el tamaño de palabra

Una de las clasificaciones más típicas de los procesadores es el *tamaño de palabra*. A grandes rasgos, el tamaño de palabra define el número de bits de los datos con los que trabaja la CPU. Estos tamaños suelen ser de 4 bits, 8 bits, 16 bits, 32 bits o 64 bits.

Para hacerse una idea del efecto del tamaño de palabra, la figura 1.3 ilustra cómo hace una suma entera un procesador de 8 bits y cómo la hace uno de 32 bits. La máquina de 32 bits es capaz de realizar la operación de golpe, así que esta clasificación se usa para indicar la potencia de cálculo de la CPU, y muchas cosas más.

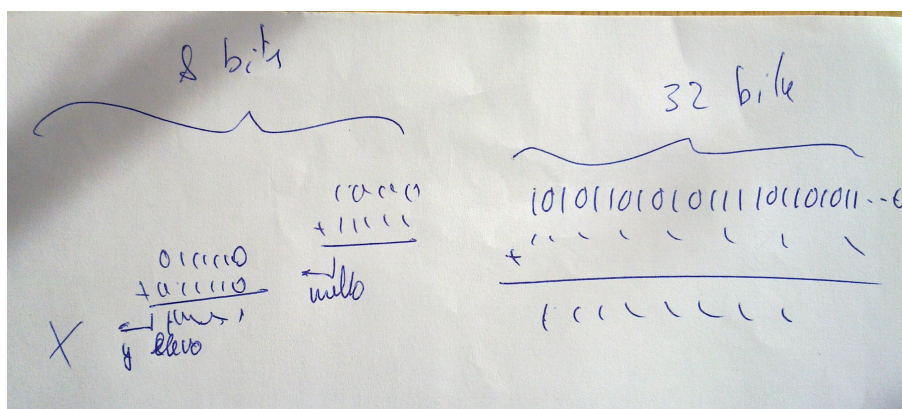


Figura 1.3: Representación del efecto de realizar una suma de 32 bits en un procesador de 8 bits (izquierda) y de 32 bits (derecha)

A primera vista, parecería ideal que todos los microcontroladores fuesen de 64 bits; sin embargo, un mayor tamaño de palabra requiere más líneas eléctricas, más consumo energético, más silicio y más precio, así que es necesario buscar un equilibrio.

Actualmente, el tamaño palabra dominante en el mercado de microcontroladores es de 8 bits; pero está siendo desplazado por el tamaño de 32 bits gracias a nuevos modelos de licenciamiento y a los beneficios competitivos que está aportando al mercado de consumo.

1.5.2 Por el enfoque en la ejecución de instrucciones: CISC o RICS

Al principio de los tiempos de los procesadores, el conjunto de instrucciones máquina estaba orientado a ahorrar espacio en memoria, lo que llevaba a instrucciones capaces de ejecutar acciones muy complejas con el fin de que pocas instrucciones fuesen suficiente para hacer algo interesante.

Con el abaratamiento de las memorias, aparecen nuevos planteamientos en los que se prima una reducción de la complejidad de la CPU que dan lugar a instrucciones más simples que se pueden ejecutar muy rápidamente. En este caso, para realizar una acción equivalente al caso anterior serán necesarias más instrucciones. A esta nueva aproximación se la llama RISC (*Reduced Instruction Set Computer*) y, en contraposición, se acuña el término CISC (*Complex Instruction Set Computer*).

Como ejemplos de 8 bits, la arquitectura Intel 8051 es un ejemplo típico de CISC, mientras que los Microchip PIC son un ejemplo de RISC. Como todo, no se puede decir que una aproximación sea mejor que otra, pues cada una tiene sus ventajas e inconvenientes.

1.5.3 Por el camino usado para los datos y las instrucciones: Von Neuman o Harvard

El procesador ejecuta instrucciones máquina que están en memoria principal, y muchas de esas instrucciones tienen que ver con el tratamiento de datos (sumar, restar, decidir, ...).

La cuestión ahora es dónde está el programa y dónde están los datos. Un planteamiento es entremezclar programa y datos en una misma memoria (bueno, siendo puristas, mismo “camino”) y se habla de una arquitectura *Von Neuman* (un telar, que es la base de los computadores).

Otra aproximación habitual es separar claramente el camino para el programa y el camino para los datos, con lo que se tendrá una arquitectura llamada *Harvard*.

En general, la arquitectura Harvard es más eficiente al tener dos caminos separados que pueden funcionar simultáneamente y que permiten optimizaciones extra. La realidad es que la arquitectura Von Neuman también tiene sus ventajas, así que los procesadores actuales suelen tener una mezcla de las dos aproximaciones.

1.6 Eligiendo el microcontrolador adecuado

Elegir el microcontrolador adecuado es tremendamente difícil debido a la enorme oferta de modelos, arquitecturas y fabricantes. Incluso con experiencia en la materia, es fácil equivocarse en la elección.

La elección depende de innumerables factores, por lo que se reducirá esta sección a ilustrar elementos que puedan orientar a la elección de una determinada arquitectura. En cualquier caso, se listan a continuación algunos de los criterios del autor, que son, por tanto, subjetivos y discutibles:

- Elegir una familia que cubra un rango de problemas lo más amplio posible para que el esfuerzo de aprendizaje sirva para todo el espectro de problemas.
- Elegir soluciones ampliamente aceptadas: más herramientas, más ejemplos, más comunidades.
- No vincularse a un fabricante para tener margen de maniobra en caso de fallo de suministro.
- Asegurarse de la disponibilidad de kits de evaluación de bajo coste. Dada la coyuntura, es bueno tener la posibilidad de probar antes de decidirse
- Primar la facilidad de diseño/elección con respecto al precio del chip. Un chip grande/caro puede resolver distintos proyectos y no hay que empezar de nuevo. El coste de mano de obra es muy importante en los proyectos pequeños.

En definitiva, es importante elegir un *ecosistema* sano y actual como el de la figura 1.4.

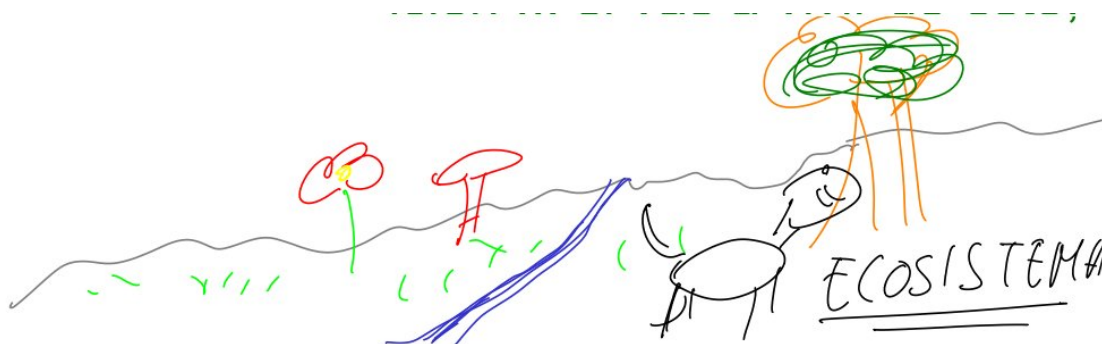


Figura 1.4: Un ecosistema equilibrado con sus plantitas, animalitos y todos felices.

1.6.1 El mercado de microcontroladores

Un buen punto de partida para elegir es conocer cómo está el mercado actual de microcontroladores en base a los estudios de las consultoras.

Para empezar, la 1.5 muestra la evolución del mercado por tamaño de palabra en bits. Se aprecia un claro crecimiento de las arquitecturas de 16 y de 32 bits, y un estancamiento de las de 8 bits. El crecimiento en el segmento 32 bits se debe al lanzamiento de nuevo productos al mercado con grandes exigencias de procesamiento y a la reducción de precios debido a la competencia entre fabricantes de la arquitectura ARM Cortex-M que se presentará más adelante.

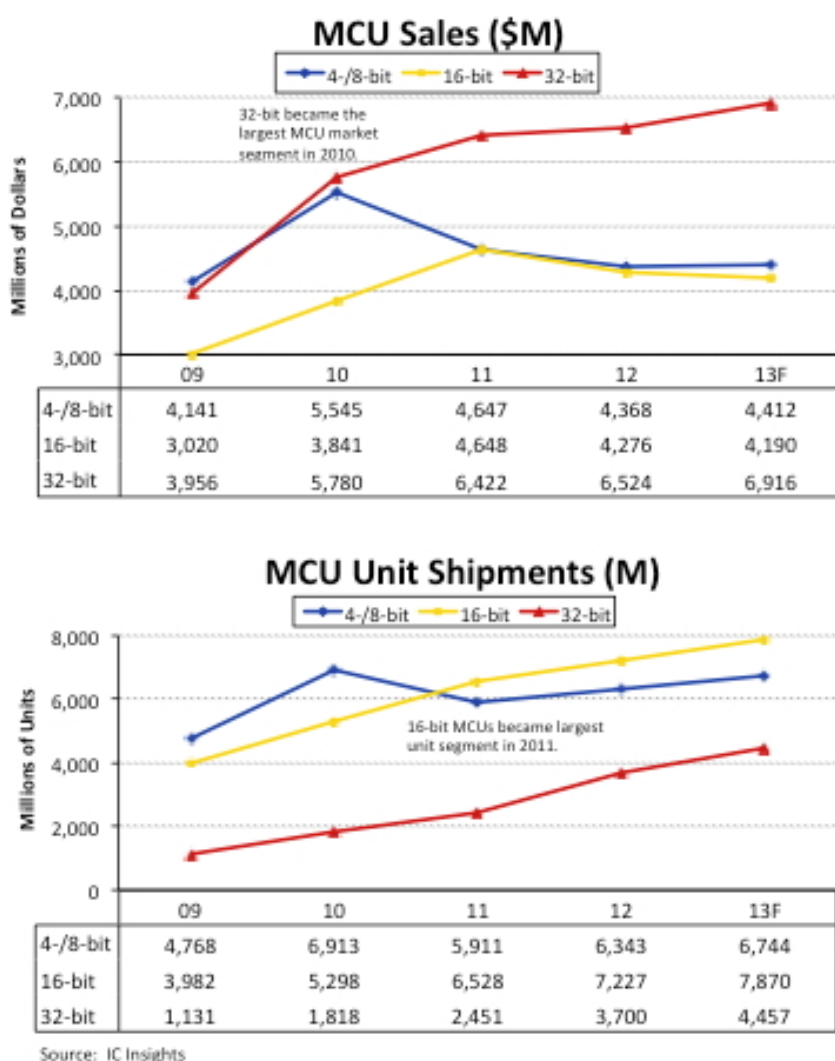


Figura 1.5: Evolución del reparto de microcontroladores por tamaño de palabra (Fte. ICInsigts)

La figura 1.6 representa una estimación del reparto del mercado de microcontroladores por fabricante. Se observa un claro predominio del gigante Renesas como cabe esperar. Indicar aquí que cada fabricante puede proveer distintas arquitecturas de microcontrolador y que hay un fuerte componente geoestratégico. Así, la japonesa Renesas es la principal proveedora del mercado asiático del segmento de automoción (un coche tiene decenas de microcontroladores, por lo que es un segmento muy competitivo).

Company	2011 Rank	2011 \$M	2011 Share	2010 Rank	2010 \$M	2010 Share	Y/Y %
Renesas Electronics	1	2,624	17.30%	1	2,649	17.90%	-1%
Freescale Semiconductor	2	1,538	10.10%	2	1,484	10.00%	4%
Atmel	3	1,114	7.40%	5	895	6.00%	25%
Microchip Technology	4	1,010	6.70%	4	956	6.50%	6%
Infineon Technologies	5	1,007	6.60%	6	857	5.80%	18%
Texas Instruments	6	838	5.50%	7	809	5.50%	4%
Fujitsu	7	834	5.50%	3	1,000	6.80%	-17%
NXP Semiconductors	8	721	4.80%	8	733	5.00%	-2%
STMicroelectronics	9	661	4.40%	9	680	4.60%	-3%
Samsung	10	510	3.40%	10	515	3.50%	-1%
Others		4,298	28.40%		4,222	28.50%	2%
Total		15,155			14,799		2%

Figura 1.6: Reparto del mercado de microcontroladores del año 2011 (Fte. Databeans)

Añadir ahora que las empresas consolidadas son reacias a los cambios, por lo que las oscilaciones en las ventas de un fabricante se deben más a la evolución del mercado donde se implantan sus microcontroladores. Por ejemplo, un fabricante fuertemente orientado al segmento del automóvil, verá reducidas sus ventas debido a crisis en este segmento en su zona geográfica de influencia.

Por otra parte, una nueva empresa que necesite hacer uso de microcontroladores y deba competir en un determinado mercado, tendrá la ventaja de que puede elegir arquitectura. Una empresa ya consolidada tiene mucho más difícil hacer un cambio. Elíjase en consecuencia sin tomar como fundamental el orden mostrado en la figura 1.6.

1.6.2 Familias, fabricantes y licencias

Para entender un poco más la organización del mundo actual de los microcontroladores, es interesante introducir los conceptos de *familia*, *licencia* y *fabricante*.

Una familia es el conjunto de microcontroladores compatibles en cuanto a su *arquitectura*, es decir, el juego de instrucciones máquina, la organización de la memoria, de los periféricos, etc. Dos integrados de una misma familia son compatibles entre sí en muchos aspectos, aunque no lleguen a serlo completamente.

Por otro lado están los fabricantes de microcontroladores, que pueden trabajar de distintas maneras. Por ejemplo, los hay que solo se encargan del diseño y fabricación del chip; otros diseñan y mandan fabricar a otros (en inglés, *fabless*); otros compran la licencia y fabrican el circuito integrado ... en definitiva, todas las posibles combinaciones. Además, es muy habitual que un mismo fabricante produzca distintas familias/arquitecturas.

Y, por último, el término *licencia* que acaba de aparecer se refiere al “permiso” para emplear determinado diseño pagando *royalties*. En este mundo son habituales las empresas de ingeniería que licencian sus diseños a otros.

1.6.3 Elección de la familia de microcontroladores ARM Cortex-M

Al final hay que decidirse por una familia y cargar con las consecuencias, por lo que se recomienda no tomárselo a la ligera y menos aún fiarse de los reclamos publicitarios (ni de lo que se diga aquí).

Dados los criterios subjetivos ya comentados, la actual coyuntura económica y el tipo de proyectos a resolver, el autor considera que la familia de microcontroladores *ARM Cortex-M* de la empresa Advanced RISC Machines Holdings Limited (ARM) [ARM] como el más adecuado y la que mejor expectativas de futuro tienen (y que se han cumplido). ARM solo diseña y licencia a terceras empresas que adquieren estas licencias en forma de *Verilog Intellectual Property* (IP) junto con herramientas adicionales para su explotación.

Por su parte, los clientes tienen derecho a adaptar y complementar los IP para que sean fabricados por el mismo cliente, o encargadas a terceros (*fabless*), o relicenciadas. Al final queda algo como lo mostrado en la figura 1.7 donde hay partes de ARM y partes añadidas que no son comunes entre distintos licenciarios.

Para situar mejor esta familia, es importante destacar que la marca *Cortex* es usada por ARM para una gama muy amplia de procesadores. La figura 1.8 representa gráficamente esta gama.

En la gama ARM hay subfamilias con distinto objetivo, y que son:

- Cortex-A de "Application". Para aplicaciones de alto rendimiento con sistemas operativos generales. Por ejemplo: tablets, e-books, móviles, smart-TV, etc.
- Cortex-R de "Real-time". Para sistemas de tiempo real críticos con buen rendimiento. Por ejemplo, impresoras, control electrónicos de motores, robótica, etc.
- Cortex-M de "Microcontroller". Para aplicaciones típicas de microcontroladores. Por ejemplo: lavadoras, microondas, mandos, nodos inalámbricos, etc.



Figura 1.7: Una implementación ARM da miedo al principio. Hay que partir de la base que las implementaciones reales son la suma de subimplementaciones de distintas fuentes. Cada subimplementación tendrá sus propias especificaciones, sus manuales, etc. A partir de esa idea es más fácil entender el todo

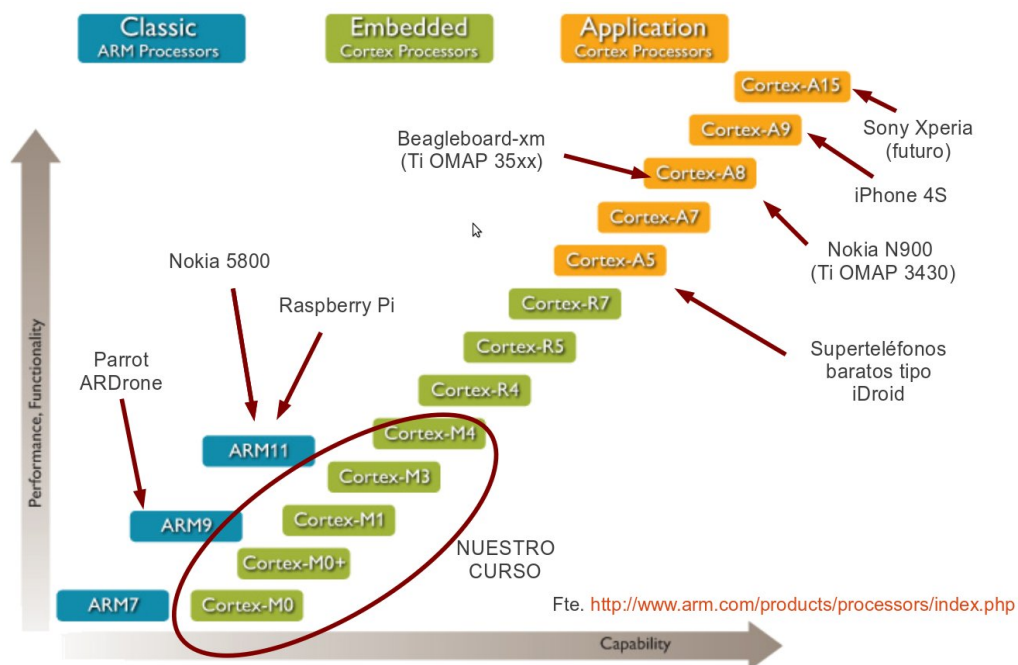


Figura 1.8: Representación de la gama ARM Cortex (derechos de imagen)(ya obsoleta)

Tabla 1.1: Tabla resumen de las versiones de ARM Cortex-M (falta el próximo M5)

Modelo ARM	Juego instrucciones Thumb-2	Multiplicación hardware	División hardware	Extensión DSP	Como flotante	Versión arquitectura	Arquitectura CPU
Cortex-M0	parcial	1 o 32 ciclos	no	no	no	ARMv6-M	Von Neuman
Cortex-M0+	parcial	1 o 32 ciclos	no	no	no	ARMv6-M	Von Neuman
Cortex-M1	parcial	3 o 33 ciclos	no	no	no	ARMv6-M	Von Neuman
Cortex-M3	completo	1 ciclo	sí	no	no	ARMv7-M	Harvard
Cortex-M4	completo	1 ciclo	sí	sí	opcional	ARMv7E-M	Harvard

En realidad la familia Cortex-M está formada por distintas subfamilias que se adaptan a diferentes problemáticas. La tabla 1.1 resume las características de cada subfamilia. No es interesante profundizar aquí en cada variante, destacando simplemente que los modelo M0 son menos potentes y requieren menos silicio para su fabricación y los M4 ofrecen mucho más rendimiento a costa de mayor superficie de silicio.

Este libro trata de la familia ARM Cortex-M, que proporciona beneficios como:

- UNA arquitectura MUCHAS implementaciones. Cada fabricante añade su especialidad: automoción, aeroespacial, bajo consumo, FPGA, ...
- Muchas herramienta hardware y software. Comerciales y libres. Emuladores, compiladores, IDEs, bibliotecas, ...
- Comunidad muy activa: blogs, foros, proyectos (mbed, lpcxpresso, mapple, ...).
- “Starter kits” prácticamente regalados.
- Depuración, depuración, depuración ... esta característica es fundamental para dedicarse profesionalmente a los microcontroladores.

Actividad: Localizar un fabricante de ARM Cortex-M. Una vez localizado, avisa al profesor para que te reserve el fabricante a ti. A continuación, busca uno de los modelos que te interese y anota brevemente las características y el precio por unidad en lotes "bulk"(lotes grandes para fabricación). Presenta al resto de compañeros la página web con el modelo elegido, las características básicas y el precio unitario.

1.6.4 Elección del microcontrolador St STM32F4xxx

Para empezar a trabajar, hay que elegir un chip concreto de entre la amplia disponibilidad de fabricantes y modelos. En este libro se ha elegido el chip STM32F4F407VGT6 de la empresa St por ser el primer integrado que incorporó la arquitectura ARM Cortex-M4. Este chip en particular supuso una revolución en el mercado al introducirse agresivamente mediante un kit de evaluación muy económico: la popular placa *STM32-Discovery*.

Entre las características destacables de este microcontrolador, indicar que tiene unidad de coma flotante, tiene un rendimiento de 210 DMIPS (168 MHz de reloj), 1 MB de memoria Flash ROM, 196 KB de memoria RAM, USB OTG HS/FS, Ethernet, 17 TIMERS, 3 ADCs, 2 DACs, etc.

Quizá este microcontrolador tan potente sea excesivo para un libro de iniciación, pero deja la cancha libre para cualquier aplicación imaginable. En cualquier caso, se hubiese podido elegir perfectamente un ARM Cortex-M de otro fabricante (Texas Instruments, NXP, Freescale, Silabs y un laaaargo etc.) y se hubiese estado en una situación similar.

1.7 Las herramientas

Tal como se indicó en la sección 1.6.3, es necesario disponer de herramientas software y hardware para desarrollar sistemas embebidos con microcontroladores.

Se introducirán ahora las principales herramientas con las que se puede contar para los desarrollos.

1.7.1 El lenguaje de programación

El software para el microcontrolador se puede desarrollar empleando algunos de los lenguajes de programación más habituales. Por ejemplo, Ensamblador, C/C++, ADA, Java, Python, Pascal, Forth, Basic, etc.

Los dos lenguajes predominantes con diferencia en el ámbito de los microcontroladores con el lenguaje ensamblador y el lenguaje C.

El lenguaje ensamblador (o código máquina) es un lenguaje muy cercano a la arquitectura basado en órdenes de bajo nivel (suma, resta, salta si, ...). Este lenguaje es particular a cada arquitectura de microcontrolador, y requiere una destreza y curva de aprendizaje muy larga, por lo que no es recomendable en la mayoría de situaciones.

El lenguaje C es, con mucho, el más utilizado en el desarrollo para microcontroladores por la amplia disponibilidad de herramientas y por la facilidad de acceso a las

características de bajo nivel. El lenguaje C permite desarrollar aplicaciones con el rendimiento de ensamblador/código máquina y con las ventajas de ser portable y de alto nivel.

1.7.2 Sistemas operativos o microkernels

Un microcontrolador es un dispositivo totalmente vacío de software, siendo el desarrollador el responsable de “llenarlo” con código. En contraposición, un ordenador personal típico lleva un sistema operativo que soluciona y oculta la problemática de gestionar los recursos de manera supersencilla, por lo que desarrollar aplicaciones es mucho más sencillo.

Dada la creciente complejidad en los desarrollos con microcontrolador, se puede recurrir a un sistema operativo para microcontrolador o *microkernel*, que simplifica enormemente de aplicaciones y que no es más que código añadido que proporciona los servicios de más bajo nivel de un SO, por ejemplo, multitarea, sincronización, regiones críticas, semáforos, buffers, etc.

1.7.3 Simuladores

En el contexto de los microcontroladores, un simulador es software que se ejecuta en un ordenador personal e imita el funcionamiento del microcontrolador, tanto a nivel de ejecución de aplicaciones como de periféricos.

Un simulador es adecuado en las etapas iniciales de aprendizaje o en entornos en que se integre el microcontrolador simulado junto a otros elementos simulador (circuitos electrónicos, señales, etc.).

Dada la relativa novedad de los microcontroladores ARM Cortex-M, los simuladores disponibles para esta arquitectura suelen ser de los primeros modelos. Por ejemplo, la figura 1.9 muestra un ejemplo del entorno de desarrollo electrónico Proteus simulando un circuito con un ARM Cortex-M3 de Texas Instruments.

1.7.4 Sondas de depuración

Los modernos microcontroladores incluyen internamente mecanismos que permiten la depuración del código en forma de ejecución paso-a-paso, trazas, etc. Estas características son de enorme interés para los desarrolladores profesionales.

Para acceder a estos servicios del chip es necesario emplear *sondas de depuración* que emplean protocolos tipo JTAG o SWD para descargar la aplicación en el microcontrolador, controlar su ejecución y extraer la información de interés. La figura 1.10 muestra el aspecto de una de estas sondas.

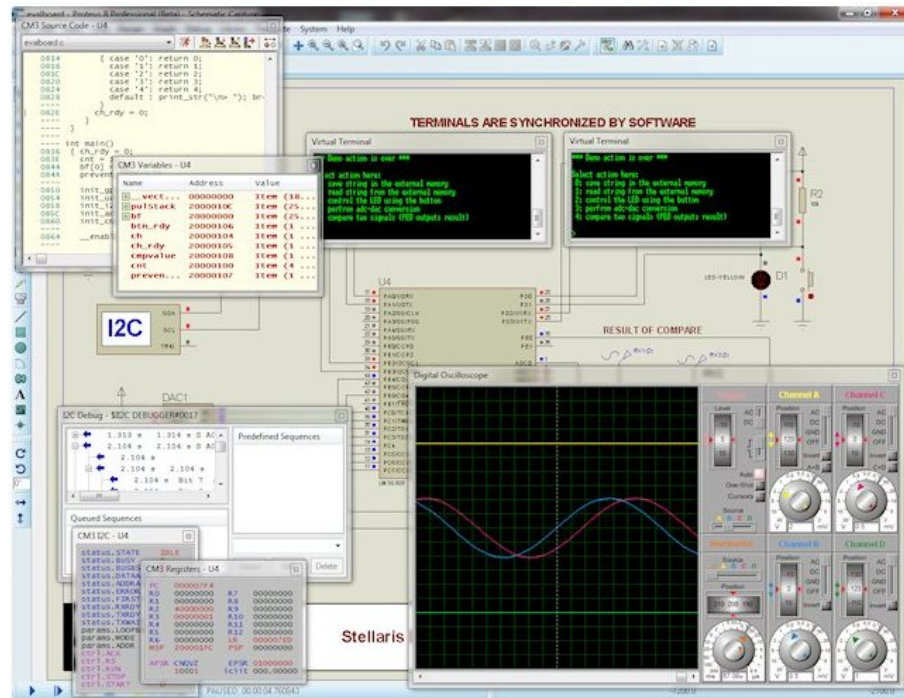


Figura 1.9: ISIS-Proteus simulando un circuito con ARM Cortex-M3.



Figura 1.10: Sonda de depuración profesional Segger J-Trace.

1.7.5 Kits de desarrollo/evaluación

En las etapas iniciales de un desarrollo con microcontrolador se suele recurrir a los llamados kits de desarrollo como el mostrado en la figura 1.11 que incluye gran cantidad de dispositivos típicos. Para usar los kits más profesionales suele ser necesario añadir una sonda de depuración. Estos kits tienen precios alrededor de la centena de Euros.



Figura 1.11: Sonda de depuración profesional Segger J-Trace.

Los fabricantes de microcontroladores suelen ofrecer kits de evaluación limitados a unos precios muy competitivos (por debajo de coste) con el objetivo de promocionar sus productos. Estos kits son ideales para empezar a trabajar inmediatamente, pues suelen incluir depuradores básicos y algunos periféricos. El precio oscila alrededor de la decena de euros. De la fig a la fig...

Figura 1.12: Aquí un kinetis, TI LAUNCHPAD, ...

Figura 1.13: Aquí un kinetis, TI LAUNCHPAD, ...

Finalmente, en los últimos años ha habido un espectacular crecimiento en la oferta de kits para la comunidad DIY (*Do-it-Yourself*) con soluciones interesantes a precios competitivos. De la fig a la fig ... ejemplos.

Figura 1.14: Aquí un mbed, Arduino DUE, Olimex, ...

Figura 1.15: Aquí un mbed, Arduino DUE, Olimex, ...

Capítulo 2

Arquitectura del microcontrolador St STM32F4xxx (ARM Cortex-M4F)

2.1 Introducción

(Al terminar este capítulo, faltaría redactar intro con resumen y objetivos). (enfocar de lo particular St a lo general M4F)

2.2 Familia St STM32F4

Entre otras familias de microcontroladores, la empresa *St Microelectronics* produce una amplia gama de microcontroladores basados en la arquitectura ARM Cortex-M. En la figura 2.1 se representa esta gama cuya nomenclatura incluye el término STM32 para representar la arquitectura ARM Cortex-M y una sufijo para la gama. Por ejemplo, un microcontrolador STM32F3 es un Cortex-M4 con unidad de coma flotante y un STM32L0 es un Cortex-M0+.

Dentro de cada gama hay variantes que cubren precios y necesidades distintas. Por ejemplo, la figura 2.2 resume las características destacables de la gama STM32F4, que se diferencian en el equilibrio precio-prestaciones-consumo.

En este libro se elige un miembro de la serie STM32F4 por cubrir todo el rango de posibilidades de los microcontroladores ARM Cortex-M, proporcionando cancha más que suficiente para diseños complejos. Aquí destacar la política de St de procurar mantener la compatibilidad pin-a-pin entre miembros, con lo que suele facilitarse el sustituir un modelo por otro con mínimos cambios, pero a costa de falta de flexibilidad.

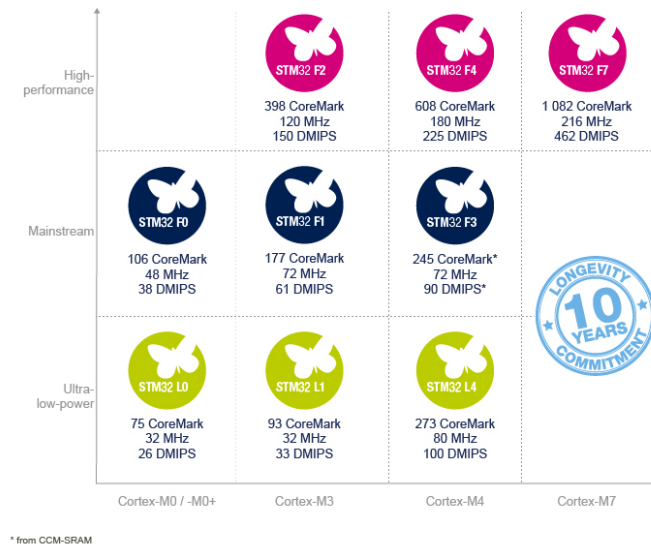


Figura 2.1: Gama de microcontroladores STM32 de St (Fte. St)

ARM® Cortex®-M4 (DSP + FPU) – Up to 180 MHz • ART Accelerator™ enabling 0 wait state executing from internal Flash • Up to 2x USB2.0 OTG FS/HS • SDIO • USART, SPI, I2C • PS + audio PLL • 16 and 32-bit timers • Up to 3x 12-bit ADC (0.41 µs) • Up to 2x 12-bit DAC • External memory controller (except for access lines) • Low voltage 1.7¹ to 3.6 V	Lines		F _{cpu} (MHz)	Flash (bytes)	RAM (KB)	Ethernet I/F IEEE 1588	Camera I/F	SDRAM I/F	SAP I/F	Chrom-ART Graphic Accelerator™	TFT LCD controller	MPI DSI
		Product										
	Advanced	STM32F469 ²	180	512 K to 2 M	384	•	•	•	•	•	•	•
		STM32F429 ²	180	512 K to 2 M	256	•	•	•	•	•	•	•
		STM32F427 ²	180	1 to 2 M	256	•	•	•	•	•	•	•
	Foundation	STM32F446	180	256 K to 512 K	128	•	•	•	•	•	•	•
		STM32F407 ²	168	512 K to 1 M	192	•	•	•	•	•	•	•
		STM32F405 ²	168	512 K to 1 M	192	•	•	•	•	•	•	•
	Access	Line	STM32F4 Access line	F _{cpu} (MHz)	Flash (KB)	RAM (KB)	RUN current (µA/MHz)	STOP current (µA)	Small package (mm)	BAM (Batch Acquisition Mode)		
			STM32F411	100	256 to 512	128	Down to 100	Down to 12	Down to 3.034x3.22	•		
			STM32F410	100	64 to 128	32	Down to 89	Down to 8	Down to 2.553x2.579	•		
			STM32F401	84	128 to 512	96	Down to 128	Down to 10	Down to 3x3			

Notes: 1/ 1.7 V min on specific packages
2/ The same devices are also found with embedded Hardware crypto/hash
3/ Serial Audio I/F

Figura 2.2: Miembros de la gama STM32F4 (Fte. St)

Los microcontroladores STM32F4x usan la arquitectura ARM Cortex-M4 y tiene las siguientes características principales:

- Depuración avanzada típica del los ARM Cortex-M4.
- Alto rendimiento: hasta 168 MHz de reloj + instrucciones DSP + unidad de coma flotante (FPU - Floating Point Unit) en hardware.
- Mucha memoria: hasta 1 Mbyte Flash y hasta 196 Kbytes RAM en el chip y controlador de memoria externa.
- Dominio VBAT: zona mantenida con batería externa con 4 Kbytes de memoria y reloj de tiempo real (RTC).
- Hasta 140 pines de E/S digital. Algunos tolerantes a 5 volts.
- Convertidores AD (3 de 12 bits) y convertidores DA (2 de 12 bits).
- Controladores de acceso directo a memoria (DMA).
- Hasta 17 contadores/temporizadores de 16 o 32 bits. PWM, cuadratura encoder, etc.
- Hasta 15 interfaces de comunicación típicos: I2C, USART, SPI, CAN, SDIO ...
- Interfaces USB host/slave tipo HS y FS, y OTG (on-the-go) 2.0.
- Interfaz ethernet con soporte hardware IEEE 1588v2, MII / RMII.
- Interfaz para cámara y para LCD.
- Generador de números aleatorios verdaderos. Calculador CRC. Generador criptográfico.
- Etc., etc. etc.

2.3 Arquitectura del microcontrolador STM32F407

Los microcontroladores de la arquitectura ARM Cortex-M tienen una arquitectura relativamente compleja en comparación con los microcontroladores clásicos de 8 bits (8051, PIC16, AVR, etc.), pues incorporan mecanismos avanzados más habituales en procesadores de aplicación general.

Explicar la arquitectura ARM Cortex-M daría para muchos capítulos y dificultaría el propósito de este libro, por tanto se recurrirá dar una visión muy simplista de esta arquitectura en los aspectos que sean relevantes para entender la filosofía Cortex-M y su aplicación práctica. En caso de querer profundizar más, se recomienda [Yiu2014].

En la figura 2.3 se tiene el diagrama de bloques de un microcontrolador STM32F429 que, como se ha indicado, emplea la arquitectura ARM Cortex-M4. Se observa en la zona superior-izquierda el *core* ARCore-M4, la unidad de depuración y una serie de buses que conectan esas zonas. Son estos elementos los que proporciona la empresa ARM, el resto los añade St, pudiendo ser propios o licencia de terceros.

En la notación usada por ARM abunda las siglas, lo que dificulta la comprensión de cada elemento. Se ha optado por añadir un glosario de estas abreviaturas para que sea más fácil seguir los diagramas.

Los distintos elementos están interconectados entre sí mediante buses principales y ramificaciones de estos buses. Hay una matriz de conmutación de buses que conecta la CPU, el sistema de depuración y los buses principales AHB y, usando un símil, podría asimilarse a un cruce con semáforos. Esta es la zona con tráfico de mayor velocidad.

Las memorias están repartidas en trozos para facilitar el funcionamiento en paralelo de los distintos módulos. Así es posible estar ejecutando código y, a la vez, ir trayendo datos a la memoria.

Los periféricos están colgados de distintos buses de manera que se reparte el ancho de banda entre distintos grupos de periféricos, disminuyéndose el peligro de bajo rendimiento gracias a este reparto.

En cualquier caso, la grana cantidad de elementos, la alta velocidad y las relaciones entre elementos hacen que se puedan producir cuellos de botella en las zonas de buses.

Y no sé si vale la pena indicar nada más.

2.4 Encapsulado y patillaje de un St STM32F4

Los microcontroladores STM32 se producen en distintos formatos de encapsulado. Como ejemplo, un microcontrolador cuyo encapsulado es de tipo *LQFP100* (*Low-profile Quad Flat Package* de 100 pines) se representa en la figura 2.4.

En la figura 2.5 se representa la identificación de cada uno de los pines.

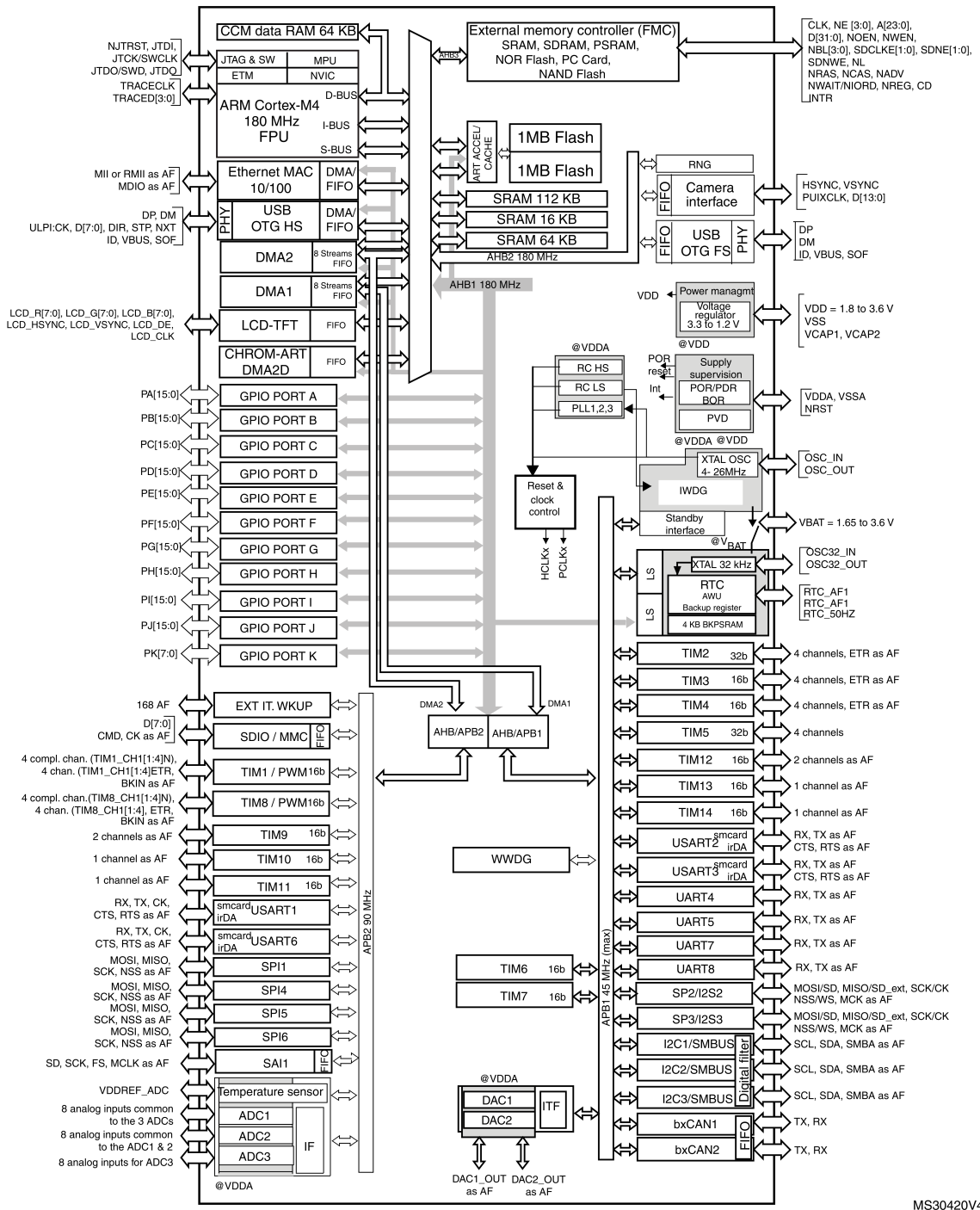


Figura 2.3: Arquitectura del microcontrolador STM32F429

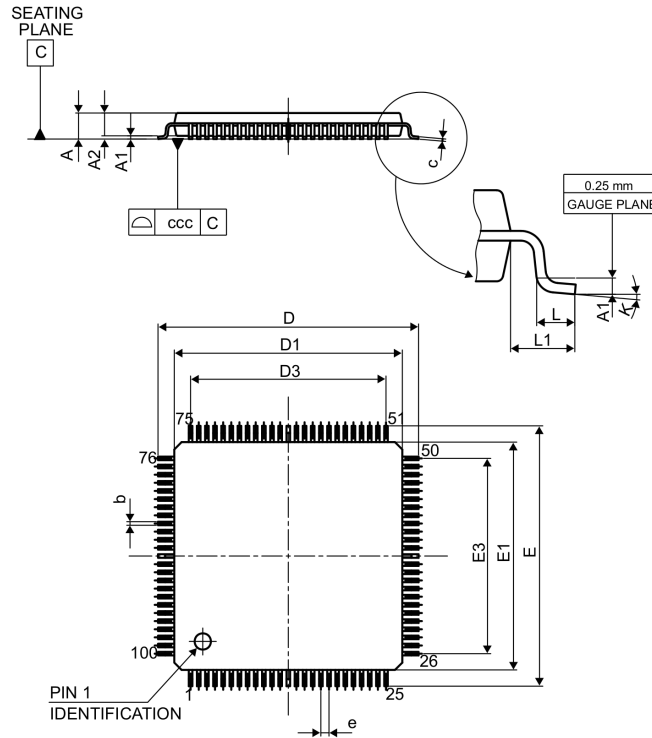


Figura 2.4: Versión con encapsulado LQFP100 de un microcontrolador STM32F4x.

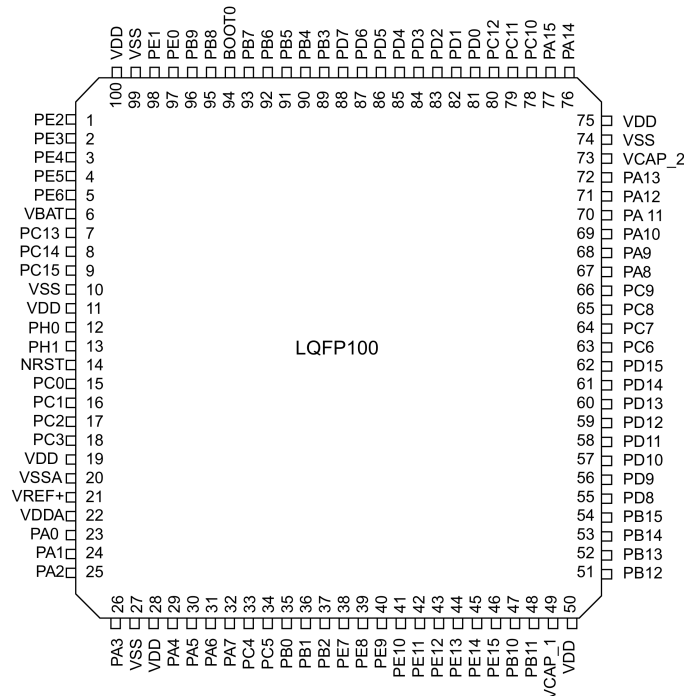


Figura 2.5: Patillaje de un microcontrolador STM32F4x en la versión LQFP100

2.5 Sistemas con un STM32F4

Internamente, el microcontrolador es complejo, pero no será más que un chip al que hay que proporcionar alimentación, un reloj que marcará el ritmo al que funcionan las cosas y una señal de *reset* o reinicio para que el micro empiece a trabajar en un estado concreto. Este chip se incorporará en un PCB como el de la figura 2.6 para formar un sistema embebido .

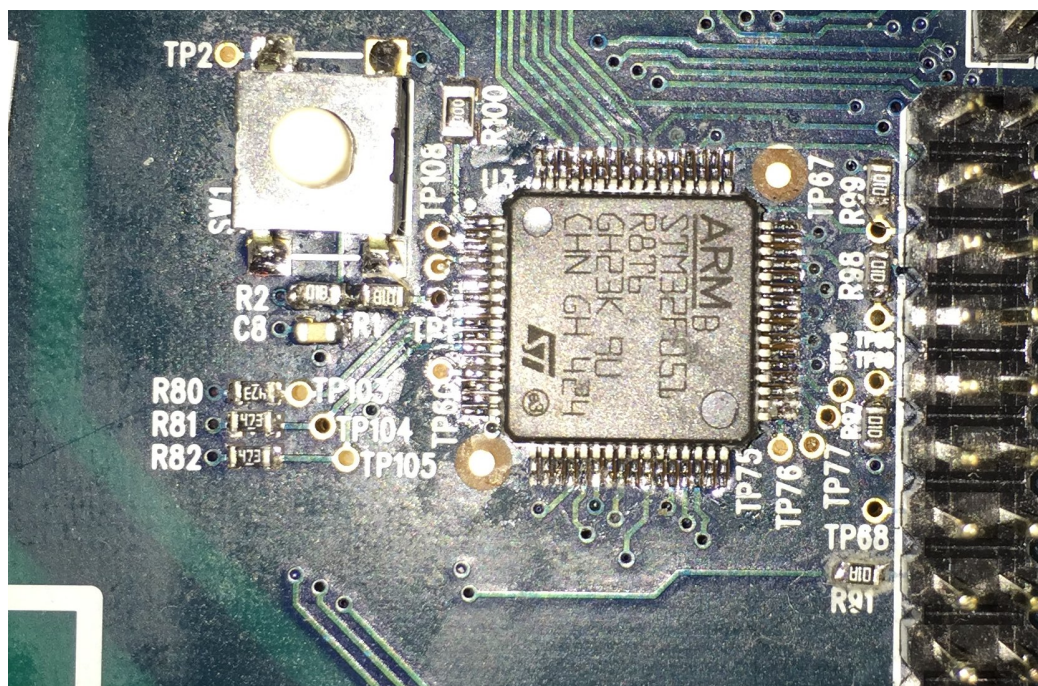


Figura 2.6: Equipo comercial desarrollado en la UPV que incorpora un microcontrolador de la gama St STM32

2.5.1 Alimentación, reloj y reset

La alimentación

Los primeros microcontroladores solían emplear tensiones TTL (5 volt.), conectándose el terminal “VSS” a masa y el terminal “VCC” o “VDD” a +5 voltios. Hoy en día, los micros y los dispositivos analógicos y digitales se suelen alimentar a tensiones inferiores, pues menor tension implica menor consumo. En las últimas hay que tener especial precaución con las tensiones negativas en cualquiera de los pines, pudiéndose proteger con diodos.

Algunos uC admiten rangos de tension amplios por el tipo de tecnología empleado, o por la inclusión de reguladores de tension internos.

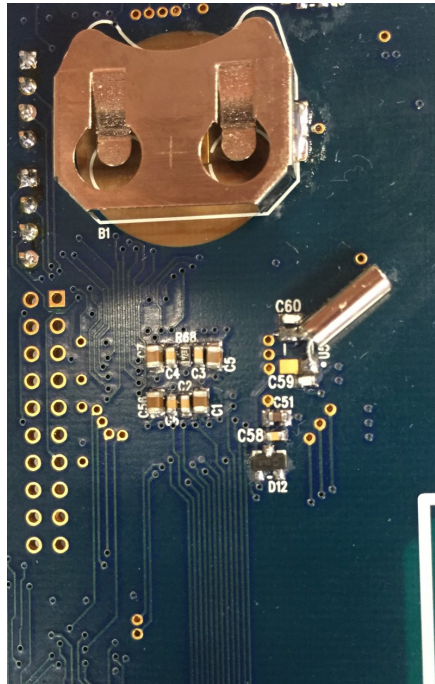


Figura 2.7: Equipo comercial desarrollado en la UPV que incorpora un microcontrolador de la gama St STM32 (bottom)

Para el correcto funcionamiento del uC es fundamental desacoplar la alimentación mediante condensadores en los pines de alimentación. Esto evitará la caída de tensión debida las conmutaciones internas en el uC y a que lleguen estos mismos efectos de otros chips (que también deberán estar desacoplados).

Un buen microcontrolador del que nos podamos fiar debería incorporar mecanismos de supervisión de la alimentación para detectar condiciones anómalas (caídas de tensión, microcortes, etc.) que reinicien el micro para llevarlo a un estado seguro. De no hacerlo, un micro puede quedar en un estado inestable, vulgarmente “colgado”.

El reloj

Un uC es, básicamente, un autómatas que evoluciona al ritmo de una señal cuadrada o “reloj”. Para tener reloj, la mayoría de los uC modernos suelen incorporar uno o varios osciladores internos y/o la electrónica parcial para tener parte del oscilador externamente mediante redes tipo R-C o cristales de cuarzo-cerámicos-sintéticos.

Es habitual que coexistan varios osciladores simultáneos, por ejemplo, un oscilador para la CPU y un oscilador para el reloj de tiempo real con un cristal de 32 kHz. O también un oscilador de baja precisión interno (alrededor del 2%) y uno de buena exactitud externo con cristal de cuarzo.

Desde el punto de vista energético, a mayor velocidad y mayor precisión habrá mayor consumo y se requerirá más tiempo para despertar el oscilador (la puesta en marcha), mientras que relojes de baja velocidad y poca precisión incorporados requerirán menos energía y permiten un despertar rápido.

El reinicio/reset

Para poner el microcontrolador en su estado de partida inicial es necesario proporcionar la llamada señal de *reset* o reinicio.

Habitualmente, esta señal se realizaba mediante un condensador y una resistencia externa que fijaban a un determinado nivel un pin cuando se producía alimentación.

Los modernos microcontroladores no suelen necesitar nada, pues incorporan unidades internas que monitorizan la tensión de alimentación (POR-power-on-reset) y llegan más allá monitorizando el funcionamiento de osciladores, memoria flash, etc. para generar la señal de reset en caso de detectar anomalías.

2.5.2 Ejemplos

2.5.3 Mínimo, mínimo

Que ellos lo diseñen rayando la hoja en la que se han impreso en patillaje.

2.5.4 Razonable

Oscilador externo, reset de usuario, ????. Que lo diseñen ellos.

2.6 ¿Más adelante?

Demasiado avanzado para este año

2.6.1 Mapa de memoria

2.6.2 Arranque

Posición inicial, ajuste de stack, salto a *inii* y a *main*. Vectores.

Capítulo 3

Entrada/salida digital

3.1 Introducción

La figura 3.1 representa la idea de “entrada” y de “salida”, donde se toma siempre como referencia al microcontrolador.

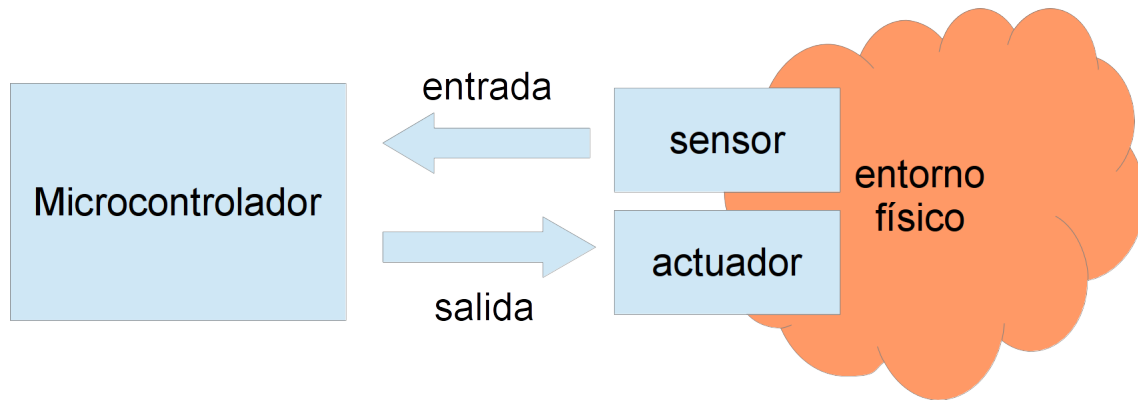


Figura 3.1: Concepto de entrada y salida.

En particular, la entrada/salida digital es un mecanismo básico de cualquier microcontrolador que permite generar y leer señales en dos posibles estados, que serán equivalentes a un “1” o a un “0” en la parte software.

En el más simple de los casos, será una única señal (abierto/cerrado, alto/bajo, hay/no-hay, etc.) que se representará en un solo bit; y, en otros casos, se empleará un conjunto discreto de valores que se representará como números binarios y, por tanto, requerirá más señales simultáneas.

El uso de este mecanismo a bajo nivel implica el acceso a los registros del hardware, lo que es complicado en un libro de iniciación al uso de microcontroladores ARM Cortex-M. Para simplificarlo, se comenzará con un enfoque basado en usar las bibliotecas derivadas de CMSIS que proporciona St. Estas bibliotecas crean una abstracción de los periféricos.

Al subsistema encargado de la generación/lectura de señales digitales se le denomina *General purpose input-output* (GPIO).

(Al terminar este capítulo, faltaría redactar intro con resumen y objetivos). (Pendent incorporar material llibre 8051).

3.2 Puertos y líneas

Los señales digitales de un microcontrolador cualquiera se organizan en puertos, que son agrupaciones de 8, 16 o 32 líneas en función del modelo de microcontrolador. En la familia STM32, las agrupaciones son de 16 líneas que se corresponden internamente a una palabra binaria de 16 bits. En la figura 3.2 se puede observar un ejemplo; los puertos se numeran con letras (A, B, C, D, ...) y la líneas con un número entre 0 y el número de líneas/bits del puerto.

Insistir en la idea de que esta notación es particular de cada fabricante. Por ejemplo, un ARM Cortex-M de NXP (los LPC), utilizan puertos de 32 bits y la notación es distinta.

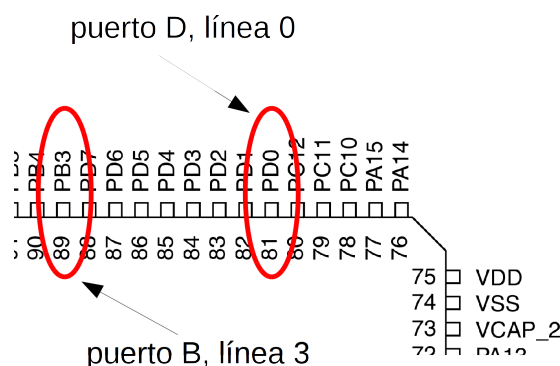


Figura 3.2: Puertos y líneas en el encapsulado LQFP100 de un STM32F4x

3.3 La célula de cada pin

Cada pin de un STM32F tiene una célula general similar a la mostrada en la figura 3.3. En la parte de la izquierda se tienen los registros a los que accede el software para gestionar la célula, y en el extremo de la derecha se tiene el pin físico del encapsulado.

La célula está protegida (hasta cierto punto) con diodos que evitan tensiones negativas y tensiones superiores a V_{DD_FT} , que en unos pines es la tensión de alimentación del chip y en otros es una tensión TTL (5 voltios). Estos últimos pines hacen al integrado compatible con señales clásicas TTL.

Para poder forzar niveles de tensión altos o bajos, se incorporan resistencias de *pull-up* para “estirar hacia arriba” y de *pull-down* para “estirar hacia abajo”. Estas resistencias son desconectables, y su valor debe comprobarse en las hojas de características del chip que se elija. Por ejemplo, en el chip STM32F407VG es de unos 40 $k\Omega$ según la hoja de características [STM32F407-datasheet].

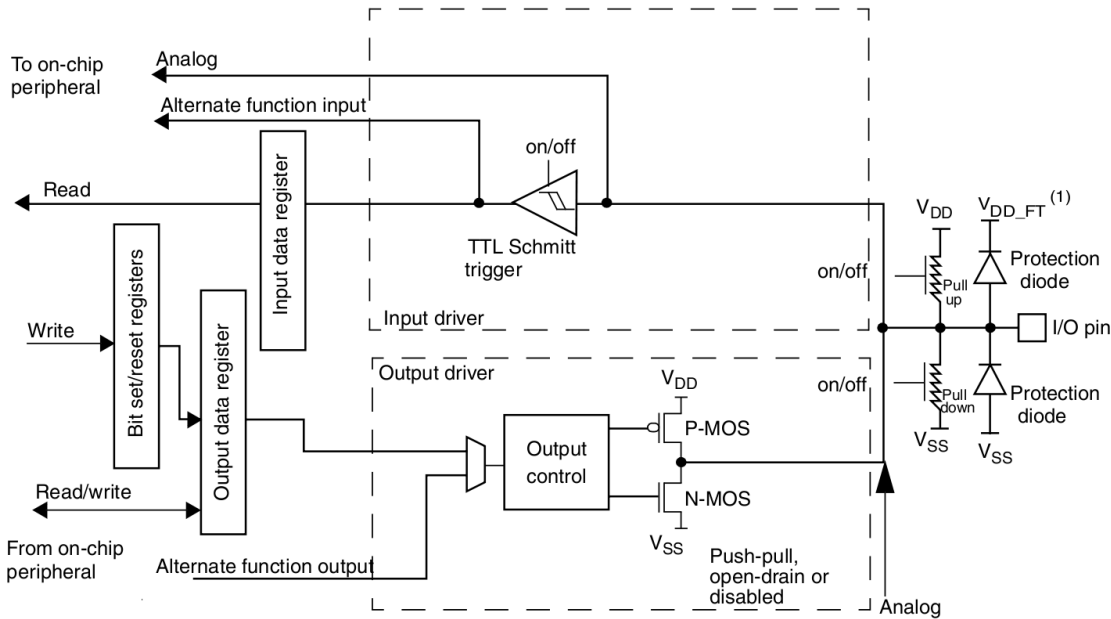


Figura 3.3: Célula interna de cada pin de puerto de un STM32F

Dentro de la célula hay un driver de entrada digital, un driver de salida digital, una posible conexión a entrada/salida analógica y una posible conexión a una función alternativa digital.

3.4 Salida digital

3.4.1 La célula en modo salida

La figura 3.4 representa la célula del pin configurada como salida.

La célula se puede gestionar de muchas maneras que se describen en el manual de referencia correspondiente, por ejemplo, para el STM32F4x están disponibles en [STM32F4-reference-manual]. Básicamente, el driver de salida se puede configurar en modo *push-pull* (“empujar-estirar”) u *open-drain* (“drenador abierto”). Para entender estas configuraciones, la figura 3.5 muestra el equivalente. En función de la configuración, se dispondrá de más o menos corriente para absorber-emitir, pero siempre será de unos pocos miliamperios que dependerán del puerto elegido y del modelo concreto de microcontrolador, debiéndose comprobar en las hojas de características.

Las distintas configuraciones permiten adaptarse a gran variedad de aplicaciones, pudiéndose complementar con distintas configuraciones de las resistencias de *pull-up* y *pull-down*.

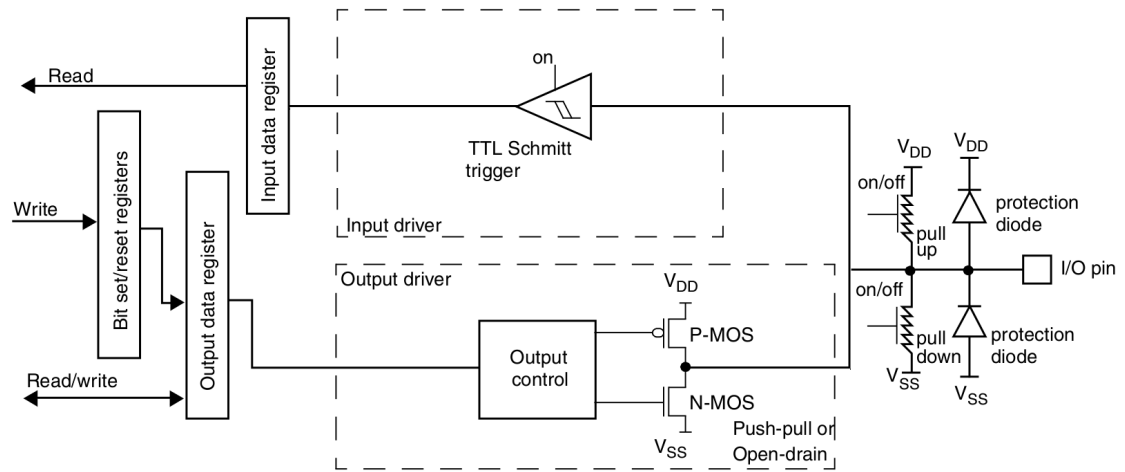


Figura 3.4: Configuración de la célula en modo salida.

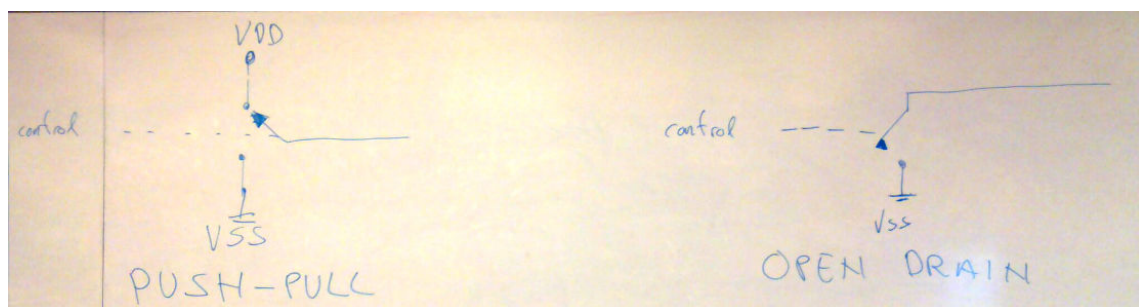


Figura 3.5: Equivalente de las configuraciones push-pull y open-drain.

Tabla 3.1: Funciones de preparación de la GPIO.

<code>void HAL_GPIO_Init (GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)</code> Initializes the GPIOx peripheral according to the specified parameters in the <code>GPIO_Init</code> .
<code>void HAL_GPIO_DeInit (GPIO_TypeDef *GPIOx, uint32_t GPIO_Pin)</code> De-initializes the GPIOx peripheral registers to their default reset values.

Para conocer el estado real de las líneas, el subsistema de entrada digital también sigue activo.

3.5 Biblioteca HAL para gestionar la GPIO

La biblioteca HAL STM32Cube proporciona funciones simples para gestionar el periférico GPIO. La tabla 3.1 contiene funciones específicas para preparar una estructura de datos que se usará después para configurar los pines físicos. Se recomienda acudir al manual incluido en las propias bibliotecas para una referencia completa sobre estas funciones (también hay una versión on-line en la web del libro [[librostm32](#)]).

Para leer/escribir el valor de los pines se pueden emplear las funciones contenidas en la tabla 3.2.

Tabla 3.2: Funciones de lectura/escritura de los pines GPIO.

<code>void HAL_GPIO_WritePin (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)</code> Activa (1) o desactiva (0) los pines (bits) seleccionados del puerto. Se puede acceder a más de un pin. Por ejemplo: <code>HAL_GPIO_WritePin (GPIOA, GPIO_PIN_2 GPIO_PIN_1, GPIO_PIN_SET);</code> <code>HAL_GPIO_WritePin (GPIOC, 0x00FF, GPIO_PIN_RESET);</code>
<code>void HAL_GPIO_TogglePin (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)</code> Conmuta (1->0, 0->1) los pines (bits) GPIO especificados. Por ejemplo: <code>HAL_GPIO_TogglePin (GPIOA, GPIO_PIN3);</code>
<code>GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)</code> Lee el pin del puerto de entrada especificado. Solo es accesible un pin con esta función. Por ejemplo: <code>uint8_t pin_state;</code> <code>pin_state = HAL_GPIO_ReadPin (GPIOF, GPIO_PIN_4);</code>

Para un primer acercamiento a estas funciones, se propone un ejemplo básico que enciende y apaga un LED. Sea el siguiente archivo de cabecera que proporciona los prototipos deseados:

```
/**
 * @file led.h
 * @brief Ejemplo basico para explicar bibliotecas. Ahora con hardware
 * @author Angel Perles
 * @date 2015/03/01
 */
```

```
#ifndef LED_H
#define LED_H

void LED_Init(void);
void LED_On(void);
void LED_Off(void);

#endif
```

Supóngase ahora el LED conectado al pin PG13 según el esquemático de la figura 3.6. Dicho LED requeriría una configuración de tipo *push-pull* para funcionar. Respecto a las resistencias, no tiene sentido que estén activadas.

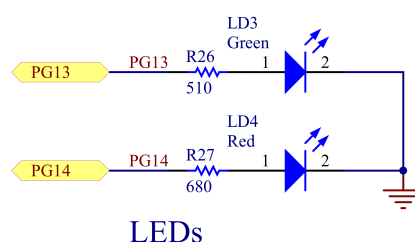


Figura 3.6: Esquemático de los LED en la STM32F429Discovery.

Para implementar esta funcionalidad, el primer paso será inicializar el hardware tal como se ilustra en la implementación de `LED_Init()`. Básicamente se proporciona primero reloj al periférico para activarlo., a continuación se prepara una estructura de datos específica para cada periférico y, finalmente, se aplica la estructura a periférico en sí.

Una vez inicializado el periférico, se puede acceder a sus características seleccionando las funciones del HAL o accediendo a bajo nivel al propio dispositivo. En el listado se ilustra la manera de hacerlo a alto nivel mediante las HAL.

```
/**
 * @file led.c
 * @brief Basic LED handling for libraries example
 * @author Angel Perles
 * @date 2016/02/11
 */

#include "stm32f4xx_hal.h" // cabeceras proporcionadas por St para simplificar el uso de los peri
#include "led.h"

/*****
 **
 * @brief Preparing pin corresponding to LED green (PG13)
 * @return none
 */
void LED_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct; // estructura donde se pone la configuracion deseada

    __HAL_RCC_GPIOG_CLK_ENABLE(); // darle reloj al periferico, AHORA VIVE!
```

```

/* Configure the GPIO_LED pin */
GPIO_InitStruct.Pin = GPIO_PIN_13;                // pin que desamos configurar
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // lo vamos a usar como salida en push-pull
GPIO_InitStruct.Pull = GPIO_NOPULL;             // desactivar pulls
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;     // actualizacion pin

HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);          // hacer efectiva configuracion puerto
}

/*****
**
* @brief  Encender el LED
* @return none
*/
void LED_On(void)
{
    HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET); // poner a "1" la linea PG13
}

/*****
**
* @brief  Apagar el LED
* @return none
*/
void LED_Off(void)
{
    HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET); // poner a "0" la linea PG13
}

/** End of file *****/

```

Entrar en el manual del HAL para localizar la descripción de la estructura

Actividad: de datos específica para la GPIO y de los posibles valores de cada campo.

Si no se sabe nada de estructuras, repasar en un libro de C.

Dado que es el primer enfrentamiento con el plateamiento de las HAL, se añade a continuación la definición de la estructura `GPIO_InitTypeDef`.

```

typedef struct
{
    uint32_t Pin;           // Specifies the GPIO pins to be configured.
    uint32_t Mode;          // Specifies the operating mode for the selected pins.
    uint32_t Pull;         // Specifies the Pull-up or Pull-Down activation for the selected
    uint32_t Speed;        // Specifies the speed for the selected pins.
    uint32_t Alternate;    // Peripheral to be connected to the selected pins.
} GPIO_InitTypeDef;

```

Para facilitar el uso de esta estructura, las HAL incluyen definiciones que simplifican la escritura de las aplicaciones. El siguiente listado incluye algunas de las definiciones para comprensión de la idea.

```

#define GPIO_PIN_0          ((uint16_t)0x0001) /* Pin 0 selected */
#define GPIO_PIN_1          ((uint16_t)0x0002) /* Pin 1 selected */
#define GPIO_PIN_2          ((uint16_t)0x0004) /* Pin 2 selected */
#define GPIO_PIN_3          ((uint16_t)0x0008) /* Pin 3 selected */
#define GPIO_PIN_4          ((uint16_t)0x0010) /* Pin 4 selected */
#define GPIO_PIN_5          ((uint16_t)0x0020) /* Pin 5 selected */
#define GPIO_PIN_6          ((uint16_t)0x0040) /* Pin 6 selected */

```

```
#define GPIO_PIN_7          ((uint16_t)0x0080) /* Pin 7 selected */
#define GPIO_PIN_8          ((uint16_t)0x0100) /* Pin 8 selected */
#define GPIO_PIN_9          ((uint16_t)0x0200) /* Pin 9 selected */
#define GPIO_PIN_10         ((uint16_t)0x0400) /* Pin 10 selected */
#define GPIO_PIN_11         ((uint16_t)0x0800) /* Pin 11 selected */
#define GPIO_PIN_12         ((uint16_t)0x1000) /* Pin 12 selected */
#define GPIO_PIN_13         ((uint16_t)0x2000) /* Pin 13 selected */
#define GPIO_PIN_14         ((uint16_t)0x4000) /* Pin 14 selected */
#define GPIO_PIN_15         ((uint16_t)0x8000) /* Pin 15 selected */
#define GPIO_PIN_All        ((uint16_t)0xFFFF) /* All pins selected */

#define GPIO_MODE_INPUT      ((uint32_t)0x00000000) /*!< Input Floating Mode
*/
#define GPIO_MODE_OUTPUT_PP  ((uint32_t)0x00000001) /*!< Output Push Pull Mod
*/
#define GPIO_MODE_OUTPUT_OD  ((uint32_t)0x00000011) /*!< Output Open Drain Mo
*/
#define GPIO_MODE_AF_PP      ((uint32_t)0x00000002) /*!< Alternate Function P
*/
#define GPIO_MODE_AF_OD      ((uint32_t)0x00000012) /*!< Alternate Function O
*/

#define GPIO_MODE_ANALOG     ((uint32_t)0x00000003) /*!< Analog Mode
*/

#define GPIO_MODE_IT_RISING  ((uint32_t)0x10110000) /*!< External Interrupt M
*/
#define GPIO_MODE_IT_FALLING ((uint32_t)0x10210000) /*!< External Interrupt M
*/
#define GPIO_MODE_IT_RISING_FALLING ((uint32_t)0x10310000) /*!< External Interrupt M
*/

#define GPIO_MODE_EVT_RISING ((uint32_t)0x10120000) /*!< External Event Mode
*/
#define GPIO_MODE_EVT_FALLING ((uint32_t)0x10220000) /*!< External Event Mode
*/
#define GPIO_MODE_EVT_RISING_FALLING ((uint32_t)0x10320000) /*!< External Event Mode
*/

#define GPIO_SPEED_FREQ_LOW  ((uint32_t)0x00000000) /*!< IO works at 2 MHz, please refer
#define GPIO_SPEED_FREQ_MEDIUM ((uint32_t)0x00000001) /*!< range 12,5 MHz to 50 MHz, please
#define GPIO_SPEED_FREQ_HIGH ((uint32_t)0x00000002) /*!< range 25 MHz to 100 MHz, please
*/
#define GPIO_SPEED_FREQ_VERY_HIGH ((uint32_t)0x00000003) /*!< range 50 MHz to 200 MHz, please
*/

#define GPIO_NOPULL          ((uint32_t)0x00000000) /*!< No Pull-up or Pull-down activation
*/
#define GPIO_PULLUP          ((uint32_t)0x00000001) /*!< Pull-up activation
*/
#define GPIO_PULLDOWN        ((uint32_t)0x00000002) /*!< Pull-down activation
*/
```

Actividad: Modifica el módulo led.c para que gestione un LED montado según la figura 3.7.

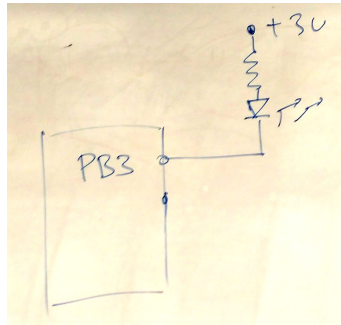


Figura 3.7: Un LED montado de una manera clásica en antiguos micros.

3.6 Entrada digital

3.6.1 Célula en modo entrada

La figura 3.8 ilustra la célula configurada en modo entrada. La entrada se realiza mediante un buffer tipo Schmitt para filtrar las señales. Mediante las resistencias de *pull-up* y *pull-down* se amplía el rango de posibilidades de conexión sin necesidad de añadir más componentes externos.

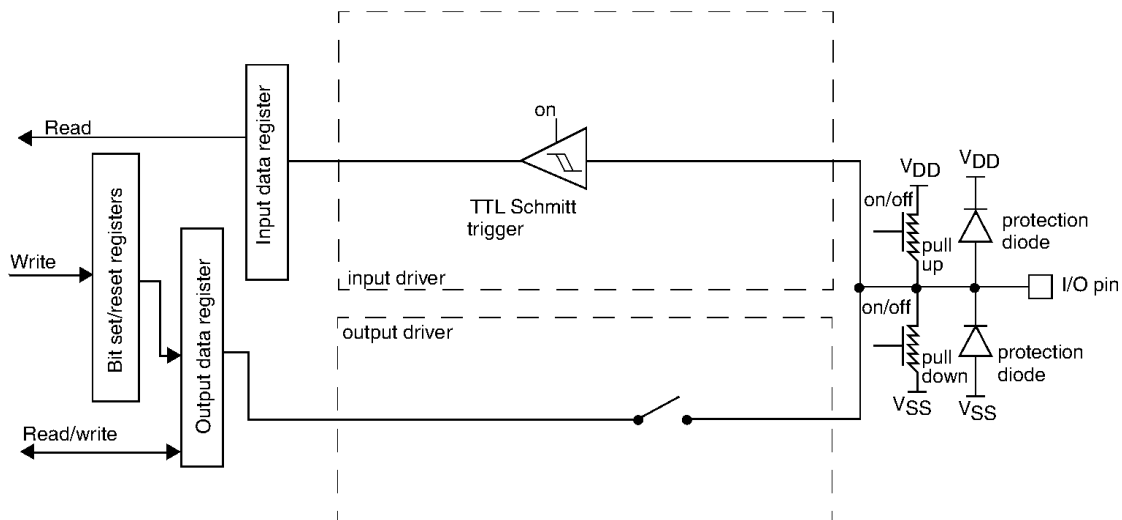


Figura 3.8: Configuración de la célula en modo entrada (*input*).

3.6.2 Ejemplo: Un pulsador

Como primer ejemplo, en la figura 3.9 se ilustra la conexión del pulsador de usuario en la placa STM32F429Discovery. No es un diseño afortunado para una primera aproximación, pero es lo más inmediato de lo que se dispone.

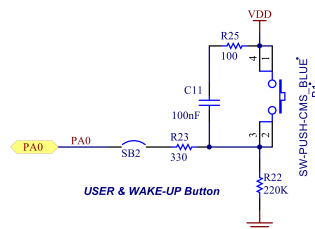


Figura 3.9: Esquemático del pulsador de usuario en la placa STM32F429Discovery.

En el manual de la placa Discovery, localizar el esquema de conexión del *Actividad:* pulsador azul y el pin al que está conectado. Exponer el funcionamiento de dicho pulsador.

NOTA: No he provat aquestos exemples.

Para aprovechar el pulsador, se propone las siguiente cabecera:

```
/* button.h */

#ifndef BUTTON_H
#define BUTTON_H

typedef enum {BUTTON_PRESSED, BUTTON_RELEASED} TButtonState;
void button_Init(void);
TButtonState button_Read(void);

#endif
```

Una posible implementación empleando las bibliotecas HAL sería:

```
/* button.c */

#include "stm32f4xx_hal.h"
#include "button.h"

void button_Init(void)
{
    GPIO_InitTypeDef port;

    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure the GPIO_LED pin */
    port.Pin = GPIO_PIN_0;
    port.Mode = GPIO_MODE_INPUT;
    port.Pull = GPIO_NOPULL;
    HAL_GPIO_Init(GPIOA, &port);
}
```

```
TButtonState button_Read(void)
{
    if (HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0) == GPIO_PIN_RESET) {
        return BUTTON_RELEASED;
    } else {
        return BUTTON_PRESSED;
    }
}
```

Para usar este módulo, se podría hacer:

```
#include "button.h"

void main(void)
{
    button_Init();

    while(1) {
        if (button_Read() == BUTTON_PRESSED) {
            // actions here
        }
    }
}
```

El esquema de conexión del pulsador se puede simplificar mucho más si se tienen en cuenta las resistencias internas. Por ejemplo, es habitual utilizar la configuración de la figura 3.10.

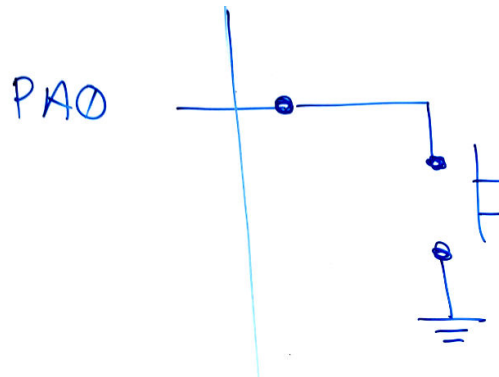


Figura 3.10: Una posible configuración mínima del pulsador.

Para esa configuración mínima, bastaría modificar la función `button_init()` de la siguiente manera.

```
void button_Init(void) {
    GPIO_InitTypeDef port;

    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure the GPIO_LED pin */
    port.Pin = GPIO_PIN_0;
    port.Mode = GPIO_MODE_INPUT;
    port.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &port);
}
```

Tabla 3.3: Algunos registros de interés de la estructura GPIO_TypeDef.

<code>uint32_t IDR;</code> Accede al registro de entrada digital del puerto GPIO.
<code>uint32_t ODR;</code> Accede al registro de salida digital del puerto GPIO.
<code>uint32_t BSRR;</code> Accede al registro de set/reset del puerto GPIO. Permite poner a 1 y/o a 0 bits individuales. Los 1 en los 16 bits de menor peso, pone a uno el bit del puerto correspondiente. Los 1 en los 16 bits de mayor peso pone a 0 el bit correspondiente del puerto, correspondiendo igualmente estos bits a las líneas 15 a 0.

Actividad: Idear otra posible configuración mínima para el pulsador.

3.7 Acceso directo a los registros de periférico

Las funciones HAL solo exponen una parte de las características del periférico GPIO. Para acceder a todas las posibilidades, la única opción es manipular directamente los registros del controlador del GPIO.

La estructura C GPIO_TypeDef representa internamente estos registros y se mapea para cada puerto sobre la posición de memoria correspondiente a cada GPIOx, por lo que es inmediato acceder directamente al hardware.

En la tabla 3.3 se listan los registros a los que puede ser interesante acceder directamente.

Por ejemplo, para leer toda la entrada digital del puerto GPIOC se podría hacer:

```
uint16_t my_input;
my_input = GPIOC->IDR;
```

Para poner los 8 bits de mayor peso del puerto GPIOA a 1 y los 8 de menor peso a 0 se podría hacer:

```
GPIOA->ODR = 0xFF00;
```

Y, para poner a 1 los bits 7,3 y 2 y a 0 los bits 15,12 y 11 en el puerto A se podría hacer (ver figura 3.11):

```
GPIOA->BSRR = 0x9800008C;
```

Como ejemplo de uso, el siguiente fragmento de código hace uso de máscaras y estas funciones para la actividad del botón.

```
/* button.c */

//#include "stm32f4xx_hal.h"
#include "stm32f4xx_ap_gpio.h"
```

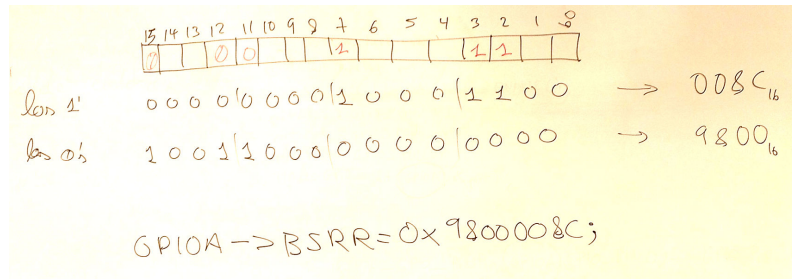


Figura 3.11: Ejemplo de uso del registro BSRR.

```
#include "button.h"

void button_Init(void) {

    GPIO_InitTypeDef GPIO_InitStructure; // estructura donde se pone la configuracion deseada

    __HAL_RCC_GPIOA_CLK_ENABLE();          // darle reloj al periferico, AHORA VIVE!

    /* Configure the GPIO_LED pin */
    GPIO_InitStructure.Pin = GPIO_PIN_0;           // pin que desamos configurar
    GPIO_InitStructure.Mode = GPIO_MODE_INPUT;      // lo vamos a usar como salida en push-pull
    GPIO_InitStructure.Pull = GPIO_NOPULL;          // desactivar pulls

    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);    // hacer efectiva configuracion puerto
}

TButtonState button_Read(void)
{
    if ((GPIOA->IDR & 0x0001) != 0) {
        return BUTTON_PRESSED;
    } else {
        return BUTTON_RELEASED;
    }
}
}
```

3.7.1 Ejemplo: dial selector

Se desea desarrollar un módulo C para gestionar el selector de programas de una lavadora según la figura 3.12.

Para ello el equipo de desarrollo de alto nivel propone un archivo de cabecera que lo abstrae y tiene el siguiente listado.

```
/**
 * @file selector.h
 * @brief Header for abstracting the selector (or tumble??)
 */

#ifndef SELECTOR_H
#define SELECTOR_H

// available positions of the selector
typedef enum {
    SELECTOR_OFF,          // maquina apagada
    SELECTOR_COTTON_1,     // algodón tipo 1

```



Figura 3.12: Un selector de programas de una lavadora inteligente

```

SELECTOR_COTTON_2,    // algodón tipo 2
SELECTOR_SYNTHETIC,  // tejidos sintéticos
SELECTOR_QUICK,       // lavado rápido
SELECTOR_DELICATE,    // lavado delicado
SELECTOR_WOOL,        // lavado lanas
SELECTOR_CARE,        // lavado muy delicado
SELECTOR_DESAGUE,    // desagüe
SELECTOR_SPIN_1,      // centrifugado nivel 1
SELECTOR_SPIN_2,      // centrifugado nivel 2
SELECTOR_ACLARADO,    // aclarado + centrifugado nivel 2
SELECTOR_UNDEFINED    // valor no esperado
} TSelectorStatus;

void selector_Init(void);
TSelectorStatus selector_Read(void);

#endif

```

Por su parte, el equipo de diseño hardware desarrolla un mando con el diseño de la figura 3.13, donde se asigna cada señal a un pin de un microcontrolador STM32F4x.

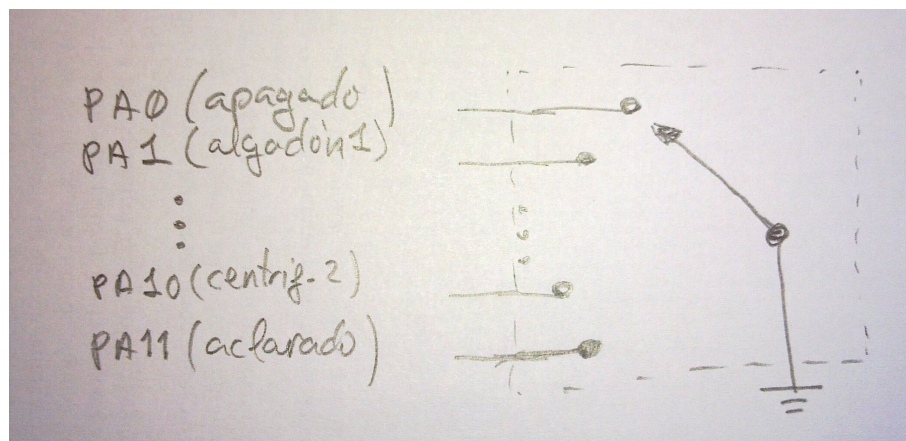


Figura 3.13: Un selector de programas de una lavadora inteligente

Un módulo C que implementa la funcionalidad deseada teniendo en cuenta que hay pines del puerto no asignados (se deberían enmascarar) y hay combinaciones no contempladas. Sería el siguiente:

```
#include "stm32f4xx_hal.h"
#include "selector.h"

void selector_Init ( void )
{
    GPIO_InitTypeDef port;

    __HAL_RCC_GPIOA_CLK_ENABLE();

    port.Pin = GPIO_PIN_12 | GPIO_PIN_11 | GPIO_PIN_10
              | GPIO_PIN_9 | GPIO_PIN_8 | GPIO_PIN_7
              | GPIO_PIN_6 | GPIO_PIN_5 | GPIO_PIN_4
              | GPIO_PIN_3 | GPIO_PIN_2 | GPIO_PIN_1 | GPIO_PIN_0;
    port.Mode = GPIO_MODE_INPUT;
    port.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &port);
}

TSelectorStatus selector_Read ( void )
{
    uint16_t data;
    TSelectorStatus value;

    data = GPIOA->IDR;
    data = data & 0xFFFF;

    switch (data) {
        case 0xFFFE:
            value = SELECTOR_OFF;
            break;
        case 0xFFD:
            value = SELECTOR_COTTON_1;
            break;
        case 0xFFB:
            value = SELECTOR_COTTON_2;
            break;
        case 0xFF7:
            value = SELECTOR_SYNTHETIC;
            break;
        case 0xFEf:
            value = SELECTOR_QUICK;
            break;
        case 0xFDF:
            value = SELECTOR_DELICATE;
            break;
        case 0xFBF:
            value = SELECTOR_WOOL;
            break;
        case 0xF7F:
            value = SELECTOR_CARE;
            break;
        case 0xEFF:
            value = SELECTOR_DESAGUE;
            break;
        case 0DFF:
            value = SELECTOR_OFF;
            break;
    }
}
```

```
    case 0x0BFF:
        value = SELECTOR_SPIN_1;
        break;
    case 0x07FF:
        value = SELECTOR_SPIN_2;
        break;
    case 0x0FFF:
        value = SELECTOR_ACLARADO;
        break;
    default:
        value = SELECTOR_UNDEFINED;
        break;
}

return value;
}
```


3.8 Ejemplo: un display de 7 segmentos

Un display de 7 segmentos es un conjunto de LEDs encapsulados formando una imagen similar a la mostrada en la figura 3.14.

La configuración interna del display puede ser en cátodo común o en ánodo común según si todos los cátodos o ánodos se unen internamente, accediéndose a ellos mediante una única línea. Como ejemplo, la figura 3.15 muestra una configuración en cátodo común con la típica una serie de resistencias externas que limitarían la corriente que circula por los diodos.

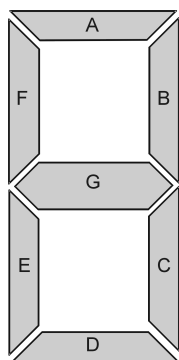


Figura 3.14: Identificación de segmentos de un display de 7 segmentos.

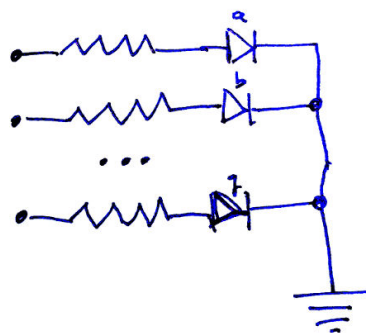


Figura 3.15: Conexión típica en cátodo común.

Suponiendo la configuración de la figura 3.15, la siguiente tabla sería adecuada para representar los dígitos de 0 a 9 y usar la letra E para indicar un estado anómalo. Dado el montaje anterior, destacar que un “1” encendería el segmento y un “0” lo apagaría.

Símbolo	G	F	E	D	C	B	A	Hexadecimal
0	0	1	1	1	1	1	1	3Fh
1	0	0	0	0	1	1	0	06h
2	1	0	1	1	0	1	1	5Bh
3	1	0	0	1	1	1	1	4Fh
4	1	1	0	0	1	1	0	66h
5	1	1	0	1	1	0	1	6Dh
6	1	1	1	1	1	0	1	7Dh
7	0	0	0	0	1	1	1	07h
8	1	1	1	1	1	1	1	7Fh
9	1	1	0	1	1	1	1	6Fh
E	1	1	1	1	0	0	1	79h

Se propone crear un módulo con funciones para manejar este display. Por ejemplo, las funciones listadas en el archivo de cabecera `display7seg.h` mostrado a continuación.

```
/**
 * @file display7seg.h
 * @brief Module header for handling a 7 segments display
 */
#ifndef DISPLAY7SEG_H
#define DISPLAY7SEG_H

#include <stdint.h>

void display7seg_Init (void);
void display7seg_SetNumber (uint8_t value);
void display7seg_Off (void);

#endif
```

Para comprobar dicho módulo se podría desarrollar el siguiente programa de test.

```
uint8_t n;

display7seg_Init();

while(1) {
    for (n=0; n <= 10; n++) {
        display7seg_SetNumber(n);
        HAL_Delay(500);
    }
}
```

Ahora toca implementarlo. Este display es una excusa estupenda para introduciendo distintas mejoras en el estilo de programación y en la adaptación a un sistema embebido con restricciones en cuanto a espacio y velocidad.

Supóngase que se ha de crear un módulo C con nombre `display7seg.c` que use la siguiente configuración de la electrónica: display de cátodo común, pines PB6 hasta PB0 en modo salida, drenador abierto, push/pull desactivado y velocidad de actualización baja.

Una primera solución de novato sería:

```
/**
 * @file display7seg-v1b.c
 * @brief Handles a 7-segment display.
 */

#include <stm32f4xx_hal.h>
#include <stm32f4xx_ap_gpio.h>
#include "display7seg.h"

void display7seg_Init(void)
{
    GPIO_InitTypeDef port;

    __HAL_RCC_GPIOB_CLK_ENABLE();

    /* Configure the GPIO_LED pin */
    port.Pin = GPIO_PIN_6 | GPIO_PIN_5 |
```

```
        GPIO_PIN_4 | GPIO_PIN_3 |
        GPIO_PIN_2 | GPIO_PIN_1 | GPIO_PIN_0;
port.Mode = GPIO_MODE_OUTPUT_PP;
port.Pull = GPIO_NOPULL;
port.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOB, &port);
}

void display7seg_SetNumber(uint8_t value)
{
    uint8_t out;

    switch(value){
        case 0:
            out = 0x3F;
            break;
        case 1:
            out = 0x06;
            break;
        case 2:
            out = 0x5B;
            break;
        case 3:
            out = 0x4F;
            break;
        case 4:
            out = 0x66;
            break;
        case 5:
            out = 0x6D;
            break;
        case 6:
            out = 0x7D;
            break;
        case 7:
            out = 0x07;
            break;
        case 8:
            out = 0x7F;
            break;
        case 9:
            out = 0x6F;
            break;
        default:
            out = 0x79;
            break;
    }

    GPIOB->ODR = out;
}

void display7seg_Off (void)
{
    GPIOB->ODR = 0x0000;
}
```

Esta solución se puede mejorar en cuanto a espacio ocupado y velocidad de ejecución si sustituimos la estructura **switch-case** con tablas de búsqueda *look-up* (ver sección 6.5). El siguiente listado corresponde a aplicar dicha optimización en la función `display7seg_SetNumber()`.

```
static const uint8_t display7seg_tabnum[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

void display7seg_SetNumber(uint8_t value)
{
    uint8_t out;

    if (value <= 9) {
        out = display7seg_tabnum[value];
    } else {
        out = 0x79; // symbol E
    }

    GPIOB->ODR = out;
}
```

Obsérvese el uso del atributo `const` para mantener los datos en la memoria de programa.

Aunque el programa se ha mejorado, estas soluciones modifican también las líneas del puerto que no tienen relación con display. Esto es una chapuza en toda regla, así que se deberían emplear opciones que modifiquen solo las líneas implicadas, para ello hay que emplear máscaras y/o las funciones que permiten modificar bits individuales. El siguiente listado corresponde a esta mejora.

```
static const uint8_t display7seg_tabnum[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

void display7seg_SetNumber(uint8_t value)
{
    uint32_t out;

    if (value <= 9) {
        out = display7seg_tabnum[value];
    } else {
        out = 0x79; // symbol E
    }

    /* option 1: first clear with 0s, then add 1's */
    HAL_GPIO_WritePin (GPIOB, 0x7F, GPIO_PIN_RESET);
    HAL_GPIO_WritePin (GPIOB, out,GPIO_PIN_SET);

    /* option 2: get 1's and gets 0's. Can be interchanged */
    //HAL_GPIO_WritePin (GPIOB, out, GPIO_PIN_SET);
    //HAL_GPIO_WritePin (GPIOB, (~out) & 0x7F, GPIO_PIN_RESET);

    /* option 3: both 1's and 0's in a wonderful atomic operation */
    //GPIOB->BSRR = ((~out) & 0x7F) << 16) | out;
}
```

En el listado se ofrecen tres soluciones posibles, la primera se basa en la limpieza de los bits destino y consecución de 1s; la segunda en consecución de 0s y despues de 1s (son intercambiables) y, por último, una solución que aprovecha el acceso directo al registro BSRR. Cualquiera de las tres soluciones para manejar el display es ya de nivel profesional.

3.9 Teclados matriciales

Es un buen ejemplo para introducir la idea de multiplexación en el tiempo de líneas, usándose un teclado multiplexado como ejemplo.

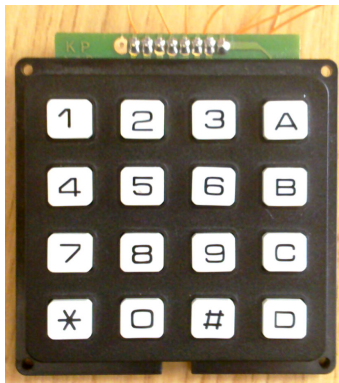


Figura 3.16: Un teclado típico de 4x4 teclas.

Se pretende crear un módulo/biblioteca que cre una abstracción para facilitar su manejo. Se propone entonces implementar funciones según el archivo de cabecera `keyboard.h` listado a continuación:

```
/**
 * @file keyboard.h
 * @brief Matrix 4x4 keyboard scan
 */

#ifndef KEYBOARD_H
#define KEYBOARD_H

#include <stdint.h>

void keyboard_Init(void);
uint8_t keyboard_Scan(void);

#endif
```

donde se tiene la función de prearación del módulo `keyboard_Init()` y la función `keyboard_Scan()` que se propone que devuelva el valor 0 si no hay una tecla pulsada y el código ASCII correspondiente si hay una tecla pulsada.

Internamente, un teclado matricial es una matriz de pulsadores organizada en filas y columnas, bla, bla.

Solución:

```
/**
 * @file keyboard.c
 * @brief Matrix 4x4 keyboard scan
 */

#include "stm32f4xx_hal.h"
#include "keyboard.h"
```

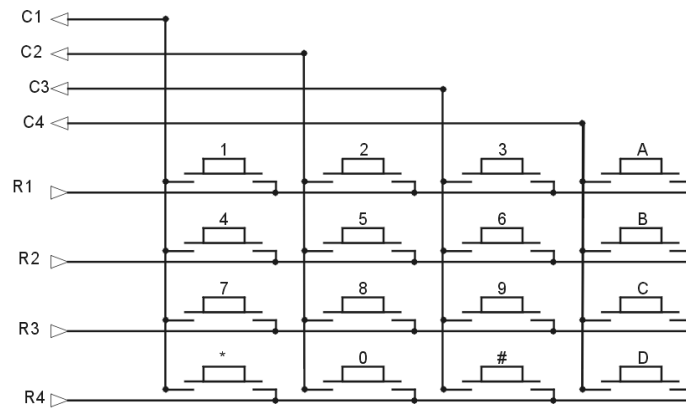


Figura 3.17: Esquema interno.

```
static const uint8_t key_table[4][4]= {
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

/*****
**
* @brief Hardware configuration
* @param none
* @returns none
*/
void keyboard_Init(void)
{
    GPIO_InitTypeDef g;

    __HAL_RCC_GPIOE_CLK_ENABLE();

    /* rows config */
    g.Pin = GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7;
    g.Mode = GPIO_MODE_OUTPUT_PP;
    g.PULL = GPIO_NOPULL;
    g.Speed = GPIO_SPEED_FREQ_LOW;

    HAL_GPIO_Init(GPIOE, &g);

    /* cols config */
    g.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11;
    g.Mode = GPIO_MODE_INPUT;
    g.Pull = GPIO_PULLUP;

    HAL_GPIO_Init(GPIOE, &g);
}

/*****
**
* @brief Scan keyboard
* @retval x
*/
```

```
uint8_t keyboard_scan(void)
{
    uint8_t row;
    int8_t col;
    uint16_t data;

    for (row=0; row<4; row++) {
        HAL_GPIO_WritePin(GPIOE, GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7, GPIO_PIN_SET);
        // clear
        switch(row) {
            case 0: HAL_GPIO_WritePin(GPIOE, GPIO_Pin_4, GPIO_PIN_RESET); break;
            case 1: HAL_GPIO_WritePin(GPIOE, GPIO_Pin_5, GPIO_PIN_RESET); break;
            case 2: HAL_GPIO_WritePin(GPIOE, GPIO_Pin_6, GPIO_PIN_RESET); break;
            case 3: HAL_GPIO_WritePin(GPIOE, GPIO_Pin_7, GPIO_PIN_RESET); break;
        }

        HAL_Delay(1);          // this is excessive, but ...
        data = GPIOE->IDR;
        data = (data >> 8) & 0xF;

        col = -1;
        switch(data) {
            case 0xE: col = 0; break;
            case 0xD: col = 1; break;
            case 0xB: col = 2; break;
            case 0x7: col = 3; break;
        }
        if (col != -1) {
            break; // exit loop
        }
    }

    if (col == -1) {
        return 0;
    } else {
        return key_table[row][col];
    }
}

/** End of file *****/
```


3.10 Multiplexado temporal con varios display de 7 segmentos



Figura 3.18: Un ejemplo típico de uso de la técnica de multiplexación en tiempo.

Esto es avanzado para este tema. Hay ejemplo de tarea asociada a una interrupción y de acceso directo al hardware. Si alguien lo necesita, se la paso.

Cabecera propuesta:

```
/**
 * @file displaymx.h
 * @brief Module header for handling a multiplexed 7 seg. display
 */
#ifndef DISPLAYMX_H
#define DISPLAYMX_H

#include <stdint.h>

void displaymx_Init (void);
void displaymx_Off (void);
```

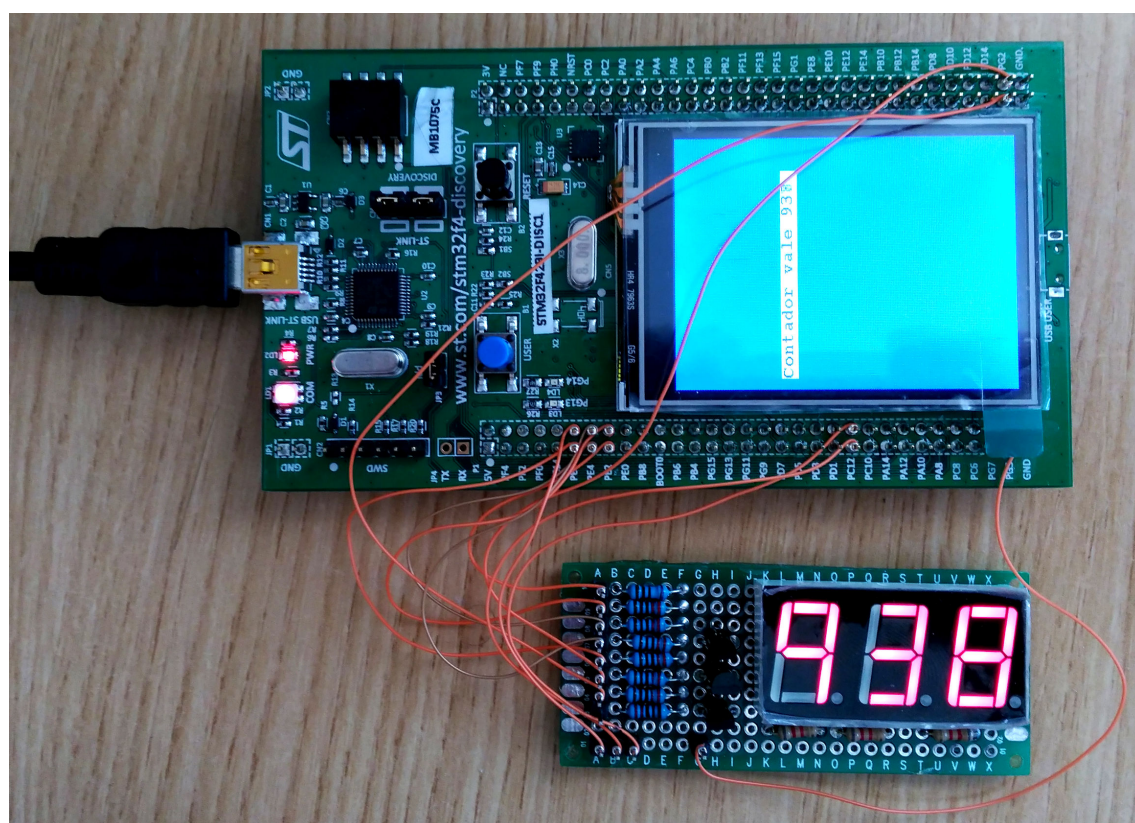


Figura 3.19: Y una implementación funcionando en la Discovery.

```
void displaymx_On (void);
void displaymx_SetNumber (int16_t value);
void displaymx_SetSegments (uint8_t num_display, uint8_t config);
void displaymx_Refresh(void);

extern volatile uint8_t displaymx_refresh_active;

#endif
```

Implementación:

```
/**
 * @file displaymx.h
 * @brief Implementation for handling a multiplexed 7 seg. display
 *
 * Lines PE6 to PE2 for segments EDCBA
 * Lines PG3 to PG2 for segments GF
 * 1 = segment on, 0 = segment off
 * Lines PC13, PC12, PC11 display 2 to 0
 * 1 = active display. 0 = inactive
 *
 * @author Angel Perles
 * @date 2016-05-29
 */

#include <stm32f4xx_hal.h>
#include <displaymx.h>

#define NUM_DISPLAYS 3 // number of displays

// BSSR register value for each line associated to segment displays
// segments EDCBAD -> PE6-PE2
static uint32_t displaymx_data_PE6_PE2[NUM_DISPLAYS];
// segments GF -> PG3-PG2
static uint32_t displaymx_data_PG3_PG2[NUM_DISPLAYS];
// display selectors
static uint32_t displaymx_selector_PC13_PC11[NUM_DISPLAYS];

// 0 = refresh is inactive
volatile uint8_t displaymx_refresh_active = 0;

// look-up table for translating numbers from 0 to 9 to segments
// bit 7 to 0 mapped to segments g to a
// bit 1 is segment on, 0 = segment off
//static uint8_t number_to_segments_table[]={0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x67}
static const uint8_t number_to_segments_table[]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x67}

/*****
 * @brief Initializes the hardware
 * @param none
 * @retval none
 */
void displaymx_Init ( void ){

    GPIO_InitTypeDef port; // estructura donde se pone la configuracion deseada

    // segments pins initialisation
    __HAL_RCC_GPIOE_CLK_ENABLE();
    port.Pin = GPIO_PIN_6 | GPIO_PIN_5 | GPIO_PIN_4 | GPIO_PIN_3 | GPIO_PIN_2;
    port.Mode = GPIO_MODE_OUTPUT_PP;
    port.Pull = GPIO_NOPULL;
    port.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOE, &port);

__HAL_RCC_GPIOG_CLK_ENABLE();
port.Pin = GPIO_PIN_3 | GPIO_PIN_2;
port.Mode = GPIO_MODE_OUTPUT_PP;
port.Pull = GPIO_NOPULL;
port.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOG, &port);

// display selector
__HAL_RCC_GPIOC_CLK_ENABLE();
port.Pin = GPIO_PIN_13 | GPIO_PIN_12 | GPIO_PIN_11;
port.Mode = GPIO_MODE_OUTPUT_PP;
port.Pull = GPIO_NOPULL;
port.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOC, &port);

// set values for setting bits for each display, COLLINS que mal m'explique
displaymx_selector_PC13_PC11[0]=0x30000800;
displaymx_selector_PC13_PC11[1]=0x28001000;
displaymx_selector_PC13_PC11[2]=0x18002000;

//displaymx_data_PE6_PE2[0] = 0x0000007A;

displaymx_Off();
}

/*****
**
* @brief Activates the display refreshing
* @param none
* @retval none
*/
void displaymx_On (void)
{
    displaymx_refresh_active = 1;
}

/*****
**
* @brief Deactivates display refreshing
* @param none
* @retval none
*/
void displaymx_Off ( void )
{
    displaymx_refresh_active = 0;
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13 | GPIO_PIN_12 | GPIO_PIN_11, GPIO_PIN_RESET);
    //HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
}

/*****
**
* @brief Puts an integer number in the display
* @param value to be represented from 0 to 9999
* @retval Nada
*/
void displaymx_SetNumber (int16_t value)
{
    //uint16_t data;
    uint8_t i;
```

```
// clear segments
for (i=0;i<NUM_DISPLAYS;i++) {
    displaymx_SetSegments(i,0);    // delete all segments
}

// test range
if ((value<0) || (value>999) ){
    return;
}

// decompose number
for (i=0;i<NUM_DISPLAYS;i++) {
    displaymx_SetSegments(i,number_to_segments_table[value % 10]);
    value /= 10;
    if (value == 0) {
        break; // exit for
    }
}
}

/*****
**
* @brief Set the state of the 7 segments of a given display
* @param num_display elected display
* @param config segment configuration bit 7 to 0 mapped to segments g to a
* @returns none
*/
void displaymx_SetSegments (uint8_t num_display, uint8_t config)
{
    uint32_t tmp, mask;
    if (num_display >= NUM_DISPLAYS) {
        return;
    }

    // PE6 - PE2 are segments EDCBA
    tmp = config;
    tmp = (tmp & 0x1F) << 2;
    mask = ((~tmp & 0x7C) << 16) | tmp;
    displaymx_data_PE6_PE2[num_display] = mask;

    // PG3 - PG2 are segments GF
    tmp = config;
    tmp = (tmp & 0x60) >> 3;
    mask = ((~tmp & 0x0C) << 16) | tmp;
    displaymx_data_PG3_PG2[num_display] = mask;
}

/*****
**
* @brief Refresh the display
* @param none
* @returns none
*
* This function tries to be as efficient as posible. This is the reason for accessing GPIO direct
* and the relative complexity of the rest of the code
*/
void displaymx_Refresh(void) {

    static uint8_t display = 0;

    GPIOC->BSRR = displaymx_selector_PC13_PC11[display];
    GPIOE->BSRR = displaymx_data_PE6_PE2[display];
    GPIOG->BSRR = displaymx_data_PG3_PG2[display];
}
```

```
    display = (display + 1) % NUM_DISPLAYS;  
}
```

```
/* End of file *****/
```

3.11 E/S digital con el STM32CubeMX

Se puede configurar.

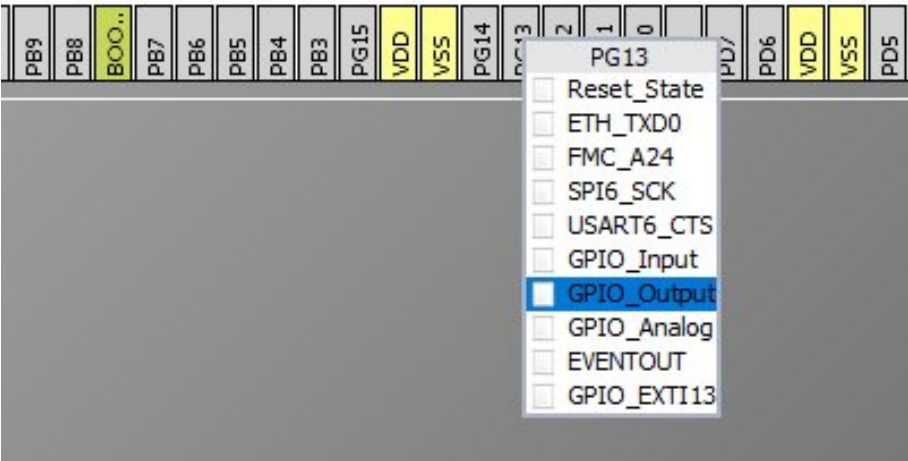


Figura 3.20: Seleccionar el pin en modo salida.

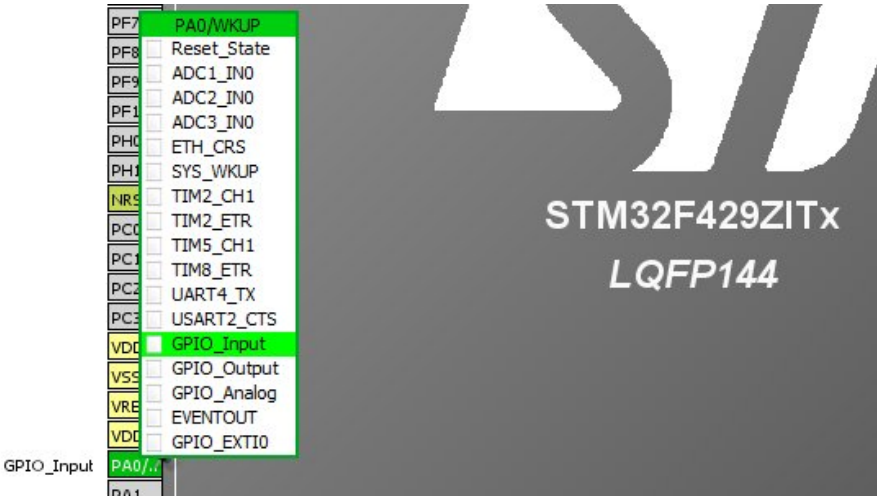


Figura 3.21: Seleccionar el pin en modo entrada.

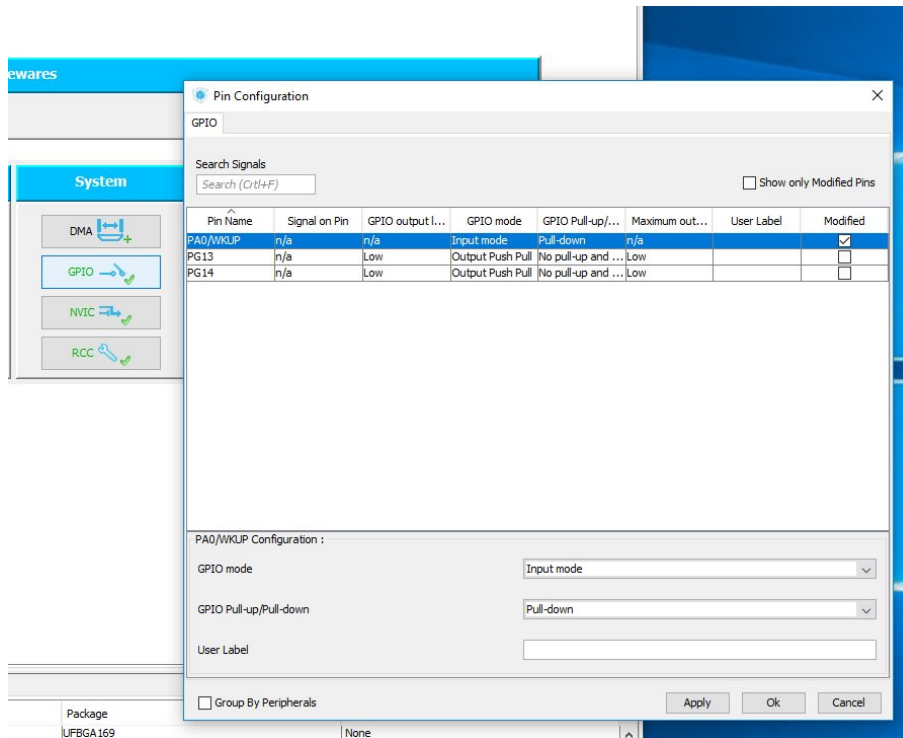


Figura 3.22: Configurar la GPIO.

```
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct;

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOG_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13|GPIO_PIN_14, GPIO_PIN_RESET);

    /*Configure GPIO pin : PA0 */
    GPIO_InitStruct.Pin = GPIO_PIN_0;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLDOWN;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /*Configure GPIO pins : PG13 PG14 */
    GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_14;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);
}
```

Figura 3.23: Y el código resultante.

Capítulo 4

Interrupciones

4.1 Introducción

Objetivos

- Comprender qué es una interrupción.
- Ser conscientes de las ventajas y peligros de las interrupciones.
- Aprender a configurar el sistema de interrupciones.
- Saber crear un servicio de interrupción.

En este capítulo se introduce el concepto de interrupción y su aplicación con los microcontroladores ARM Cortex-M.

Este importante mecanismo es común a todos los procesadores, pero es en los microcontroladores donde destaca por su agilidad de funcionamiento, lo cual es fundamental en la mayoría de las aplicaciones embebidas. Por tanto, es necesario conocer su funcionamiento y su aprovechamiento para poder sacar el máximo partido a un microcontrolador.

Antes de describir el mecanismo de interrupción, es importante introducir el concepto de "*técnica de sincronización*", que es la manera en que la CPU sabe si un determinado dispositivo necesita ser atendido. Son dos:

- La *consulta por programa* o "*polling*", que consiste en leer continuamente los registros de los dispositivos para conocer su estado y actuar en consecuencia.
- La *interrupción*, que consiste en señales que informan a la CPU de que hay que atender algo.

La figura 4.1 muestra el organigrama de una típica aplicación de un microcontrolador. En el caso a), un bucle consulta continuamente el estado de todos los posibles

eventos a tratar. La mayoría de las veces no será necesario hacer nada. En el caso b), los eventos se tratan sólo cuando se producen, haciéndose el tratamiento con subprogramas independientes. Cuando se produzca el evento, se detendrá momentáneamente la tarea cíclica en curso y se atenderá al subprograma correspondiente.

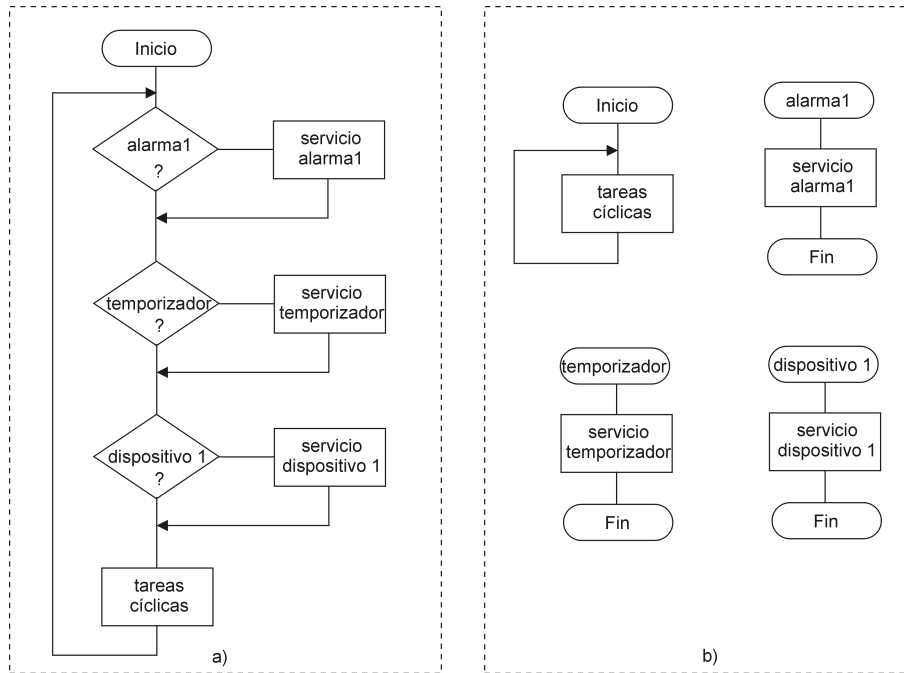


Figura 4.1: Ejemplo de aplicación realizada con técnica de polling (a) y de interrupciones (b).

El mecanismo de *polling* es sencillo de emplear pero da lugar a la llamada "*espera ocupada*", pues el procesador consume tiempo y energía analizando el estado de los dispositivos, tiempo que se podría invertir en otros quehaceres más útiles o, simplemente, en dormir para ahorrar energía. Además, la atención al evento solo se puede hacer cuando se llega al punto de consulta de estado del elemento que causa el evento.

El mecanismo de interrupción permite que las necesidades de los periféricos y los eventos importantes sean notificados a la CPU y los pueda atender inmediatamente si se dan ciertas condiciones. Insistir en que atender a un evento supone abandonar momentáneamente el trabajo que se está realizando, realizar el trabajo que tenga que ver con el evento y continuar, finalmente, con lo que se estaba haciendo antes. Esta interferencia implica un cambio en el tiempo que tardan en ejecutarse las aplicaciones y un esfuerzo extra por el coste del denominado *cambio de contexto*.

Un término medio entre la estrategia de *polling* y la de interrupción suele ser lo adecuado. Así, los periféricos que necesiten atención inmediata se conectan al mecanismo de interrupciones, y el resto de periféricos o tareas se atienden cíclicamente.

También es adecuado combinar las ventajas de ambas estrategias como se mostrará más adelante.

4.2 Funcionamiento general y jerga del sistema de interrupciones

El subsistema de interrupciones es muy similar en todos los procesadores, así que se darán unas líneas generales sobre su funcionamiento genérico y sobre la jerga asociada antes de particularizarlo para los microcontroladores ARM Cortex-M.

La figura 4.2 pretende representar el funcionamiento del mecanismo.

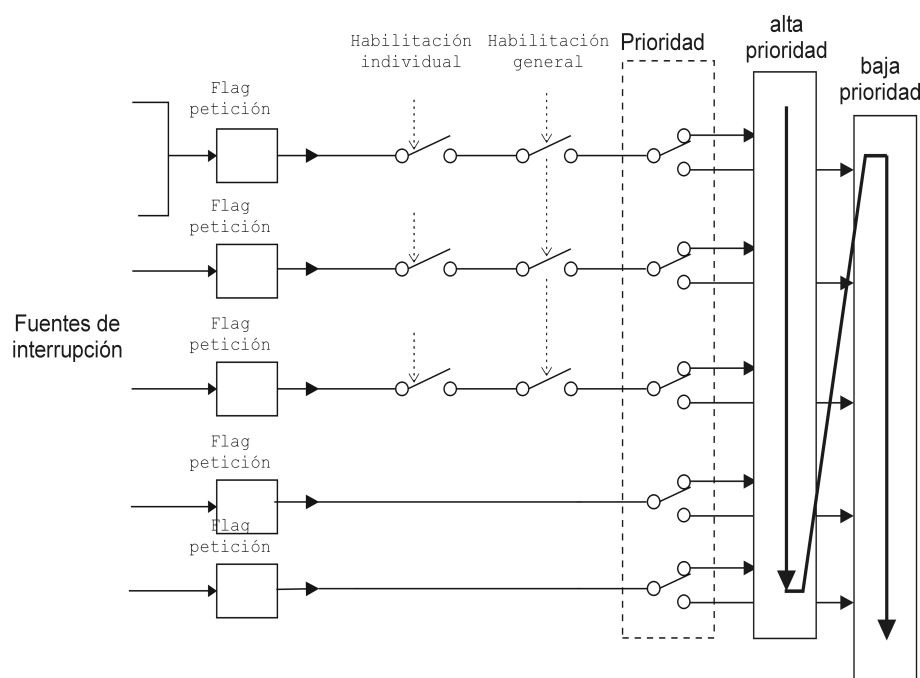


Figura 4.2: Esquema genérico de un sistema de interrupciones.

A la derecha se tiene la fuente de interrupción, que es todo aquello capaz de hacer una petición de interrupción. Pueden ser periféricos, detección de fallos de hardware/software (excepciones), etc. Destacar que se ha dicho “hacer petición” y no “interrumpir”.

En general, una o más fuentes de interrupción señalan que desean hacer la petición mediante la activación de un “*flag*” o indicador (comúnmente, un bit que se pone a “1”). La señal de ese indicador se podrá propagar si tiene el camino libre para hacerlo o, dicho en la jerga, está *habilitada*. En un microcontrolador habrá muchos “flag”s de petición junto con sus correspondientes habilitadores. Los habilitadores individuales

pueden confluir en un habilitador general que permite conectar/desconectar todo el sistema o grupos de interrupciones.

También suele haber un grupo de interrupciones no desconectables y, por tanto, que deben atenderse sí o sí. Estas interrupciones suelen estar asociadas a *excepciones* del sistema, por ejemplo, fallos graves. Para contraponer unas con otras, a las interrupciones desconectables se les llama “*enmascarables*” y a las no desconectables “*no enmascarables*”.

Para que el procesador atienda a las interrupciones, cada vez que el microcontrolador va a ejecutar una instrucción máquina, comprueba si le llega un flag de petición. La comprobación se hace siguiendo un orden secuencial y una priorización. Si se localiza una petición y se acepta, entonces se cancela la ejecución de la instrucción en curso, se guarda el estado del procesador y se localiza el fragmento de programa que debe ejecutarse asociado a esa interrupción. A localizar el fragmento se le llama *vectorizar la interrupción* y al fragmento de programa que lo sirve se le llama *rutina de servicio, servicio de interrupción o manejador*.

Una vez terminada la ejecución del manejador, el procesador recupera el estado anterior y continua ejecutando desde donde se había quedado. El mecanismo es totalmente transparente a la aplicación a excepción de un incremento en el tiempo de ejecución.

El que se acepte o no una interrupción habilitada, dependerá de si ya se está ejecutando o no una interrupción más prioritaria. En los casos más generales, durante la ejecución de una interrupción puede llegar otra que, a su vez, interrumpa a la anterior, anidándose las rutinas de servicio (hacer un dibujo en la pizarra).

Para poder hacer uso del sistema de interrupciones, el programador deberá programar el subsistema de interrupciones para que se vectorice un manejador y proporcionar el código de dicho manejador.

Hay que tener ciertas precauciones al configurar este sistema, pues pueden darse situaciones críticas durante la manipulación, así que se sugiere seguir estos pasos:

1. Deshabilitar la interrupción fuente.
2. Programar el dispositivo fuente.
3. Habilitar la interrupción fuente.

Y acordarse de que debe existir un manejar para esa interrupción.

4.3 Interrupciones en los ARM Cortex-M

(Algunas cosas dices a la lleugera. Comprovar si es vol fer una publicació seriosa).

Todos los microcontroladores de la arquitectura ARM Cortex-M comparten un sub-sistema de gestión de interrupciones llamado *Nested Vectored Interrupt Controller* (NVIC). El NVIC incluye muchos mecanismos para lograr latencias bajas para alcanzar las rutinas de servicio que quedan lejos del propósito de este texto. Para quien desee profundizar, se recomienda el libro [el del Xulin] 2a edición.

Las interrupciones se identifican mediante números enteros, asociándose números negativos a las interrupciones proporcionadas por el núcleo ARM Cortex-M y las positivas al diseño particular de cada fabricante. En el archivo `dispositivo.h` según estándar CMSIS se pueden localizar estos números. En el archivo `stm32f4xx.h` para los STM32F4 proporcionado por St se puede ver lo siguiente:

```
/**
 * @brief STM32F4XX Interrupt Number Definition, according to the selected device
 *        in @ref Library_configuration_section
 */
typedef enum IRQn
{
/***** Cortex-M4 Processor Exceptions Numbers *****/
  NonMaskableInt_IRQn      = -14,    /*!< 2 Non Maskable Interrupt
*/
  MemoryManagement_IRQn    = -12,    /*!< 4 Cortex-M4 Memory Management Interrupt
*/
  BusFault_IRQn            = -11,    /*!< 5 Cortex-M4 Bus Fault Interrupt
*/
  UsageFault_IRQn          = -10,    /*!< 6 Cortex-M4 Usage Fault Interrupt
*/
  SVCall_IRQn              = -5,     /*!< 11 Cortex-M4 SV Call Interrupt
*/
  DebugMonitor_IRQn        = -4,     /*!< 12 Cortex-M4 Debug Monitor Interrupt
*/
  PendSV_IRQn              = -2,     /*!< 14 Cortex-M4 Pend SV Interrupt
*/
  SysTick_IRQn             = -1,     /*!< 15 Cortex-M4 System Tick Interrupt
*/
/***** STM32 specific Interrupt Numbers *****/
  WWDG_IRQn                = 0,      /*!< Window WatchDog Interrupt
*/
  PVD_IRQn                 = 1,      /*!< PVD through EXTI Line detection Interrupt
*/
  TAMP_STAMP_IRQn          = 2,      /*!< Tamper and TimeStamp interrupts through the EXTI line
*/
  RTC_WKUP_IRQn            = 3,      /*!< RTC Wakeup interrupt through the EXTI line
*/
  FLASH_IRQn               = 4,      /*!< FLASH global Interrupt
*/
  RCC_IRQn                 = 5,      /*!< RCC global Interrupt
*/
  EXTI0_IRQn               = 6,      /*!< EXTI Line0 Interrupt
*/
  EXTI1_IRQn               = 7,      /*!< EXTI Line1 Interrupt
*/
  ...
}
```

Las interrupciones con número más bajo tendrán prioridad, lo que significa que si hay dos peticiones simultáneas, se atenderá la que tenga el menor número y, cuando esta termine, se podrá atender a la otra. El sistema de interrupciones es preemptivo, es decir, si durante la ejecución de una interrupción llega otra de mayor prioridad, entonces la interrupción de baja prioridad es interrumpida a su vez para poder atender a la de mayor prioridad.

Con el fin de tener flexibilidad para reorganizar las prioridades de las interrupciones, los ARM Cortex-M incorporan mecanismos de prioridad por grupos. Cada interrupción se puede asociar a un grupo con lo que se puede lograr que interrupciones con números altos puedan tener más prioridad que interrupciones con números más bajos. Por ejemplo, el microcontrolador STM32F4xx permite 32 grupos de prioridad.

En ARM Cortex-M, una rutina de servicio no es más que una función C normal y, con el fin de poner un poco de orden, el estándar CMSIS propone unas reglas para la elección de los nombres de estas funciones. Se puede echar un vistazo al archivo estándar en ensamblador `startup_XXXXX.s` (`startup_stm32f4xx.s` para los STM32F4xx) para ver estos nombres, mostrándose a continuación un fragmento del listado:

```
EXPORT  WWDG_IRQHandler           [WEAK]
EXPORT  PVD_IRQHandler            [WEAK]
EXPORT  TAMP_STAMP_IRQHandler     [WEAK]
EXPORT  RTC_WKUP_IRQHandler       [WEAK]
EXPORT  FLASH_IRQHandler          [WEAK]
EXPORT  RCC_IRQHandler            [WEAK]
EXPORT  EXTI0_IRQHandler          [WEAK]
...
```

Para escribir un manejador asociado a una interrupción, bastará con escribir una función con el nombre idéntico al del listado. Esa función pasará a ser, automáticamente, el manejador de dicha interrupción. Como las aplicaciones que hacen uso intensivo de interrupciones son complejas, es práctica habitual tener muy localizadas las rutinas de servicio. Siguiendo las buenas prácticas de CMSIS, el archivo sugerido para ello se llama `XXXX_it.c` (`stm32f4xx_it.c` para los STM32F4xx). Véase a continuación un fragmento de este archivo:

```
/**
 * @brief   This function handles NMI exception.
 * @param   None
 * @retval  None
 */
void NMI_Handler(void)
{
}

/**
 * @brief   This function handles Hard Fault exception.
 * @param   None
 * @retval  None
 */
void HardFault_Handler(void)
{
    /* Go to infinite loop when Hard Fault exception occurs */
}
```

```
    while (1)
    {
    }
}

/**
 * @brief This function handles Memory Manage exception.
 * @param None
 * @retval None
 */
void MemManage_Handler(void)
{
    /* Go to infinite loop when Memory Manage exception occurs */
    while (1)
    {
    }
}

/**
 * @brief This function handles Bus Fault exception.
 * @param None
 * @retval None
 */
void BusFault_Handler(void)
{
    /* Go to infinite loop when Bus Fault exception occurs */
    while (1)
    {
    }
}
```

En este archivo se deberían implementar todos los manejadores de interrupción. Como se puede observar, algunos de ellos están ya implementados porque están asociados a excepciones graves y/o no enmascarables.

Para ilustrar el funcionamiento de una excepción, se propone observar BusFault (fallo en bus). Esta interrupción/excepción viene habilitada y preconfigurada, y deja enganchado el microcontrolador en un bucle infinito, pues se trata de un error grave.

Para verla en acción, se propone modificar el manejador para BusFault (fallo en bus) según el siguiente listado:

```
#include <led.h>
void BusFault_Handler(void)
{
    LED_On();
    /* Go to infinite loop when Bus Fault exception occurs */
    while (1)
    {
    }
}
```

Y usar el siguiente programa principal que causa la excepción y, por tanto, hará lucir el LED y dejar al microcontrolador enganchado. La razón del fallo se debe a que el programa principal termina, lo que no es razonable en un microcontrolador. Si se descomenta el bucle `while()`, el fallo no se debería producir.

Tabla 4.1: Funciones HAL de inicialización del sistema de interrupciones NVIC.

<pre>void HAL_NVIC_SetPriorityGrouping (uint32_t PriorityGroup)</pre>
Sets the priority grouping field (pre-emption priority and subpriority) using the required unlock sequence.
<pre>void HAL_NVIC_SetPriority (IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)</pre>
Sets the priority of an interrupt.
<pre>void HAL_NVIC_EnableIRQ (IRQn_Type IRQn)</pre>
Enables a device specific interrupt in the NVIC interrupt controller.
<pre>void HAL_NVIC_DisableIRQ (IRQn_Type IRQn)</pre>
Disables a device specific interrupt in the NVIC interrupt controller.
<pre>void HAL_NVIC_SystemReset (void)</pre>
Initiates a system reset request to reset the MCU.

```
#include <led.h>
int main(void)
{
    LED_Init();

    //while (1) {};    // descomentar despues

    return 0;
}
```

Para emplear interrupciones, además de proporcionar la rutina de servicio, se deberán seguir los pasos indicados al final de la sección 4.2, que requerirán el uso de funciones CMSIS y documentadas en **[CMIS-core-manual]** y que han sido adaptadas (básicamente, renombradas) al HAL de St según tablas 4.1 y 4.2.

Tabla 4.2: Funciones HAL de control del sistema de interrupciones NVIC.

<code>uint32_t HAL_NVIC_GetPriorityGrouping (void)</code> Gets the priority grouping field from the NVIC Interrupt Controller.
<code>void HAL_NVIC_GetPriority (IRQn_Type IRQn, uint32_t PriorityGroup, uint32_t * pPreemptPriority, uint32_t * pSubPriority)</code> Gets the priority of an interrupt.
<code>void HAL_NVIC_SetPendingIRQ (IRQn_Type IRQn)</code> Sets Pending bit of an external interrupt.
<code>uint32_t HAL_NVIC_GetPendingIRQ (IRQn_Type IRQn)</code> Gets Pending Interrupt (reads the pending register in the NVIC and returns the pending bit for
<code>void HAL_NVIC_ClearPendingIRQ (IRQn_Type IRQn)</code> Clears the pending bit of an external interrupt.
<code>uint32_t HAL_NVIC_GetActive (IRQn_Type IRQn)</code> Gets active interrupt (reads the active register in NVIC and returns the active bit).

4.3.1 Cosas pendientes

Condiciones de carrera. Monitores/secciones críticas.

4.4 El periférico EXTI y las interrupciones

Este apartado pretende ilustrar todo el proceso de programación y aprovechamiento típico del sistema de interrupciones en un periférico sencillo. Para ello, se ha elegido el periférico *External interrupt/event controller (EXTI)* de los microcontroladores STM32F4.

La figura 4.3 muestra un diagrama de dicho periférico.

4.4.1 Funcionalidad

El EXTI consiste en una serie de detectores de flancos capaces de generar eventos o interrupciones. Estos detectores están conectados a pines de los puertos o a elementos como la alarma del reloj de tiempo real, el evento de despertar del puerto Ethernet, etc. En [STM32F4-reference-manual] se describe ampliamente este elemento.

EXTI se puede configurar para que genere evento/interrupción cuando detecta flanco de subida, de bajada o ambos. Para su programación, las bibliotecas HAL ocultan en parte su configuración asociándola a la programación del dispositivo objetivo. Para enterderlo, por ejemplo, GPIO, determinadas combinaciones de configuración de los pines GPIO harán que el HAL configure automáticamente también este periférico.

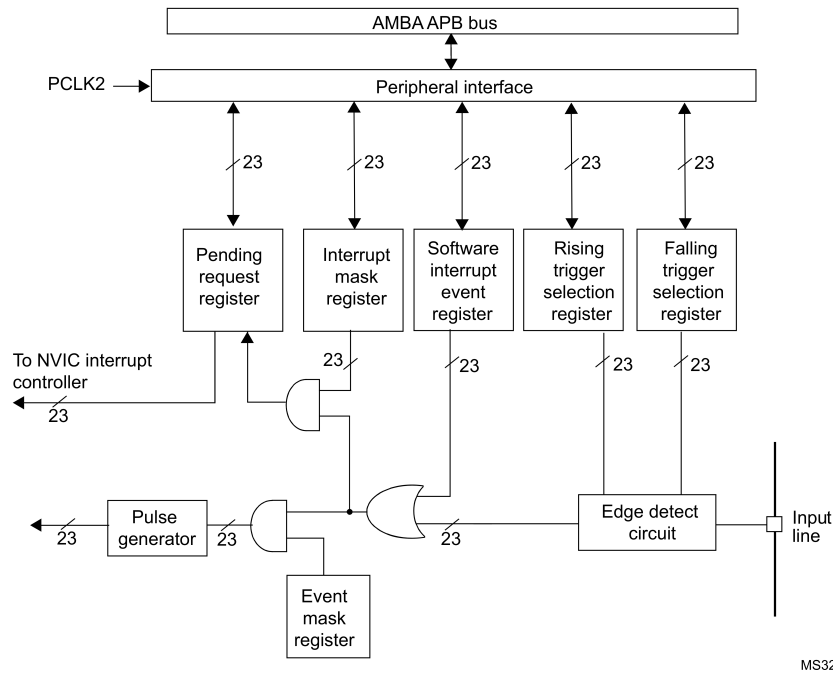


Figura 4.3: Diagrama del periférico EXTI.

4.4.2 Configuración de interrupciones en pines GPIO

Debido a la gran cantidad de pines de GPIO (más de 160 en algunos encapsulados), cada bit de un puerto está asociado a un EXTI particular (o línea), así, todos los Px0 están asociados a EXTI0, los Px1 a EXTI1 y así sucesivamente. Además, solo un Pxy puede estar asociado a su correspondiente EXTy pues emplean una configuración en multiplexor mostrada en la figura 4.4 están asociados a un multiplexor.

Como ejemplo, imagínese que se desea que el pulsador azul de la Discovery genere una interrupción asociado al sistema EXTI. Como el pulsador está conectado al pin PA0, se debe seleccionar obligatoriamente EXTI0 para este propósito.

Para configurar todo el sistema, se propone el siguiente listado:

```
#include "stm32f4xx_hal.h"

int main(void)
{
    // ...
    GPIO_InitTypeDef port;

    /* PASO 1: Disable interrupt */
    HAL_NVIC_DisableIRQ(EXTI0_IRQn);

    /* PASO 2: Configure target device */
    /* Enable GPIOA clock */
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /* Configure PA0 pin as input floating */
```

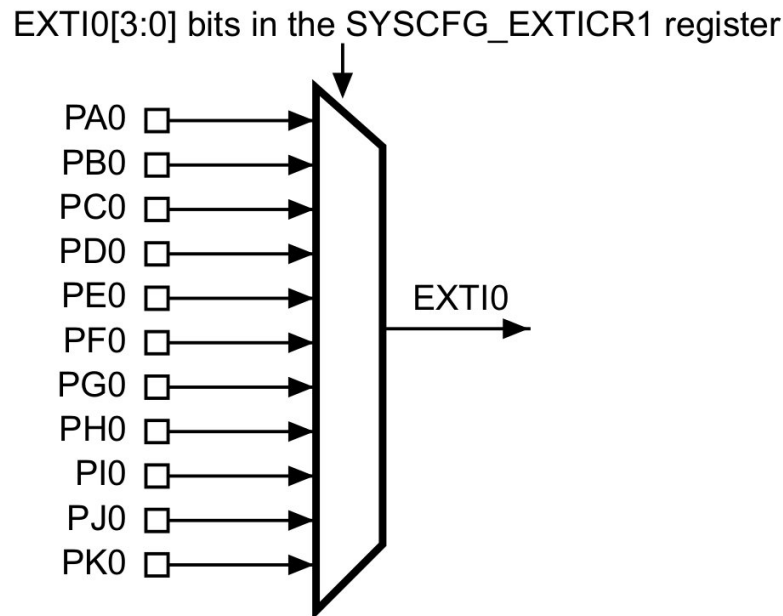


Figura 4.4: Multiplexor GPIO para seleccionar puerto asociado EXTI.

```
port.Pin = GPIO_PIN_0;
port.Mode = GPIO_MODE_IT_FALLING;
port.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &port);

/* PASO 3: Configure NVIC related interrupt */
HAL_NVIC_SetPriority(EXTIO_IRQn, 2, 0);

/* PASO 4: Enable interrupt */
HAL_NVIC_EnableIRQ(EXTIO_IRQn);

/* SUPER-LOOP */
while (1)
{
    //forever tasks!
}
```

Obsérvese como se siguen los pasos típicos de configuración de un sistema de interrupciones. En el paso 2 en el que se configuran los periféricos que, en este caso, se corresponde con un pin. Finalmente se configura la interrupción en el paso 3 y se habilita en el paso 4.

Tabla 4.3: Macros y funciones del HAL para EXTI y GPIO (posiblemente internas).

<code>__HAL_GPIO_EXTI_GET_IT(__EXTI_LINE__)</code>
Comprueba si en determinado flag de petición está establecido o no. Se le pasa un pin.
<code>__HAL_GPIO_EXTI_CLEAR_IT(__EXTI_LINE__)</code>
Limpia un determinado flag de petición de interrupción. Se le pasa un pin..
<code>void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin);</code>
Manejador predefinido del HAL de St. Se le deba pasar el pin que genera la interrupción.
<code>weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);</code>
Función de callback predefinida. El usuario la reescribe.

4.4.3 El servicio de interrupción

Para dar servicio a la interrupción, será necesario proporcionar el manejador. Como ya se indicó, el manejador será una simple función C con un nombre predefinido. En este caso, la función se deberá llamar `void EXTI0_IRQHandler(void)` para EXTI0 y debería colocarse en el archivo `stm32f4xx_it.c`.

En el siguiente listado se muestra una propuesta de manejador.

```
void EXTI0_IRQHandler(void)
{
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_0) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);
        // here your task, for example LED_On();
    }
}
```

En contraposición al ejemplo del Busfault, este manejador necesita comprobar un determinado bit para saber quién ha producido la petición y, una vez localizado, borrar el bit de petición. La razón para hacerlo así es que hay varias líneas que pueden hacer la petición y confluyen en una sola interrupción.

Este patrón será típico en la mayoría de interrupciones asociadas a periféricos. Se denomina *borrado por software* del flag de petición en contraposición a los casos en que el flag de petición es borrado automáticamente al servirse la interrupción. Para el caso de EXTI combinado con GPIO, el HAL incluye las macros (similares a funciones) de la tabla 4.3.

A la hora de hacer la tarea encomendada, hay distintos planteamientos con sus ventajas e inconvenientes.

Haciendo el trabajo en el manejador

En el propio manejador puede ejecutarse la acción deseada. Recuérdese que este aspecto es tremendamente crítico para la correcta operación del sistema, pues la interrupción detiene momentáneamente el resto del sistema.

En un microcontrolador como los ARM Cortex-M se pueden ejecutar aplicaciones complejas con muchísimas interrupciones que se interfieren unas a otras y al bucle principal, por tanto, el código a ejecutar en el manejador debe ser la más rápido y eficiente posible, con lo que se intenta minimizar este efecto.

Como ejemplo, el siguiente manejador permite cambiar el estado de un LED.

```
#include <led.h>

// ...

void EXTI0_IRQHandler(void)
{
    static uint8_t estado = 0;

    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_0) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);

        if (estado == 0) {
            estado = 1;
            LED_On();
        } else {
            estado = 0;
            LED_Off();
        }
    }
}
```

División en dos niveles

Una práctica habitual con las interrupciones consiste en dividir una tarea en dos niveles. Para ello, la interrupción atiende lo inmediato y anota que ha habido una petición, y el bucle de tareas periódicas atiende, por consulta, el trabajo pesado. Se trata de combinar las ventajas de la interrupción con la técnica de *polling*.

Como ejemplo, imagínese un reproductor de CDs con sus botones “play”,... . Para la llamada “experiencia de usuario”, no hay nada más frustrante que una botonera medio sorda, por lo que es prioritario atender a los botones; por otra parte, atender continuamente a los botones consume un precioso tiempo de CPU que es necesario invertir en otras cosas.

Para implementar esta idea, véase el siguiente manejador en el que la única acción será cambiar el estado de la variable.

```
extern uint8_t _button_state;

void EXTI0_IRQHandler(void)
```

```
{
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_0) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);

        _button_state = 1; // button pressed
    }
}
```

Destacar el uso ineludible del atributo `volatile` en la variable compartida.

El atributo `extern` para la variable permite el acceso a una misma variable desde varios módulos. Esta es una práctica de programación poco recomendable, pero a la que pueden obligar las necesidades de este tipo de aplicaciones.

En el siguiente listado se muestra un fragmento de programa principal en el que se aportan funciones para usar una variable compartida entre el bucle principal y un manejador de interrupción. Las modificaciones de la variable se comprueban, cuando se puede, en el superbucle.

```
#include <stdio.h>
#include "stm32f4xx_hal.h"
#include <led.h>

volatile uint8_t _button_state = 0;

uint8_t button_GetPressed(void)
{
    return _button_state;
}

void button_Reset(void)
{
    _button_state = 0; // not pressed
}

int main(void)
{
    // ...

    while (1)
    {
        // check if the button was pressed in the past
        if (button_GetPressed()) {
            printf("Se ha pulsado el botoncito\n");
            button_Reset();
        }

        // simulate other tasks
        for (i=0;i<100000000;i++) {};
        printf("Tarari!\n");

    }; //forever i que no siga res!
}
```

4.4.4 El servicio de interrupción con “callback”

Para facilitar la tarea del programador (en principio), el HAL de St propone un mecanismo típico de programación llamado “callback” que consiste en que el HAL proporciona las funciones encargadas de gestionare la interrupción (tenemos la faena hecha) y despues se llama a una función con nombre prefijado.

Véase con el ejemplo de EXTI.

El manejador sería ahora:

```
void EXTI0_IRQHandler(void)
{
    // St's HAL proposes to call this predefined function
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

En concreto, este manejador sigue la sugerencia del HAL de St de llamar a una función predefinida que ya está lista para hacer el trabajo sucio de comprobar qué línea es la que ha generado la interrupción y, tras la acción asociada al servicio, limpiar el flag indicador correspondiente. El siguiente listado muestra el fragmento del HAL al que se llama (que no tenemos que escribir nosotros):

```
/**
 * @brief This function handles EXTI interrupt request.
 * @param GPIO_Pin: Specifies the pins connected EXTI line
 * @retval None
 */
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    /* EXTI line interrupt detected */
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}

/**
 * @brief EXTI line detection callbacks.
 * @param GPIO_Pin: Specifies the pins connected EXTI line
 * @retval None
 */
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    /* NOTE: This function Should not be modified, when the callback is needed,
    the HAL_GPIO_EXTI_Callback could be implemented in the user file
    */
}
```

Al final del listado se observa una función definida con `__weak`. Esta es otra ayuda del HAL que pretende que la tarea a realizar se escriba con una función con dicho nombre. Ahora queda rellenar la función de callback con la tarea encomendada. Dicha función hay que escribirla en otra parte, no hay que tocar el HAL.

Por ejemplo, para el parpadeo del LED:

```
#include <led.h>

// ...

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    static uint8_t estado = 0;
    if(GPIO_Pin == GPIO_PIN_0)
    {
        if (estado == 0) {
            estado = 1;
            LED_On();
        } else {
            estado = 0;
            LED_Off();
        }
    }
}
```

Y para la versión con división de niveles:

```
extern uint8_t _button_state;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == GPIO_PIN_0) {
        _button_state = 1; // button pressed
    }
}
```

Capítulo 5

Contadores y temporizadores

5.1 Introducción

La mayor parte de las aplicaciones para microcontrolador necesitan contar eventos o generar retardos de gran precisión. Por software es posible realizar retardos de cierta precisión y contar eventos, pero la mayor parte del potencial de la CPU se invertiría en éste cometido, y no dejaría tiempo para realizar otras acciones o complicando el diseño de la aplicación.

Los contadores/temporizadores o, en la jerga, “*timers*”, son periféricos hardware que suplen este defecto, descargando de un trabajo poco grato al micro. Es tal la importancia de este tipo de dispositivos que los microcontroladores más avanzados incluyen decenas de ellos o, incluso, un coprocesador dedicado a gestionarlos.

Estos dispositivos permiten, entre otras cosas, medir anchos de pulso de señales, generar señales digitales, contar impulsos, provocar acciones periódicas, implementar relojes de tiempo real, generar el ritmo para comunicaciones, comparación/captura, generación PWM (modulación por ancho de pulso) para control digital directo, etc.

Esta capítulo pretende ilustrar el uso de los timers desde el punto de vista de sus posibles aplicaciones más que una exposición de características de un periférico dado. Como en capítulos previos, es indispensable utilizar referencias externas para complementar la información sobre estos dispositivos.

Bla, bla ... (estructura)

5.2 Los *timers* en genérico

La configuración de los contadores/temporizadores suelen seguir un esquema genérico que se representa en la figura 5.1.

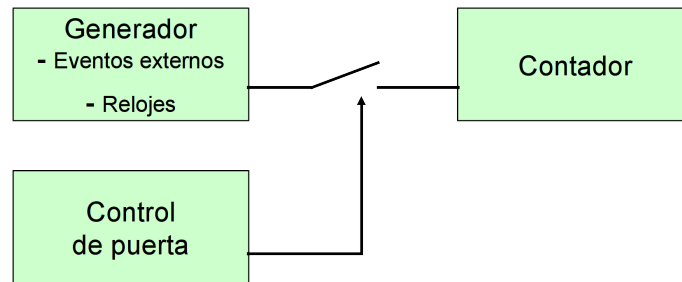


Figura 5.1: Bloques que forman un timer.

El bloque de *generación* es el origen de las señales digitales que han de ser contabilizadas y que, en general, serán flancos que deberán seguir ciertas restricciones (duración mínima a un determinado nivel, etc.). Si la señal a contabilizar procede de un reloj/oscilador, entonces la cuenta se modifica a un ritmo conocido y se habla de “temporizador”, siendo su propósito el de medir tiempo. Si las señales son esporádicas y sin patrón temporal predefinido, entonces se habla de “contador”, pues el propósito será contabilizar cosas.

La llegada de eventos al registro de cuenta puede habilitarse/deshabilitarse de muy distintas formas, siendo opciones habituales el control por software o mediante señales externas. A esta parte se la denomina “control de puerta”.

Finalmente se tiene el contador propiamente dicho, cuyo valor variará en función de los eventos que lleguen (por ejemplo, que se incrementándose o decrementándose). Además, suelen incluirse características adicionales para comparar el valor de la cuenta con un patrón (comparación/captura) y para realizar recargas automáticas de la cuenta.

Por supuesto, un timer será capaz de generar señales al subsistema de interrupciones. Para esta característica suelen ser casos habituales el desborde de un timer, el cambio en la señal de puerta, la coincidencia con un patrón, etc.

5.3 SysTick, el contador común a los Cortex-M

La especificación del núcleo ARM Cortex-M incluye un timer llamado *SysTick* que implementan casi todos los fabricantes.

Es muy sencillo y está pensado para usarlo como base de tiempos en planificadores de tareas, así que es perfecto para introducir el concepto de temporización y su aprovechamiento.

Este periférico usa un contador descendente (resta 1 por evento) con un registro de 24 bits contiene la cuenta actual. Cuando la cuenta está en 0 y llega un nuevo evento, el registro de cuenta se recarga con un valor de “precarga” establecido por programa y seguirá descontando a partir de ese valor. La figura 5.2 lo representa.

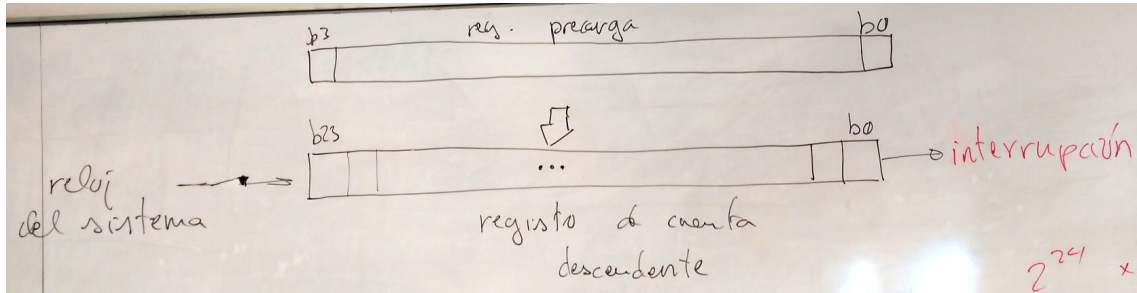


Figura 5.2: Bloques del SysTick

El ritmo de descuento se deriva del reloj del sistema. Por ejemplo, si se usa directamente el reloj del sistema y este es de 180 MHz, el tiempo de cada tick sería:

$$T = \frac{1}{F} = \frac{1}{180 \times 10^6}$$

Por tanto, para medir un lapso L de 1 ms, se tendría que usar una recarga de:

$$recarga = \frac{L}{T} = \frac{1 \times 10^{-3}}{\frac{1}{180 \times 10^6}} = 5,5 \times 10^{-9} = 5,5 ns$$

Y escrito de manera más amigable para la aritmética entera del computador sería:

$$recarga = \frac{180 \times 10^6}{1 \times 10^3} = 180000$$

Actividad:

- Cálculalo para 10 ms.

Actividad:

- Calcula el tiempo en el que desborda el contador para 180 MHz.

Teniendo en cuenta que el contador es de 24 bits, el tiempo máximo medible con la velocidad de reloj de 180 MHz sería de unos 93 ms. Es decir $T = 2^{24} * (1/180 * 10^6)$

Tabla 5.1: Funciones para gestionar SysTick.

```
uint32_t HAL_SYSTICK_Config (uint32_t TicksNumb)
```

Usar como valor de recarga el valor pasado como parámetro, configurar la interrupción correspondiente

5.3.1 Biblioteca HAL

La biblioteca HAL Stm32Cube ofrece la función de la tabla 5.1 para configurar SysTick y dejarlo listo para sus principales utilidades.

Una vez llamada esta función, el sistema empezará a generar interrupciones SysTick al ritmo establecido. Por tanto, será necesario proveer un manejador en el archivo correspondiente.

Para facilitar la programación, en un programa CMSIS habrá una variable global cuya definición es

```
uint32_t SystemCoreClock;
```

y cuyo contenido es la frecuencia de reloj del core en Hz. Por ejemplo, si se desea un ritmo de *tick* de 1 ms, bastaría con aplicar el cálculo de la siguiente manera:

$$recarga = \frac{L}{T} = \frac{1 \times 10^{-3}}{\frac{1}{SystemCoreClock}}$$

Y escrito para C sería:

$$recarga = \frac{SystemCoreClock}{1 \times 10^3}$$

Que lo trasladamos a la función de la siguiente manera:

```
HAL_SYSTICK_Config (SystemCoreClock/1000);
```

Para tener otro ejemplo, supongamos ahora 5 ms, entonces tendríamos:

$$recarga = \frac{L}{T} = \frac{5 \times 10^{-3}}{\frac{1}{SystemCoreClock}}$$

$$recarga = \frac{SystemCoreClock}{200}$$

```
HAL_SYSTICK_Config (SystemCoreClock/200);
```

Actividad:

- Calcúlalo para 20 ms.

Actividad:

- Calcúlalo para 1 s. Y no te despistes.

5.3.2 Midiendo el paso del tiempo

Una de las utilidades de SysTick es medir el paso del tiempo. La manera universal de hacerlo consiste en usar una variable contador que se incremente en cada *tick* de SysTick que, traducido, significa en cada interrupción. Dicha variable se puede procesar después para calcular datos de calendario (horas, minutos, segundos, día, mes, etc.).

El siguiente fragmento de programa muestra el manejador para SysTick con una manera para tener la variable global de cuenta:

```
volatile uint32_t TicksCount = 0;

void SysTick_Handler(void)
{
    TicksCount++;
}
```

Dado el tipo de variable utilizada, la capacidad de mantener el tiempo a un ritmo de 1 ms sería de unos 49 días. A partir de ahí, la variable desborda.

(NOTA: Siento decir que cambiarlo por uint64_t da lugar a una cosa llamada “condición de carrera” y que no puedo explicar este año).

Desde cualquier módulo se puede acceder a esta variable para hacer una medición de paso de tiempo. Por ejemplo:

```
extern volatile uint32_t TicksCount;

// ...

uint32_t toma_tiempo;

toma_tiempo = TicksCount;
// hacer un monton de cosas
toma_tiempo = TicksCount - toma_tiempo;

printf("Han pasado %u ticks\n", toma_tiempo);
```

Pregunta:

- ¿Cuál es el tiempo mínimo que se puede medir? ¿Qué error tiene?.

5.3.3 Haciendo pausas de precisión

Uno de los usos habituales de SysTick es su aprovechamiento para realizar pausas de precisión. Para ilustrar su uso, se proponen las siguientes funciones típicas:

```
#ifndef DELAY_H
#define DELAY_H

#include <stdint.h>

void delay_Init(void);
void delay_ms (uint32_t milliseconds);

#endif
```

Para implementar esta funcionalidad, se implementa el siguiente módulo delay.c:

```
#include "stm32f4xx_hal.h"
#include "delay.h"

// variable defined in stm32f4xx_it.c
extern volatile uint32_t TicksCount;

void delay_Init(void)
{
    // 1 milisecond per tick
    //SysTick_Config(SystemCoreClock / 1000);    // notacion CMSIS
    HAL_SYSTICK_Config(SystemCoreClock / 1000);
}

void delay_ms (uint32_t milliseconds)
{
    uint32_t ticks_end;

    ticks_end = TicksCount + milliseconds;
    while (TicksCount < ticks_end) {};
}
```

Actividad:

- Adapta delay() para que el tick sea cada 0,5 ms.

5.3.4 Tareas periódicas

Lo que se ha hecho en el apartado anterior se denomina una “tarea periódica”. Se puede aprovechar la funcionalidad de SysTick para hacer más tareas siguiendo distintas aproximaciones.

Lo más inmediato es colgar la tarea deseada en el propio manejadro de interrupciones. Véase el siguiente ejemplo.

```
volatile uint32_t TicksCount = 0;

void SysTick_Handler(void)
{
```

```
    TicksCount++;

    if ((TicksCount % 20)==0) {
        // task every 20 ticks
    }
}
```

Ya se ha hablado de las ventajas e inconvenientes de esta aproximación, así que aconsejarla solo si se tiene muy claro su beneficio.

También se puede aplicar aquí la aproximación a dos niveles, para ello, en SysTick se marca y en el super-bucle se comprueba.

El siguiente listado es una posible implementación en el manejador:

```
volatile uint32_t TicksCount = 0;

void SysTick_Handler(void)
{
    TicksCount++;

    if ((TicksCount % 20)==0) {
        task_pepe_activated = 1;
    }
}
```

Y aquí cómo se atendería en el super-bucle:

```
//...

while (1)
{
    // tasks polling
    if (task_pepe_activated)
    {
        // do task related work
        task_pepe_activated = 0;
    }
}
```

5.3.5 NO MIRAR: Cómo es

Esto será un tema más avanzado. En el manual de programación del Cortex-M4 de St [STM32_Program_M4] se describe completamente la funcionalidad de este timer.

Para explicar aquí el funcionamiento de SysTick se usará su *mapa de registros* según la figura 5.3, con lo que se tiene la excusa perfecta para introducir cómo se muestran los periféricos al software. En ARM Cortex-M, los periféricos exponen sus registros como direcciones de memoria normal; basta con acceder a esas posiciones para leer/escribir en los periféricos (açò o manejarè al apartart programació i afegiré variable volatile per accedir a pel).

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x00	STK_CTRL Reset Value	Reserved														0	COUNTFLAG	Reserved												1	CLKSOURCE	0	TICK INT	1	ENABLE	0
0x04	STK_LOAD Reset Value	Reserved									RELOAD[23:0] 0																									
0x08	STK_VAL Reset Value	Reserved									CURRENT[23:0] 0																									
0x0C	STK_CALIB Reset Value	Reserved									TENMS[23:0] 0																									

Figura 5.3: Mapa de registros y valores de reset de SysTick. Dirección base E000E010h

Este periférico usa un contador descendente (resta 1 por evento) de 24 bits al que se puede acceder a través del registro **STK_VAL**, que contendrá la cuenta actual. Cuando la cuenta está en 0 y llega un nuevo evento, el registro de cuenta se recarga con el valor del registro **STK_LOAD** y sigue descontando a partir de ese valor.

Mediante ciertos bits del registro **STK_CTRL** se puede controlar el funcionamiento y saber el estado del contador. Por ejemplo, la puesta a 1 del bit **ENABLE** hace que se recargue la cuenta y se inicie el descuento. Cuando la cuenta llegue a 0, se pondrá a 1 el bit **COUNTFLAG** y, si está a 1 el bit **TICK_INT**, entonces se producirá la petición de interrupción SysTick.

El ritmo de descuento está controlador por el bit **CLOCKSOURCE**, pudiendo ser el reloj del core (velocidad del AHB) o AHB/8. Como el cometido de este contador es medir tiempo, la manera de hacerlo es calcular que valor se introduce en el registro **RELOAD** para medir la fracción de tiempo deseada.

Por ejemplo, supóngase el microcontrolador STM32F407VG de la tarjeta Discovery configurado con reloj del núcleo a 168 MHz y SysTick configurado para usar esa fuente. Si se quisiese medir 1 ms, se tendría que cargar en **RELOAD** el valor:

$$RELOAD = (168 * 10^6) * (1 * 10^{-3})$$

Actividad:

- Cálculalo para 10 ms.

Teniendo en cuenta que el contador es de 24 bits, el tiempo máximo medible con la velocidad de reloj de 168 MHz sería de unos 0,99 ms.

Una vez dominada la técnica para hacer los cálculos temporales, se puede pasar a la programación. Si se quisiese programar a pelo el SysTick, se seguirían estos pasos:

- Desactivar el contador. **ENABLE=0**.
- Cargar el valor de **RELOAD**.

- Escribir cualquier valor en la cuenta para que se ponga a 0.
- Configurar los registros de control y estado, incluyendo la activación.

Se puede hacer a pelo, pero es más fácil apoyarse en las bibliotecas CMSIS.

Capítulo 6

Programación en C para ARM Cortex-M

6.1 Introducción

El lenguaje C es, con diferencia el más extendido en el desarrollo para sistemas embebidos basados en microcontrolador y, por tanto, la elección más adecuada. En cualquier caso, el lenguaje C está repleto de problemáticas que incidirán negativamente en el desarrollo si no se toman una serie de precauciones.

El objetivo de este capítulo es presentar algunas reglas, consejos y particularidades que permiten usar C adecuadamente en un microcontrolador con garantías de éxito. Téngase en consideración que un microcontrolador dispone de pocos recursos de memoria en comparación con los computadores de propósito general, y que el tipo de aplicaciones destino es bien distinto por lo que se deberá tener un cuidado exquisito en gestionar adecuadamente los recursos.

Se parte del que el lector tiene conocimientos de C, revisándose solo ciertos aspectos fundamentales y, después, yendo directamente a los aspectos concretos sobre desarrollo embebido.

(Al terminar este capítulo, faltaría redactar intro con resumen y objetivos). (¿Hablar sobre C++?)

6.2 El desarrollo es “cruzado”

Las aplicaciones para microcontrolador emplean herramientas *cruzadas*, lo significa que la plataforma destino (el microcontrolador) no es la plataforma de desarrollo. Para el desarrollo de las aplicaciones para microcontrolador se emplearán herramientas para computador de propósito general que volcarán-depurarán-etc. sobre el microcontrolador destino.

La figura 6.1 representa esta idea. En un computador se introduce el código, se “construye” y, finalmente, se vuelve y depura en la plataforma destino. La plataforma destino no tiene porque ser el producto final, sobre todo en las etapas iniciales de desarrollo.



Figura 6.1: En los sistemas embebidos, el desarrollo es cruzado

6.3 Del código al ejecutable

La experiencia del autor es que muchos usan C, pero no tienen claro las etapas que llevan de un programa escrito en C al ejecutable final. En el caso de sistemas embebidos es importante tener claro este aspecto, pues muchas veces deberán juntar y controlar todas las piezas a mano.

La figura 6.2 representa las etapas, que son *preprocesado*, *ikemphcompilado* y *enlazado* (vulgarmente, *ikemphlinkado*).

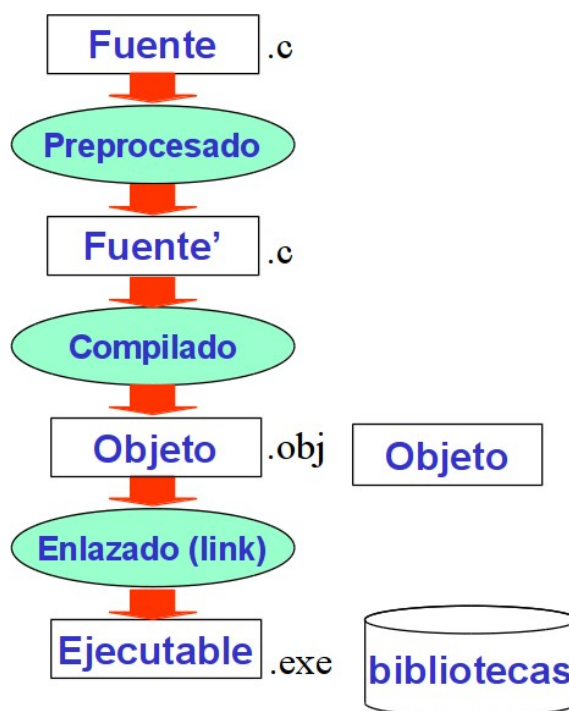


Figura 6.2: Etapas de construcción de un proyecto C.

En la figura 6.3 se representa el efecto del preprocesado, que no es más que un “buscar” y “reemplazar” un texto por otro. En esta etapa, todo lo que empieza por “#” (defines, includes, macros,...) es sustituido por el texto equivalente.

En la figura 6.4 se representan las etapas de compilado y enlazado. En el compilado se convierte a código máquina todas las construcciones de C (asignaciones, bucles, ...), volcándose un *objeto* que contiene números binarios (el código máquina) y ciertos huecos por resolver que se rellenan en la siguiente etapa. En la etapa de enlazado, se incorporan *objetos* externos de los que dependa el código anterior. Estos objetos estarán en otros módulos C previamente compilados o, almacenados en *bibliotecas* (vulgarmente, *librerías*), que no son más que agrupaciones de objetos. Por ejemplo, la función C `printf()` no es C, es una función de biblioteca.

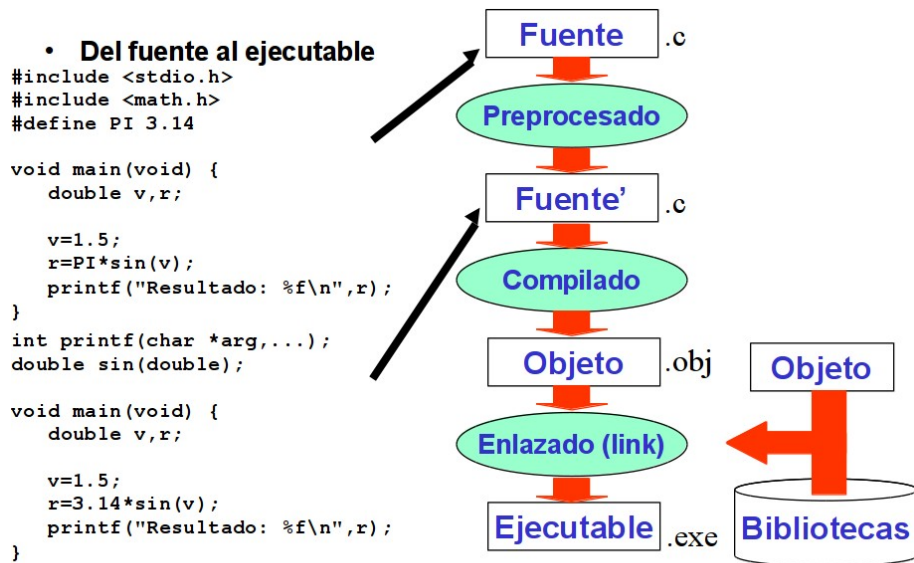


Figura 6.3: Etapa de preprocesado.

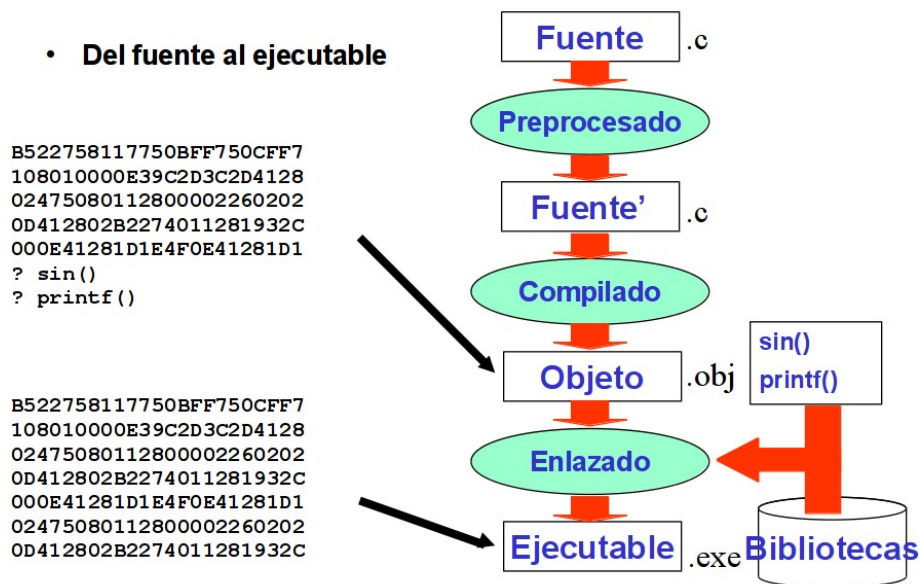


Figura 6.4: Etapa de compilado y enlazado.

6.4 Tratando con datos

6.4.1 Tipos enteros

Uno de los mayores dolores de cabeza de la gente que se dedica a desarrollo embebido es el tamaño de los tipos de datos.

Pregunta: ¿Qué tamaño en bytes tiene un `int`? ¿Qué rango de datos permite? (Se recomienda buscar en Internet).

La implementación del tipo `int` es dependiente de la arquitectura y del sistema operativo, con lo que podemos encontrarnos con aplicaciones para una arquitectura que dejan de funcionar en otra.

Por esta razón, a partir de este momento se recomienda no volver a usar *jamás* un `int` y similares si se pretende tener futuro en los sistema embebidos. En este texto se emplearán los tipos estandarizados y recurrir a tipos estandarizados como la `stdint.h`.

Un grupo concreto de tipos estandarizados que vamos a emplear son los que define el tamaño exacto que se emplea para representar los datos. La figura ?? contiene una lista de estos tipos.

Specific integral type limits					
Specifier	Signing	Bits	Bytes	Minimum Value	Maximum Value
<code>int8_t</code>	Signed	8	1	-2^7 which equals -128	$2^7 - 1$ which is equal to 127
<code>uint8_t</code>	Unsigned	8	1	0	$2^8 - 1$ which equals 255
<code>int16_t</code>	Signed	16	2	-2^{15} which equals $-32,768$	$2^{15} - 1$ which equals $32,767$
<code>uint16_t</code>	Unsigned	16	2	0	$2^{16} - 1$ which equals $65,535$
<code>int32_t</code>	Signed	32	4	-2^{31} which equals $-2,147,483,648$	$2^{31} - 1$ which equals $2,147,483,647$
<code>uint32_t</code>	Unsigned	32	4	0	$2^{32} - 1$ which equals $4,294,967,295$
<code>int64_t</code>	Signed	64	8	-2^{63} which equals $-9,223,372,036,854,775,808$	$2^{63} - 1$ which equals $9,223,372,036,854,775,807$
<code>uint64_t</code>	Unsigned	64	8	0	$2^{64} - 1$ which equals $18,446,744,073,709,551,615$

Figura 6.5: Los tipos enteros

Para usar estos tipos de datos deberos incluir la cabecera `stdint.h`.

Actividad: Declara la variable más adecuada

- Numero de día del mes
- La cantidad de petardos de una caja
- Kilómetros hasta la luna
- La temperatura actual con dos decimales (hay trampa)

Actividad: Desarrolla un programa completo que calcule la media de un vector de 30000 elementos con valores entre 0 y 7000. Debes declarar el vector con el tipo de elemento más adecuado, y crear los bucles que hacen los cálculos. Saca el valor por pantalla con `printf()`.

(??Calificadores de tipos 3UL)

6.4.2 Tipos enumerados

Los tipos enumerados pretenden resolver el dilema entre la facilidad para representar números enteros en un computador y la capacidad humana de representar mentalmente palabra “nemotécnicas”.

Imagínese que se quieren representar en un computador el estado de una válvula digital. Una manera adecuada desde el punto de vista del computador y del humano sería la siguiente:

```
typedef enum {VALVE_OPENED, VALVE_CLOSED} TValveState;
```

Lo que se ha hecho es crear una lista de palabras para representar distintos estados de un concepto. En el ejemplo anterior se va más allá creando un nuevo tipo de datos que hará también más fiable la etapa de comprobación del código a la hora de compilar.

A partir de la creación del nuevo tipo, se puede usar como se muestra a continuación:

```
TValveState state;  
  
state = VALVE_OPENED;  
...  
if (state == VALVE_CLOSED) {  
    printf("La válvula está cerrada.\n");  
}
```

En C, un enumerado no es más que un número entero disfrazado.

Crea un tipo enumerado para representar los días de la semana. Crea una *Actividad:* variable de ese tipo y asígnale el lunes.

Muestra por pantalla el valor de la variable anterior.

Tabla 6.1: Los tipos de datos habituales para operar en coma flotante

float	Simple precisión
double	Doble precisión

6.4.3 Tipos en coma flotante

Los tipos de datos en coma flotante (float, double, ...) pueden emplearse en los microcontroladores, pero es importante tener en cuenta que la mayoría de los ellos no disponen de hardware específico para ejecutar operaciones en coma flotante. Como consecuencia, emplear operaciones en coma flotante implica la incorporación de bibliotecas de apoyo.

Por ejemplo, añadir el siguiente código a nuestra aplicación implicará que este crezca varios kilobytes debido a la incorporación de estas bibliotecas.

```
float a;  
a = 3.2;  
a = a + 1.3;
```

Además de ocupar más memoria de programa, el tiempo de ejecución de las operaciones será varios órdenes de magnitud mayor que las operaciones con datos enteros. En resumen, usar datos en coma flotante tiene sus ventajas e inconvenientes, y es el desarrollador el que debe considerar su uso.

Actividad: Dada la siguiente declaración de variable en C. Conseguir que en `.a` se deposite la división de 3 entre 5. El resultado debe tener decimales.

```
float a;
```

Solución mala típica:

```
a = 3/5;
```

Se trata de una división entera.

Solución correcta:

```
a = 3.0/5.0;
```

Otra precaución importante es la elección adecuada del tipo, tanto en la definición de variables como en las expresiones. La elección dependerá de las necesidades de la aplicación a desarrollar. En la tabla 6.1 están los tipos.

Es fácil desarrollar código inadecuado en C que emplee coma flotante debido a que, por defecto, C supone que los números en coma flotante son siempre double. Por ejemplo, el siguiente listado nos es adecuado para un microcontrolador debido a que las constantes son "doubles" fuerzan operaciones más costosas y truncamientos en las asignaciones.

```
float a;
```

```
a = 3.0;
a = a + 2.7;
```

Para que el código esté bien adaptado a los cálculos con "float", de deberán emplear calificadores de tipo. Por ejemplo, la corrección al anterior listado quedaría,

```
float a;
a = 3.0F;
a = a + 2.7F;
```

Ampliación de información referente al coma flotante. (IEEE754)

Actividad



Figura 6.6: EL PID manteniendo el ralentí de la moto

La ECU de un ciclomotor emplea un regulador PID para mantener al ralentí con la siguiente fórmula:

$$respuesta = k_p * error_{posicin} \quad (6.1)$$

Expresar en C la expresión siendo tanto los errores, la respuesta y el tiempo de muestreo (tm), variables y $k_p = 3.2$, $k_d = 95.3$ y $k_i = 0.001$

Solucion:

```
#define Kp (3.2F)
#define Kd (95.3F)
#define Ki (0.001F)

float ep, ed, ei;
float respuesta, tm;

main()
{
    tm=50.3F;
    respuesta=kp*ep+kd*ed+ki*(ei+(ep+tm));
}
```

6.4.4 Algunos atributos fundamentales

El atributo `const`

Una limitación habitual de los sistemas embebidos es la cantidad de memoria disponible, tanto para variables como para código. El atributo de datos `const` puede ser de gran ayuda para gestionar óptimamente los espacios.

Para introducir la idea, supóngase el siguiente fragmento de programa en el que se define e inicializa una variable,

```
int32_t ratio = 3597;
```

Al compilar este fragmento se creará una constante con el valor 3597 en la zona de memoria de constantes y se generará código máquina en la zona de programa para copiar el valor constante a la variable real durante la ejecución. Todo esto se traducirá en espacio ocupado en la zona de constantes/programas, espacio en la zona de variables y tiempo para copiar el valor a la variable.

Cuando una variable o unos datos son constantes, es decir, no van a variar en el ciclo de vida del programa, puede ser aconsejable forzar la colocación, tanto de la variable como su valor, en la zona de constantes. Con este planteamiento se puede ahorrar mucho espacio en la memoria de programa/constantes.

En C, la manera de lograr que una variable no se copie al espacio de datos es usar el atributo `const`. El siguiente fragmento ilustra la idea para el caso anterior:

```
const int32_t ratio = 3597;
```

El precio que se paga es que la variable no se podrá modificar. También es posible que hayan personalizaciones temporales al acceder a dicha variable, pues estará en un espacio distinto y suele ser habitual que las memoria FlashROM sean más lentas que las RAM.

El siguiente ejemplo crea un vector con unos datos preestablecidos, cosa que es muy habitual en las aplicaciones para microcontrolador. Gracias al atributo `const`, se evita el copiado a la memoria de datos, ahorrándose una cantidad apreciable de memoria para variables.

```
const uint16_t heat_table[]={327,2935,9512,536,9254,554,
                             9769,2,0,2314,57477,345,1900,1794};
```

El atributo `extern`

En C, este atributo permite compartir una variable entre distintos módulos. La idea es que la variable se define una sola vez y se declara también en los módulos que la vayan a emplear.

Por ejemplo, definimos la siguiente variable global en un archivo C. Obsérvese que se a aprovechado para asignarle un valor.

```
float acceleration = 37.5;
```

Y se puede declarar en cualquier sitio donde se quiera emplear. Lo correcto es que se declare en un archivo de cabecera, pero no es obligatorio.

```
extern float acceleration;
```

El módulo que disponga de la declaración podrá acceder a esta variable común.

Hay que evitar su uso en la medida de lo posible. En sistemas embebidos tiene sentido emplearlo en variables a las que haya que acceder eficientemente, por ejemplo, en servicios de interrupción.

Actividad: Crea la variable tick de tipo entero de 32 bits en el módulo clock.c. Añade su definición en la cabecera clock.h

El atributo static

Fer l'ejemple? (no recorde quin)

En C, el atributo static tiene dos usos.

El primer uso es proteger una variable global de manera que solo sea visible al módulo que lo contiene. Por ejemplo,

```
static uint32_t grades;
```

El segundo en usarlo en variables dentro de una función para mantener el valor entre invocaciones. Por ejemplo,

```
float accumulator (float value)
{
    static float my_value = 0.0F;
    my_value += value;

    return my_value;
}
```

La primera vez que se usa la función gumersindo(), se desea inicializar antes
Actividad: un periférico llamando a la función gallataInit(). Desarrolla el fragmento de función que sabe que no se ha llamado a dicha función y la llama

El atributo volatile

Los compiladores actuales son muy inteligentes, y aplican optimizaciones que les permiten acelerar los programas reduciendo el número de accesos a memoria e instrucciones.

En el ámbito de los sistemas embebidos, los sistemas de tiempo real y las aplicaciones concurrentes, esta "inteligencia" puede ser problemática, pues las aplicaciones no harán lo que el programador inexperto espera.

El caso concreto que se pretende ilustrar aquí es el de acceso a variables en memoria. Imagínese el siguiente fragmento de programa:

```
uint32_t i;  
for (i=0; i<1000000; i++) {};
```

Obsérvese que la variable `i` que es utilizada para la cuenta. Esa misma observación la hará el compilador y, con mucha probabilidad, tomará la decisión de hacer un solo acceso a memoria de datos para acceder a la variable `i` y guardársela en un registro de la CPU durante la ejecución de todo el bucle. El efecto será que la aplicación, literalmente, "volará"; y el efecto colateral es que la variable `i` de la memoria no se habrá tocado hasta que termine el bucle.

En la mayoría de casos, estas optimizaciones son adecuadas, pero hay situaciones en las que es adecuado forzar a que el acceso se haga a la verdadera variable.

La misión del atributo `volatile` es forzar (intentar) a que los accesos sean explícitos a la variable. Aplicado al fragmento anterior, quedaría:

```
volatile uint32_t i;  
for (i=0; i<1000000; i++) {};
```

Este bucle se ejecutará ahora mucho más lentamente, pero se estará accediendo siempre a la variable.

Uno de los intereses para utilizar `volatile` es cuando distintas partes de la aplicación pueden acceder concurrentemente a una misma variable. Esto es típico de las interrupciones y de las aplicaciones multitarea.

En la arquitectura ARM, otro caso es el acceso a los periféricos, cuyos registros son ofrecidos como direcciones de memoria. Si se aplicase la optimización, entonces podría ocurrir que no se leyese/escribiese realmente los registros. Con `volatile` se evita el problema.

Es tal la importancia de esto, que el estándar CMSIS usa la recomendación MISRA-C de incluir las siguientes definiciones para simplificar su uso.

Calificador E/S MISRA-C	tipo ANSI C	Acceso
#define __I	volatile const	solo lectura
#define __O	volatile	solo escritura
#define __IO	volatile	lectura/escritura

En este libro se utilizarán las construcciones ANSI C, pues introducir particularidades del ámbito de automoción o aerosespacial complicarían la comprensión del texto.

(L'any que ve, ficar exemple d'accés a perifèric a pel.).

El atributo weak

Fer l'exemple

6.5 Tablas “look-up”

Los sistemas embebidos utilizan una técnica llamada tablas “look-up” con el propósito de almacenar datos que son utilizados como resultado de una función matemática. Esta manera de proceder suele ser eficiente en cuanto a tiempo de ejecución.

Básicamente, una tabla “look-up” pretende sustituir una función $y = f(x)$ por un acceso a un vector o matriz de C del tipo `y = tabla[x]`. Para lograrlo, en la posición `x` de la tabla se precalcula y almacena el resultado de evaluar $y = f(x)$.

Como ejemplo, imagínese que se quiere calcular rápidamente la función seno. Para ello, se puede emplear una tabla de búsqueda en la que están precalculados, de 1 en 1 grado, los valores del primer cuadrante de la función seno. En la tabla, se almacenarían los valores $\text{seno}(0)$, $\text{seno}(1)$, $\text{seno}(2)$, ... quedando de la siguiente manera:

```
const float sin_table [] = {0.0F, 0.0017F, 0.0034F, ...};
```

Para usar la tabla y calcular, por ejemplo, $\text{sen}(12)$, se haría:

```
valor = sin_table[12];
```

Destacar ahora que no sería factible calcular un valor con parte fraccionaria como $\text{seno}(3,45)$. Para resolverlo, bastaría con aplicar una interpolación.

La aplicación de esta técnica es muy amplia. Véase un ejemplo más cercano imaginando que se desea implementar un decodificador digital 3x8 cuya tabla de verdad se muestra a continuación.

decimal	C	B	A	b7	b6	b5	b4	b3	b2	b1	b0	hexadecimal
0	0	0	0	0	0	0	0	0	0	0	1	01h
1	0	0	1	0	0	0	0	0	0	1	0	02h
2	0	1	0	0	0	0	0	0	1	0	0	04h
3	0	1	1	0	0	0	0	1	0	0	0	08h
4	1	0	0	0	0	0	1	0	0	0	0	10h
5	1	0	1	0	0	1	0	0	0	0	0	20h
6	1	1	0	0	1	0	0	0	0	0	0	40h
7	1	1	1	1	0	0	0	0	0	0	0	80h

La forma más inmediata y eficiente de implementar esta tabla de verdad en un microcontrolador es representar la tabla como una matriz C donde el índice sea el dato de entrada (CBA) y el contenido en ese índice sea el valor de salida correspondiente. Una posible escritura en C sería:

```
const uint8_t deco3x8_table [] = {0x01, 0x02, 0x04,
                                   0x08, 0x10, 0x20, 0x40, 0x80};
```

6.6 Tratamiento bit a bit. Máscaras

Tanto la programación de dispositivos periféricos del computador como la generación/recogida de señales digitales está cargada de números binarios; por tanto, es necesario dominar las técnicas que permiten manipular dichos números a nivel de bit. Esto es fundamental, por ejemplo, en la programación de microcontroladores.

La manipulación a nivel de bit se hace mediante las operaciones del álgebra de Boole AND, OR, X-OR y complementos que, aplicadas sistemáticamente, permitirán obtener 0's, 1's, complementar bits, etc. allí donde se necesite. Al uso de estas operaciones sobre un dato se le suele llamar *máscara*.

A parte de las técnicas, se verá aquí su aplicación práctica desde el lenguaje C.

6.6.1 Representación externa. Representación interna

Antes de entrar en materia, es interesante hacer notar que el computador digital es “digital” como se acaba de decir y, por tanto, representa internamente toda la información como “0”s y “1”s.

La *representación interna* de los números enteros es un caso claro de esta idea. Internamente, un tipo de dato entero es un número binario de una determinada cantidad de bits que se representa habitualmente en *binario natural* para enteros sin signo y en *complemento a dos* para enteros con signo. Hay otras posibles representaciones, pero serán siempre secuencias de “0”s y “1”s.

Para un humano, usar directamente números binarios no está en su naturaleza, así que los lenguajes de programación permiten escribir los números en otras notaciones para facilitar su introducción; se llama a esto *representación externa*. Insistir en que se trata de un artificio; internamente seguirán siendo números binarios.

Aunque la representación externa decimal es la más comprensible para nosotros, también se proveen otras representaciones más o menos abstractas; por ejemplo, se usa la codificación ASCII para representar símbolos (letras,...), y la representación octal y hexadecimal para representar grupos de bits.

Para ilustrar la idea en el lenguaje C, sea el siguiente programa:

```
#include <stdio.h>
int main(void)
{
    int a;

    a = 'A';          // meter el código ASCII de la A
    a = a + 011;       // sumar 11 en octal (9 en decimal)
    a = a + 1;         // sumar 1 en decimal
    a = a + 0xA;       // sumar A en hexadecimal, (10 en decimal)

    printf("En a hay %d en decimal\n", a);
    printf("En a hay %x en hexadecimal\n", a);
    printf("En a hay %c en ASCII\n", a);

    return 0;
}
```

Obsérvese que se “escriben” números empleando distintos artificios, pero internamente todo será binario. Se recomienda probarlo para verificar esta idea, y modificarlo para experimentar.

La elección de una notación u otra es a conveniencia del programador y nunca al contrario.

6.6.2 Representación hexadecimal

Como el lenguaje C estándar no admite la introducción directa de números binarios, la manera recomendada aquí para escribir un número binario es emplear la representación externa *hexadecimal*. Esta representación se diseñó para representar números binarios en un equivalente que representa un grupo de 4 bits.

La siguiente tabla contiene las 16 posibles combinaciones binarias y sus equivalentes en decimal y en hexadecimal.

decimal	binario	hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0101	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Lo habitual es que el programador realice sus cálculos en binario y que, después, tenga que traducirlos a hexadecimal. Para pasar un número de binario a hexadecimal, simplemente se dividirá en grupos de 4 bits empezando desde la izquierda y se sustituirá por su equivalente hexadecimal.

Por ejemplo,

```
Número: 1010101011110101010111110101010
A grupos de 4: 101 0101 0111 1010 1010 1111 1010 1010
Equivalente hexadecimal: 5 5 7 A A F A A
```

El número resultante es 557AAFAAh. Obsérvese como se ha puesto un "h." al final como notación habitual para hacer notar que el número es hexadecimal. No es lo mismo 1356 que 1356h.

Una vez obtenido el número hexadecimal, se introducirá en el programa utilizando la notación C 0xHH. .HHH, donde HH. . .HHH es el número hexadecimal calculado.

Por ejemplo,

```
int a;
a = 0x557AAFAA;
```

6.6.3 Operadores de bit

Para manipular los números enteros a nivel de bit, se recurre a los operadores C contenidos en la tabla 6.2. En la tabla 6.3 se tienen las tablas de verdad de las operaciones AND, OR y X-OR.

Tabla 6.2: Operadores C para la manipulación a nivel de bit.

$v \& w$	AND a nivel de bit entre v y w
$v w$	OR a nivel de bit entre v y w
$v \wedge w$	X-OR a nivel de bit entre v y w
$v \ll n$	desplazar n bits a la derecha a v
$v \gg n$	desplazar n bits a la izquierda v
$\sim v$	complementar los bits de v

Tabla 6.3: Tablas de verdad de las operaciones AND, OR y X-OR.

AND	OR	X-OR
$0 \& 0 = 0$	$0 0 = 0$	$0 \wedge 0 = 0$
$0 \& 1 = 0$	$0 1 = 1$	$0 \wedge 1 = 1$
$1 \& 0 = 0$	$1 0 = 1$	$1 \wedge 0 = 1$
$1 \& 1 = 1$	$1 1 = 1$	$1 \wedge 1 = 0$

6.6.4 Máscaras

Conociendo los operadores anteriores y aplicando simples reglas repetitivas es fácil manipular adecuadamente los números enteros a nivel de bit. Se dan a continuación las recetas para ello.

Extraer bits

Dada una palabra binaria, una operación muy habitual es extraer unos bits de interés y eliminar el resto (ponerlos a 0). Para lograrlo, se empleará el operador AND aprovechándose de que $b \text{ AND } 0=0$ y $b \text{ AND } 1=b$.

Ejemplo: Se desean extraer los 3 bits menos significativos de una palabra binaria. Para hacerlo, se empleará el operador AND con el valor binario $0\ldots 0111_2$.

	b_N	...	b_5	b_4	b_3	b_2	b_1	b_0
$\&$	0	...	0	0	0	1	1	1
	0	...	0	0	0	b_2	b_1	b_0

Expresado en C sería,

```
resultado = dato & 0x7;
```

Como comprobación, se podría mostrar por pantalla el contenido de la variable `resultado` de la siguiente manera.

```
printf("\nresultado\nde valor %d en decimal, %x en hexadecimal\n", resultado, resultado);
```

Suele ser interesante combinar las extracciones de bits con las operaciones de desplazamiento.

Ejemplo: En los bits 5 al 2 de un número entero está contenido un dato de 4 bits. Para extraerlo, se podrá aplicar la máscara binaria $0\dots0111100_2$ y, a continuación, hacer un desplazamiento de 2 posiciones.

$$\begin{array}{rcccccccc}
 & b_N & \dots & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\
 \& & 0 & \dots & 1 & 1 & 1 & 1 & 0 & 0 \\
 \hline
 & 0 & \dots & b_4 & b_4 & b_3 & b_2 & 0 & 0
 \end{array}$$

$$\begin{array}{rcccccccc}
 >> 2 & 0 & \dots & b_5 & b_4 & b_3 & b_2 & 0 & 0 \\
 \hline
 & 0 & \dots & 0 & 0 & b_5 & b_4 & b_3 & b_2
 \end{array}$$

Y, expresado en C sería,

```
resultado = (dato & 0x36) >> 2;
```

Si se tiene un poco de picardía, otra manera correcta de hacer la extracción sería,

```
resultado = (dato >> 2) & 0xF;
```

Comprobar estado de un bit

La comprobación del valor de un bit es un caso particular de la extracción de bits donde se elige un solo bit y se comprueba si el valor resultante es 0 o distinto de 0.

Ejemplo: Se desea comprobar el valor del bit 2 de una palabra binaria. Para hacerlo, se empleará el operador AND con el valor binario $0\dots0100_2$.

$$\begin{array}{rcccccccc}
 & b_N & \dots & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\
 \& & 0 & \dots & 0 & 0 & 0 & 1 & 1 & 1 \\
 \hline
 & 0 & \dots & 0 & 0 & 0 & b_2 & 0 & 0
 \end{array}$$

A continuación, se puede comprobar la igualdad o no con 0 del valor resultante. Expresado en C sería,

```
if ((dato & 0x4) == 0) {
    printf("Vale_0\n");
} else {
    printf("Vale_1\n");
}
```

O, también,

```
if ((dato & 0x4) != 0) {
    printf("Vale_1\n");
} else {
    printf("Vale_0\n");
}
```

Es habitual combinar esta operación con la de desplazamiento para facilitar la escritura de código. Por ejemplo,

```
if ((dato & (1 << 2)) == 0) {
    printf("Vale_0\n");
} else {
    printf("Vale_1\n");
}
```

Poner bits a 0

Otra necesidad típica es la de poner determinados bits a 0. Para lograrlo, se empleará el operador AND aprovechándose de que $b \text{ AND } 0=0$ y $b \text{ AND } 1=b$.

Ejemplo: Se desea poner a 0 los bits 4, 3 y 2 de una palabra binaria de 16 bits. Para hacerlo se empleará el operador AND con el valor binario $1\ldots,100011_2$.

	b_{15}	...	b_5	b_4	b_3	b_2	b_1	b_0
&	1	...	1	0	0	0	1	1
	b_{15}	...	b_5	0	0	0	b_1	b_0

Expresado en C sería,

```
resultado = dato & 0xFFE3;
```

Poner bits a 1

También suele ser necesario poner determinados bits a 1. Para lograrlo, se empleará el operador OR aprovechándose de que $b \text{ OR } 0=b$ y $b \text{ OR } 1=1$.

Ejemplo: Se desea poner a 1 los bits 5, 2 y 1 de una palabra binaria. Para hacerlo se empleará el operador OR y con el valor binario $0\ldots,0100110_2$.

	b_N	...	b_5	b_4	b_3	b_2	b_1	b_0
	0	...	1	0	0	1	1	0
	b_N	...	1	b_4	b_3	1	1	b_0

Expresado en C sería,

```
resultado = dato | 0x26;
```

Complementar bits

Otra operación de máscara muy habitual es la que permite complementar bits individuales. Para lograrlo, se emplea el operador X-OR aprovechándose de que $b \text{ X-OR } 0 = b$ y $b \text{ X-OR } 1 = \bar{b}$.

Ejemplo: Se desean complementar los bits 3, 1 y 0 de una palabra binaria. Para hacerlo, se empleará el operador X-OR con el valor binario $0\dots001011_2$.

$$\begin{array}{cccccccc} & b_N & \dots & b_5 & b_4 & b_3 & b_2 & b_1 & b_0 \\ \wedge & 0 & \dots & 0 & 0 & 1 & 0 & 1 & 1 \\ \hline & b_N & \dots & b_5 & b_4 & \bar{b}_3 & b_2 & \bar{b}_1 & \bar{b}_0 \end{array}$$

Expresado en C sería,

```
resultado = dato ^ 0xB;
```

Componer/descomponer palabras

6.7 Bibliotecas

En sentido amplio, las bibliotecas son código fuente o compilados de terceros que se añaden a las aplicaciones en la etapa de enlazado. Las bibliotecas proporcionan funcionalidades adicionales a las aplicaciones sin necesidad de desarrollarlas desde cero.

El principal interés de las bibliotecas es lograr productividad aprovechando el trabajo de otros en forma de paquetes comerciales o código libre/abierto/etc. La idea es no inventar la rueda si esta ya ha sido inventada por otro. Las bibliotecas también las podemos desarrollar nosotros y reutilizar en otros proyectos.

Algunas bibliotecas se las considera estándar y suelen venir ya incorporadas en las distribuciones de herramientas de desarrollo. Por ejemplo, en C, las funciones `printf()`, `sin()`, `rand()`, etc. suelen estar disponibles por defecto en una biblioteca preinstalada. Es un error habitual considerar a estas funciones de biblioteca estándar como parte del lenguaje C.

En el desarrollo de sistemas embebidos, una de las tareas tediosas es lograr que las bibliotecas externas funcionen adecuadamente con la aplicación a desarrollar. En

muchos casos, el esfuerzo será grande por depender del estilo del desarrollador, de la documentación disponible, de las dependencias, etc.

Al crecer la aplicación, también crecerá el número de bibliotecas externas de las que dependa. Para poder tener acotado este problema se debe proceder a organizarlas adecuadamente. Por ejemplo, es habitual que las bibliotecas se organicen en un subdirectorio específico en el que cada biblioteca se almacenará en un subdirectorio propio. El nombre de ese directorio suele ser *ThirdParty* o similar.

Siguiendo este principio, la figura 6.7 muestra la organización propuesta para trabajar los contenidos del libro. En la subcarpeta *ThirdParty* se colocarán las bibliotecas externas en subdirectorios y un archivo documentando como resolver las dependencias, es decir, donde localizar las bibliotecas externas. El documento *readme.txt* es fundamental para explicar como construir el proyecto (herramientas necesarias, que archivo abrir para configurar, etc.).

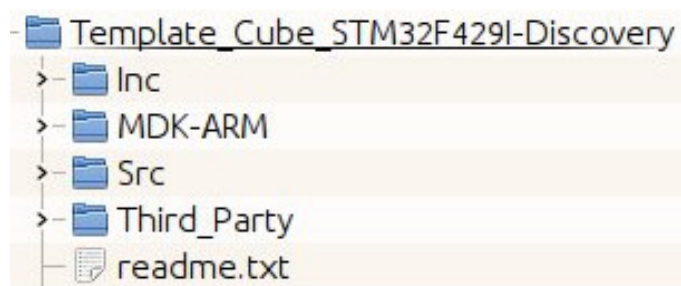


Figura 6.7: Propuesta de organización de carpetas.

En las rutas y nombres de archivos es recomendable evitar los espacios en blanco y símbolos extraños como “ñ”, acentos, “ç”, etc. Las herramientas de programación/-depuración han sido históricamente problemáticas en este sentido.

Para entender la filosofía de las bibliotecas, se propone crear una biblioteca mínima que consistirá en un simple módulo C y una cabecera asociada. Todo en código fuente.

El siguiente listado correspondería a la cabecera. Obsérvense los protectores de cabecera para evitar inclusiones múltiples y el prototipo de una función exportada.

```

/**
 * @file retardo.h
 * @brief Basic example for explaing libraries
 *
 * @author Angel Perles
 * @date 2016/02/11
 */

#ifndef RETARDO_H
#define RETARDO_H

#include <stdint.h>

void retardo(uint32_t cuenta);
  
```

```
#endif
```

A continuación se lista el módulo C que implementa la función. Destacar que, tanto la cabecera como el módulo, incluyen comentarios en formato *doxygen*, lo que simplifica la generación automática de documentación.

```
/**
 * @file retardo.c
 * @brief Basic example for explaining libraries
 *
 * @author Angel Perles
 * @date 2016/02/11
 */

#include "retardo.h"

/*****
 **
 * @brief Basic delay function
 * @param cuenta time to delay "a ojo"
 * @return none
 */
void retardo(uint32_t cuenta)
{
    volatile uint32_t cnt;

    cnt = cuenta;

    while(cnt > 0) {
        cnt--;
    }
}
/**** En of file *****/
```

Siguiendo con la idea de organización. Los archivos anteriores se deberían colocar en un subdirectorio dentro de **Third_Party**, por ejemplo, en el directorio **retardo**, tal y como se muestra en la figura 6.8.

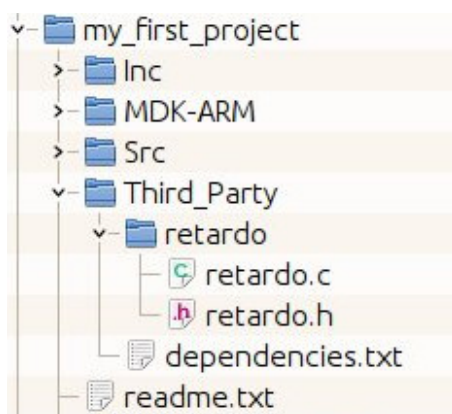


Figura 6.8: Propuesta de organización de carpetas.

Para que las aplicaciones desarrolladas tengan acceso a esta biblioteca básica, serán necesarias dos cosas:

- Establecer la ruta de búsqueda de cabeceras del entorno. La manera de hacerlo depende de la herramienta.
- Incorporar el módulo a nuestra aplicación. En este caso, al ser un módulo C, se incorporará directamente al proyecto.

Una vez realizados los pasos anteriores, se podrá hacer uso de la biblioteca. Para ello, bastará con incluir la cabecera en el módulo que haga uso de la función como muestra el siguiente ejemplo.

```
#include <stdio.h>
#include <retardo.h>

int main(void)
{
    while(1) {
        retardo(1000000);
        printf("Hola\n");
    }

    return 0;
}
```

Desarrollar un módulo con nombre `maxmin` que contenga dos funciones: una con nombre `max` que devuelva el máximo de dos parámetros de tipo `float`, y una con nombre `min` que devuelva el mínimo de dos parámetros `float`. Sigue estos pasos:

- Actividad:*
- Crea la cabecera apropiada.
 - Realiza la implementación en el módulo apropiado.
 - Desarrolla una aplicación que demuestre que funciona.
 - Comprueba el funcionamiento en tu entorno de programación favorito (Qt, BCB, DevC++, ...).

6.8 El Cortex Microcontroller Software Interface Standard (CMSIS)

Los microcontroladores ARM Cortex-M son muy complejos en comparación con los microcontroladores clásicos de 8 bits. Esto forzó a los fabricantes a proporcionar una serie de bibliotecas que abstrayesen distintos aspectos del microcontrolador con el fin de simplificar el desarrollo de aplicaciones y, por tanto, competir mejor en el mercado. Esta realidad llevó a una fragmentación del ecosistema ARM Cortex-M, pues las bibliotecas de un fabricante no eran compatibles con las de otro.

Esta fragmentación puede ser beneficiosa para algunos fabricantes predominantes, pero no es ventajosa ni para ARM ni para los desarrolladores de aplicaciones ni para fabricantes pequeños.

Para poder hacer frente a esta problemática, ARM, distintos fabricantes de micros (ST, Energy Micro, NXP, Toshiba, etc.), fabricantes de herramientas, etc. trabajaron en una propuesta común que permitiera la interoperabilidad de herramientas y facilitase la migración entre microcontroladores de distintos proveedores. Como resultado, se propone el *Cortex Microcontroller Software Interface Standard (CMSIS)* [CMSIS]. La figura 6.9 muestra el logo oficial.



Figura 6.9: Logo de CMSIS

Entre otras cosas, CMSIS propone una arquitectura a distintos niveles para simplificar la manera de resolver distintos problemas. En la figura 6.10 se representa esta arquitectura.

El bloque *CMSIS-core* provee las funcionalidades para arranque del sistema (reloj, ...) hasta llegar al `main()`, acceso características específicas del núcleo y periféricos básicos, una visión consistente de los registros de periféricos y servicios de interrupción, etc.

El bloque *CMSIS-RTOS* provee una abstracción de las primitivas de los *Real-time operating systems (RTOS)* para microcontroladores. Intenta facilitar la migración entre distintos proveedores de microkernels.

El bloque *CMSIS-DSP* pretende proveer de funciones típicas de DSP de manera sencilla para que sea fácil explotar este potencial sin necesidad de ser un experto.

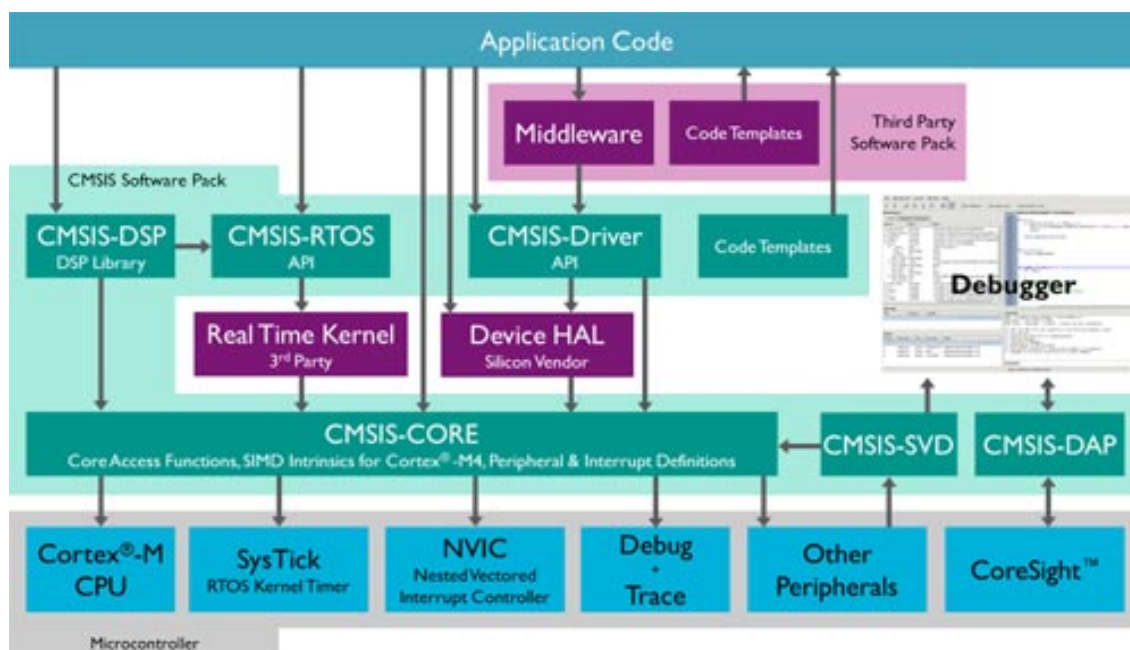


Figura 6.10: Diagrama de la arquitectura CMSIS (v4)

A este nivel se proporcionan primitivas para filtros digitales, PIDs, transformada de Fourier, operaciones con matrices, etc.

El bloque *CMSIS-SVD* pretende proveer una manera de especificar la descripción del sistema muy portable para permitir la rápida adaptación de las herramientas de depuración a nuevos miembros que aparezcan en el mercado.

CMSIS intenta ser simple, y propone (y no obliga) una manera de desarrollar el código siguiendo una reglas sencillas que se basan en la experiencia de los ingenieros y en buenas prácticas de desarrollo de código como por ejemplo cumplir MISRA 2004 [MISRA] de la *Motor Industry Research Academy* como base para aplicaciones en sistema críticos, documentar el código fuente empleando doxygen [doxygen] para que se automatice la generación de manuales, el empleo de la estandarización de datos enteros (stdint), el uso de camelCase (separación por mayúsculas) para nombres de variables, etc.

6.9 El firmware STM32Cube HAL y LL de St

AFEGIR ACÍ LA FERRAMENTA STM32CubeMX

La empresa St es una de las que se ha adherido a la iniciativa CMSIS, así que las bibliotecas y ejemplos proporcionados por esta empresa siguen los principios CMSIS. Al analizar un paquete de firmware de St, se podrá comprobar el conjunto de bibliotecas proporcionadas por ARM para el *core*, y las de periféricos y demás servicios que proporciona St y siguen las reglas CMSIS. Como ejemplo, la figura 6.11 muestra la organización en directorios de dicho firmware. Obsérvese el subdirectorio CMSIS y el subdirectorio para el llamado HAL.

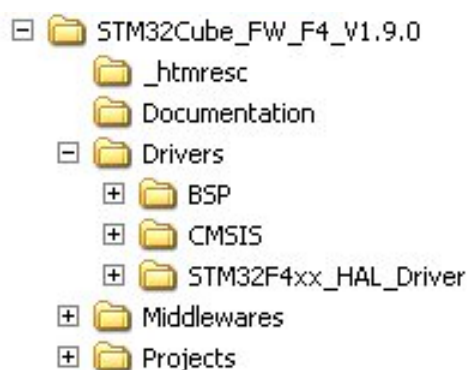


Figura 6.11: Estructura del paquete de firmware para los microcontroladores de la serie STM32F4

Un proyecto completo en C para un microcontrolador de la gama STM32 debería seguir CMSIS y, por tanto, estará formado por el conjunto mínimo fijo de archivos indicados en la tabla 6.4. Esta estructura será la misma en microcontroladores ARM Cortex-M de otros fabricantes que sigan este estándar, facilitando la migración.

Además de estos archivos, es muy probable que el propio fabricante provea de algún tipo de abstracción de mayor nivel. St sigue esa idea mediante dos conjuntos de bibliotecas: las *STM32Cube HAL (Hardware Abstraction Layer)* y las *LL Low-level*. Las HAL son bibliotecas de alto nivel orientadas a la aplicación que pretende maximizar la portabilidad a través del portafolio STM32 y proporcionar un conjunto coherente de componentes de middleware (RTOS, USB, TCP/IP, gráficos, etc.). Por su parte, las bibliotecas LL proveen funcionalidad para acceder a los periféricos a nivel de registro, permitiendo aplicaciones más ligeras pero menos portables. Ambas bibliotecas son complementarias y pueden combinarse.

Para hacerse una idea del uso del HAL, el siguiente fragmento de código hace uso de él.

```
#include "stm32f4xx_hal.h" // cabeceras proporcionadas por St para simplificar el uso de los peri
#include "led.h"

/*****
**
```

Tabla 6.4: Archivos mínimos de un proyecto completo para STM32 que siga CMSIS.

Archivo	Descripción
startup_stm32f4xx.s	STM32F4xx devices startup file. Es un archivo en ensamblador y punto de arranque del microcontrolador. En C, antes de llegar al <code>main()</code> , se pasa por aquí.
system_stm32f4xx.h/.c	CMSIS Cortex-M4F STM32F4 devices peripheral access layer system. Son las funciones del core de ARM común a todos los fabricantes adheridos.
stm32f4x_???h /.c	Cabecera y código del driver de un dispositivo ???. Por ejemplo, stm32f4x_spi.h es la cabecera para el driver del SPI.
stm32f4xx_conf.h	Peripheral's drivers configuration file. Configura que cabeceras de periféricos se añaden. No lo toques al principio.
stm32f4xx.h	CMSIS Cortex-Mx STMFx device peripheral access. Incluyendo esta cabecera en nuestros módulos, debería ser suficiente para acceder a los periféricos.
stm32f4xx_it.h/.c	Cabecera y plantillas para todos los servicios de interrupción. Basta añadir nuestro código de servicio dentro.

```

* @brief Preparing pin corresponding to LED green (PG13)
* @return none
*/
void LED_Init(void) {

    GPIO_InitTypeDef  GPIO_InitStructure;           // estructura donde se pone la configuracion des

    __HAL_RCC_GPIOG_CLK_ENABLE();                   // darle reloj al periferico, AHORA VIVE!

    /* Configure the GPIO_LED pin */
    GPIO_InitStructure.Pin      = GPIO_PIN_13;       // pin que desamos configurar
    GPIO_InitStructure.Mode     = GPIO_MODE_OUTPUT_PP; // lo vamos a usar como salida en push-pull
    GPIO_InitStructure.Pull     = GPIO_PULLUP;       // activar el pullup
    GPIO_InitStructure.Speed    = GPIO_SPEED_FAST;   // actualizacion rapida

    HAL_GPIO_Init(GPIOG, &GPIO_InitStructure);      // hacer efectiva configuracion puerto
}

```

Obsérvese en el listado la inclusión del archivo de cabecera `stm32f4xx_hal.h`. En el código se hace uso de características CMSIS; la que empieza por `__HAL_RCC_*` son las de core de ARM y las que empiezan por `GPIO_*` son las proporcionadas por St para sus periféricos. Si se emplease un ARM Cortex-M de otra empresa, por ejemplo, un LPC1768 de NXP, la función `RCC_*` sería la misma, pero la función `GPIO_*` sería distinta o no existiría.

Como ayuda extra, hay una herramienta gráfica denominada *STM32CubeMX* [STM32CubeMX] que simplifica la configuración de los microcontroladores STM32 y genera el código de inicialización C correspondiente a través de un proceso paso a paso. La figura 6.12 muestra la imagen oficial de esta STM32Cube.

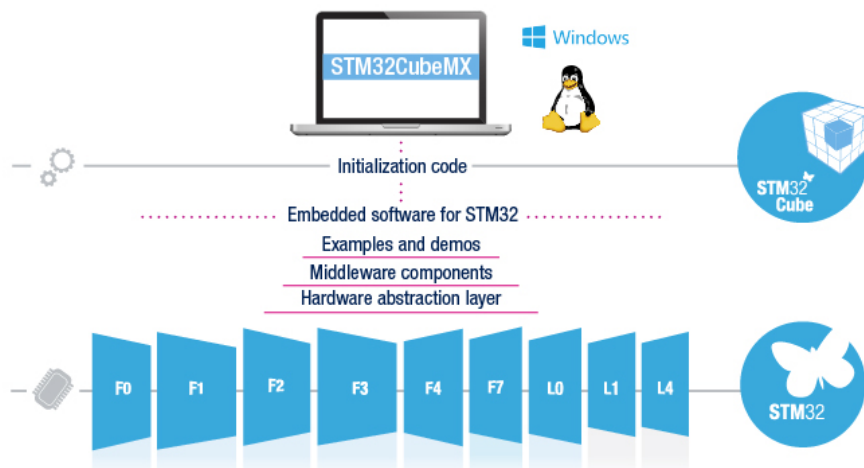


Figura 6.12: Imagen oficial del STM32Cube

Capítulo 7

Entorno de trabajo

7.1 Introducción

Este capítulo contiene las instrucciones básicas para poner a punto el puesto de trabajo con el que desarrollar las actividades del libro.

El enfoque es emplear software gratuito y versiones limitadas de paquetes comerciales que permitan a cualquiera practicar con los contenidos del libro. La única limitación será invertir unos pocos euros en las placas de desarrollo propuestas.

Como resumen, la siguiente lista contiene el conjunto de herramientas hardware y necesarias y la versión empleada.

Hardware:

- Ordenador PC con Microsoft Windows 7 o superior.
- Cable USB de tipo A a mini-B.
- Placa Discovery STM32F429I-DISC1 y Nucleo L476RG

Software:

- St-Link utility (STSW-LINK004) ver. 4.2.0
- ARM Keil MDK-Arm ver. 5.25
- STM32Cube initialization code generator ver. 4.26.1
- STM32Cube MCU Package for STM32F4 series ver. 1.19.0
- STM32Cube MCU Package for STM32L4 series ver. 1.12.0

7.2 Ordenador personal

El primer requisito es disponer de un ordenador personal con las siguientes características:

- Sistema operativo Microsoft Windows 7 o superior (probado hasta Windows 10). Linux es factible.
- RAM mínima de 4 GB.
- Espacio en disco duro libre de 10 GB.
- Permisos de administración para instalación de drivers o USBs.
- Acceso a Internet (mejor cable que WiFi).

NOTA: Se recomienda desactivar el antivirus para instalar el software.

7.3 Placas de evaluación

En el libro se emplean dos placas. Una de la serie Discovery y otra de la serie Nucleo.

7.3.1 STM32F429ZI Discovery

Para las primeras prácticas se emplea la placa de evaluación Discovery STM32F429I-DISC1 (ref. 32F429IDISCOVERY) [32F429IDISCOVERY]. En la figura 7.1 se puede ver el aspecto de la placa.

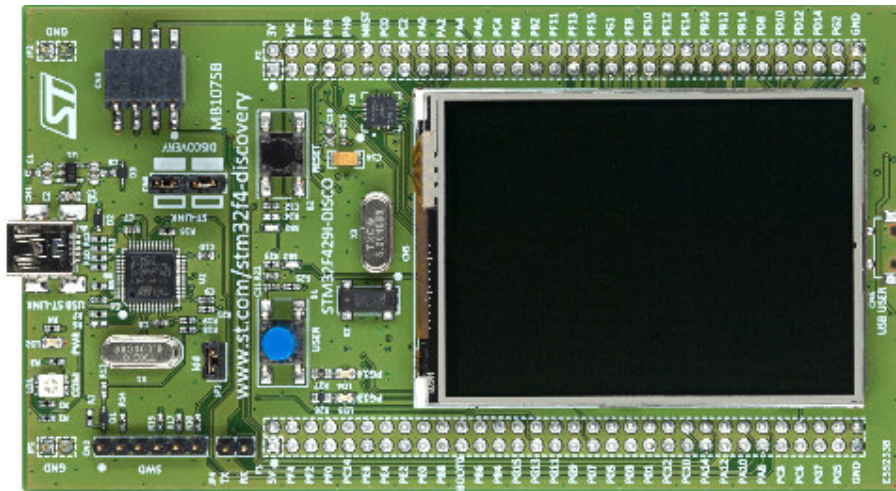


Figura 7.1: Placa STM32F429 Discovery kit

La placa se puede adquirir por unos 30 Eur. en distribuidores de componentes electrónicos típicos (Mouser, Digikey, Farnell, Arrow, Venco, ...).

También será necesario un cable USB de tipo A a mini-B. Evitar cables malos (por ejemplo, los enrollables del Carrefour).

Para entender y usar la placa se recomienda trabajar las guías [STM32F4Discovery-user-guide].

7.3.2 STM32L476 Nucleo-64

Los aspectos más avanzados se tratarán con la placa de evaluación NUCLEO-L476RG [NUCLEO-L476RG] que incorpora un microcontrolador de bajo consumo STM32L476. Con respecto a la Discovery, esta placa no incluye electrónica extra de tipo sensores, pantallas, etc., dando más libertad para la conexión externa de otros elementos. En la figura 7.2 se puede ver el aspecto de la placa.

Como en el caso anterior, será necesario un cable USB de tipo A a mini-B. La guía [STM32L4Nucleo-user-guide] es necesaria para conocer y empezar a trabajar con esta placa.

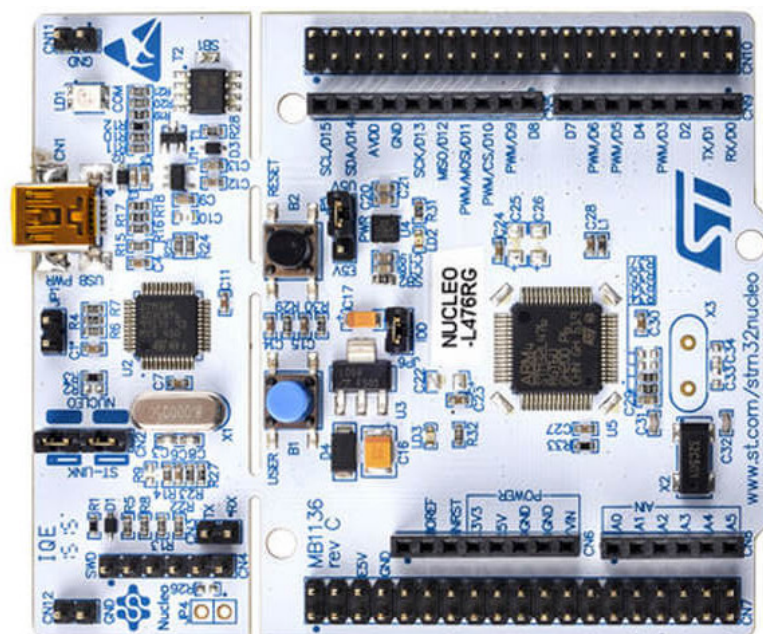


Figura 7.2: Placa Nucleo L476RG

7.4 Sistema de depuración St-Link

Todo equipo profesional que se precie dispone de sondas de depuración que permiten volcar los proyectos en el microcontrolador y monitorizar la ejecución del código. Para realizar esta tarea se puede acudir a productos específicos como las excelentes sondas de Segger o recurrir a las soluciones del propio fabricante.

St provee un sistema de depuración denominado *St-Link* que permite actuar sobre el microcontrolador objetivo en muchos aspectos como la manipulación de posiciones de memoria, borrado del dispositivo, programación de bits de seguridad, monitor de la salida del bus ITM, serializar piezas, etc.

Para hacer uso de él, es necesario disponer del hardware apropiado y de controladores software específicos. Las placas de evaluación Discovery incorporan esta sonda, así que es muy fácil y económico aprovecharse de estas características profesionales. Más aún, se puede cablear la discovery para que el St-Link embebido se pueda conectar a nuestros propios diseños y realizar la depuración. En la figura 7.3 se puede ver un St-Link original y la parte de la Discovery que se corresponde a un St-Link embebido.

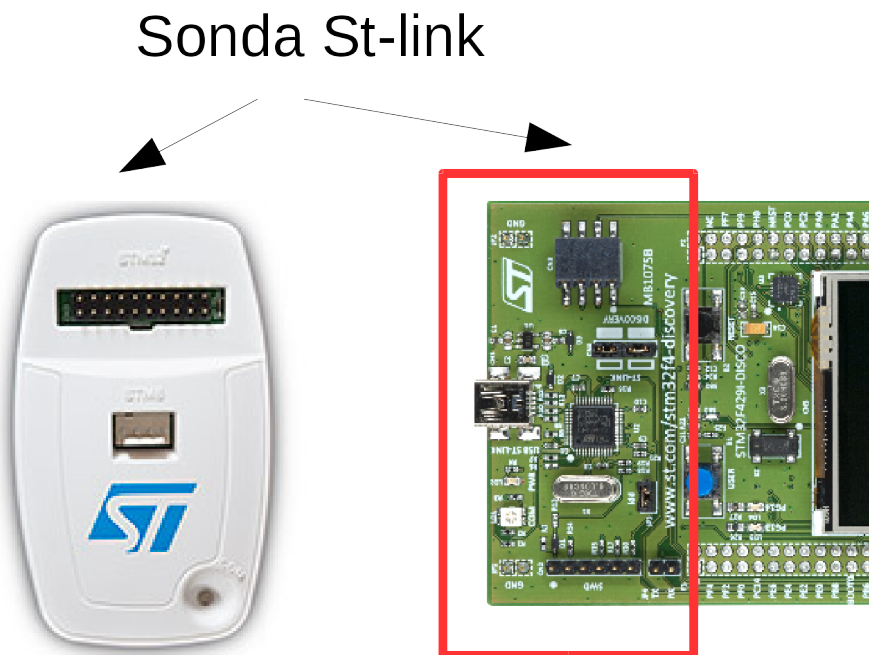


Figura 7.3: Izquierda: sonda St link. Derecha: sonda st-link embebida en la Discovery.

7.4.1 Instalación y configuración

Antes de conectar cualquier sonda compatible St-Link es necesario instalar los controladores.

Para aprovechar características extra, se propone instalar el conjunto de utilidades *St-Link utility* [**stlinkutility**] que, además de los controladores, incorpora utilidades que permiten actualizar el firmware de la sonda.

Una vez instalado el software, se deberá conectar la sonda (o la discovery) al computador y esperar a que Windows la detecte y proceda a la instalación de los controladores. En el caso de la placa Discovery, será necesario un cable USB tipo A a mini-B que conecte la placa al ordenador personal.

La figura 7.4 muestra la lista de controladores que se configuran al conectar la Discovery por primera vez al computador. A partir de ese momento, se puede comprobar si el sistema reconoce la Discovery mediante el icono de expulsión de dispositivos USB de Windows según se muestra en la figura 7.5.

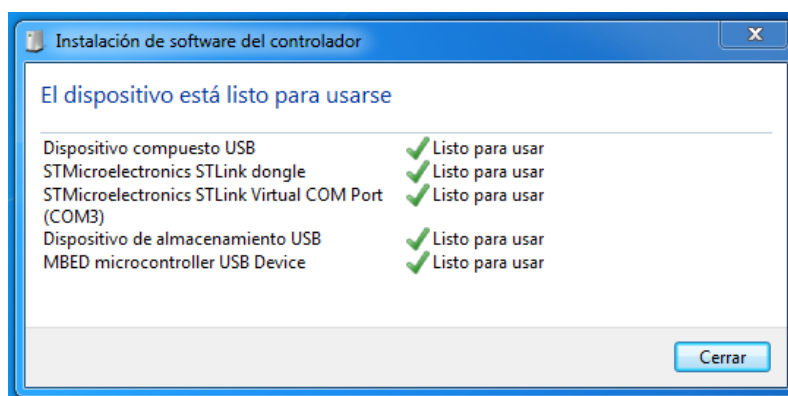


Figura 7.4: Controladores instalados por St-Link al conectar la Discovery

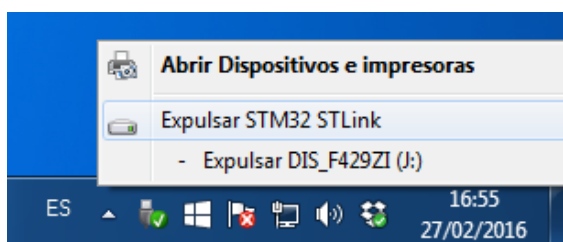


Figura 7.5: Icono de expulsión de dispositivos USB de Windows

7.4.2 Comprobación de la placa

Con la utilidad *St-Link utility*, se puede comprobar si el dispositivo objetivo está accesible y operativo. Para ello insertar la Discovery (o cualquier conjunto sonda + diseño con micro), y abrir la aplicación *St-Link utility* mostrada en la figura para conectar con el dispositivo objetivo y manipularlo.

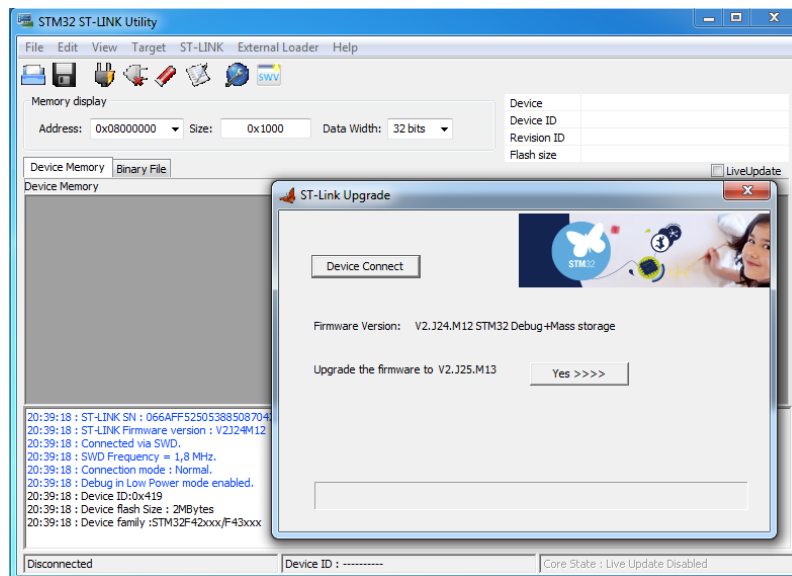


Figura 7.6: St-link utility.

7.4.3 Actualización de la sonda

Antes de meterse en harina, es importante tener el firmware de la sonda actualizado a la última versión. Para ello se conectará la sonda (o la Discovery), se abrirá la aplicación *St-Link utility* y seleccionará el menú ST-Link->Firmware upgrade. Debería aparecer un menú como el mostrado en la figura 7.6 que permite actualizar el firmware. Una vez actualizada, deberá desconectarse y volverse a conectar.

7.4.4 Volcado de ejecutables

Las aplicaciones que se desarrollen para el micro se pueden distribuir en distintos formatos que esta aplicación puede grabar en el microcontrolador. Un formato bastante universal para esta tarea es el llamado *Intel-HEX*.

Para volcar este tipo de archivos en la memoria de microcontrolador se hará: File->Open file y se selecciona el archivo. A continuación se hará Target->Program-and-verify y se procederá según instrucciones. La figura 7.7 muestra el aspecto de la ventana de grabación.

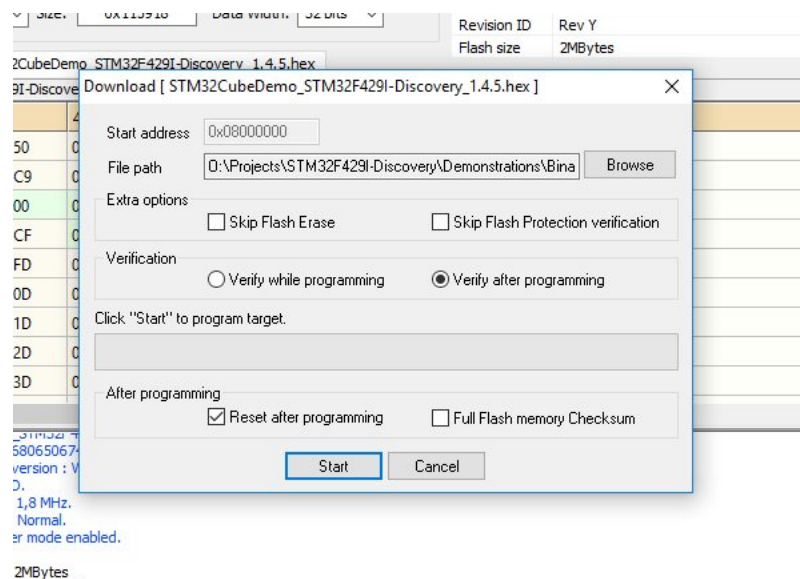


Figura 7.7: Ventana de programación de St-link utility.

7.5 Keil MDK-ARM 5

Keil MDK-ARM versión 5 [**keil-mdk-arm**] es un entorno de desarrollo comercial para microcontroladores. El conjunto de herramientas se compone del propio MDK-ARM, que incluye un entorno integrado de desarrollo, el compilador, enlazador, depurador, paquetes básicos, etc. y un gestor de paquetes que permite instalar paquetes para distintas familias de dispositivos junto con ejemplos de proyectos.

Se ha seleccionado este entorno por su simplicidad y por ser la herramienta oficial de ARM, que compró esta empresa hace unos años. Además de ARM Cortex-M, soporta otras arquitecturas.

7.5.1 Obtener e instalar Keil

Para probar Keil MDK, la opción más sencilla es obtener una versión de evaluación (MDK-Lite) del sitio de la empresa (ver figura 7.9) cuya principal restricción es un límite de 32 KBytes para el objeto de salida (el programa). La descarga requiere que se proporcione información de registro que no compromete a nada, así que se puede proceder sin problema.

Para emplear el entorno, será necesario un computador tipo PC con sistema operativo Microsoft Windows 7 o superior y suficiente espacio en disco duro.

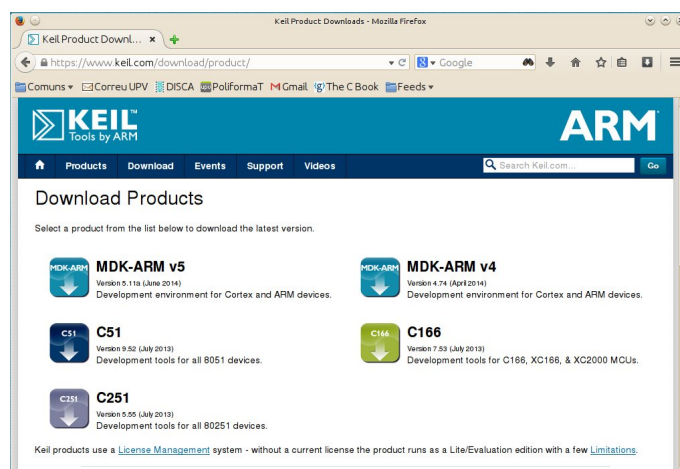


Figura 7.8: Página web de descarga del entorno Keil MDK-ARM

Al hacer la instalación es recomendable continuar con conectividad a Internet, pues se solicitará la información de registro (se pueden dejar los campos de registro en blanco o con valores no sensibles). Tras la instalación, se ejecutará automáticamente el denominado *Package installer* mostrado en la figura 7.9 que permite añadir bibliotecas, controladores, ejemplos, etc. de distintos fabricantes. En esta primera fase se puede omitir este paso, pero este gestor de paquetes se abrirá la primera vez

que se construya un proyecto para microcontrolador con la intención de descargar complementos necesarios.

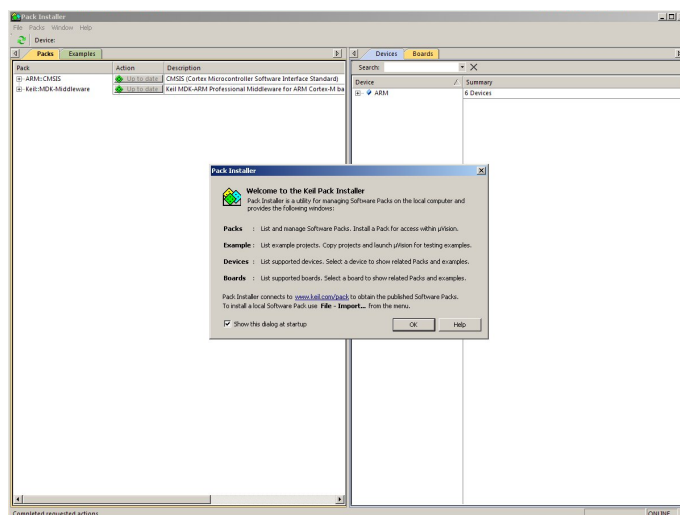


Figura 7.9: Package manager de Keil

7.5.2 Construir un proyecto

Un proyecto es el conjunto de archivos de código y configuraciones que permiten obtener un ejecutable. Los proyectos son una característica común a todos los entornos de desarrollo para C.

En el caso de Keil, se debe abrir un archivo con extensión `uvprojx` que contiene la configuración del proyecto. La localización de este archivo puede ser tediosa en Windows debido a su manía por ocultar las extensiones de archivos. Se recomienda configurar Windows para que no oculte estas extensiones.

La figura 7.10 muestra el aspecto de Keil cuando se abre un proyecto. Para abrir un proyecto, seleccionar la opción *File->Project->Open project ...* de Keil y localizar el archivo con extensión `uvprojx` que contiene el proyecto.

En la ventana *Project* se nos mostrarán los archivos que forman el proyecto (`.c`, `.h`, `.lib`, `.txt`, etc) organizados en una estructura que asemeja carpetas, pero que no se corresponde a la estructura real en disco. Si se selecciona cualquier archivo, este se abrirá en la ventana de edición y se podrá modificar.

Para obtener el ejecutable a partir de un proyecto es necesario *construir el proyecto*, es decir, preprocesar, compilar y enlazar todas las piezas. La manera más sencilla de hacerlo en Keil es usar los iconos correspondientes mostrados en la figura 7.11. La primera vez que se construya un proyecto, o cuando la construcción es errática, se recomienda emplear la opción *reconstruir proyecto* que forzará la realización de todos los pasos de construcción. En etapas subsiguientes se recomienda la opción

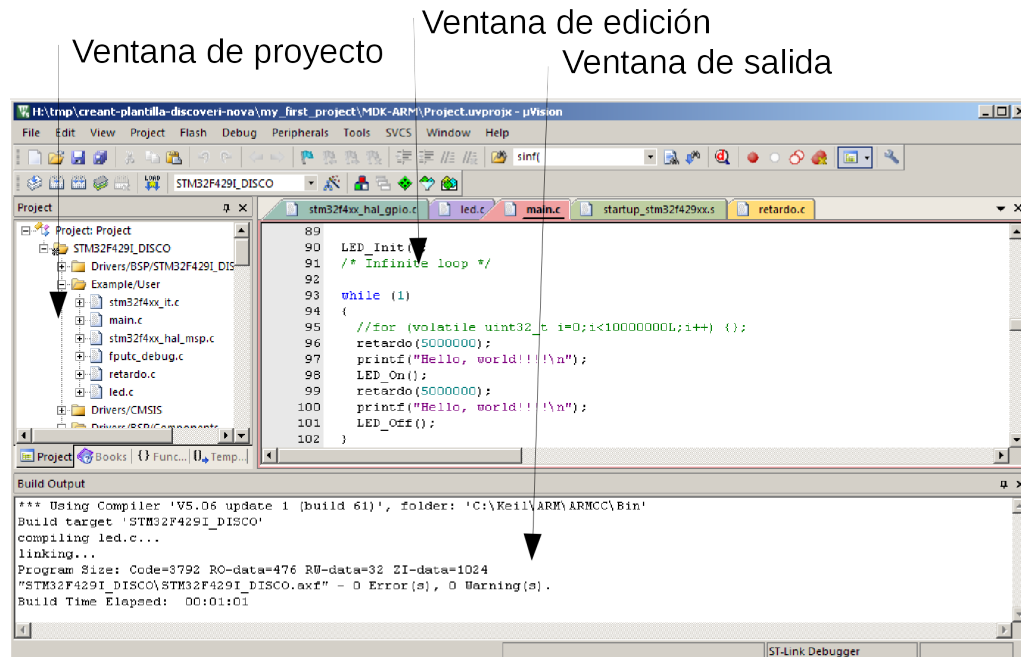


Figura 7.10: Keil con un proyecto abierto

construir, pues solo trata los archivos que han sido modificados o cuyas dependencias han variado, haciendo más rápida la construcción. Finalmente, la opción *compilar* realiza la compilación del módulo que se esté editando en ese momento y, por tanto, es útil para comprobar que no contiene errores de compilación.

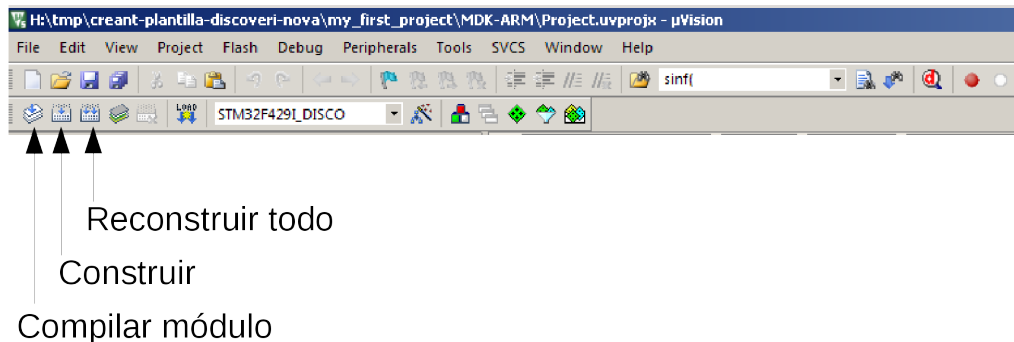


Figura 7.11: Iconos para construir el proyecto

En la ventana de salida se nos informará de si la construcción del proyecto ha sido correcta. La figura 7.12 muestra un ejemplo de construcción correcta donde se informa de los requisitos de memoria de la aplicación construida (explicar altre dia o passar????).

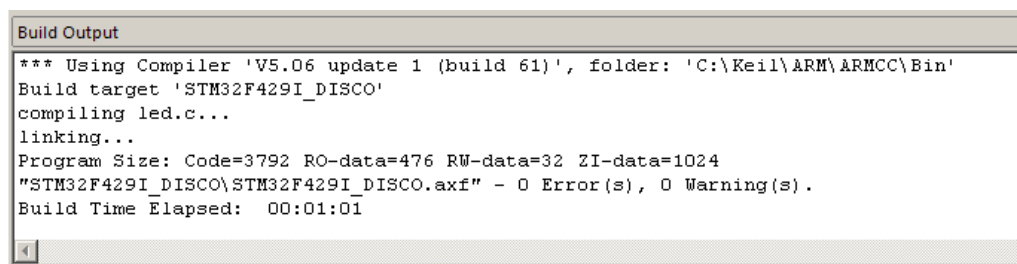


Figura 7.12: Iconos para construir el proyecto

7.5.3 Configurar Keil para usar St-Link

Para el resultado de un proyecto construido pueda volcarse en el microcontrolador es necesario configurar el sistema de depuración. Asumiendo que se va a utilizar St-Link, los pasos a seguir ... ya lo escribiré cuando haga falta. Si llega el momento. Por si alguien lo necesita, aquí dejo una explicación antigua

<http://armcortexm.blogs.upv.es/stm32f4-discovery-and-printf-redirection-to-debug->

Los proyectos de demostración que proporciona St ya suelen venir preconfigurados para la sonda St-link.

7.5.4 Volcado de un proyecto en un microcontrolador

Suponiendo configurado Keil, una vez construido correctamente un proyecto Keil, se pueden emplear los iconos mostrados en la figura 7.13 para hacer la grabación en la memoria flash del microcontrolador objetivo. El microcontrolador ejecutará la aplicación volcada cuando se lo alimente o se pulse RESET.

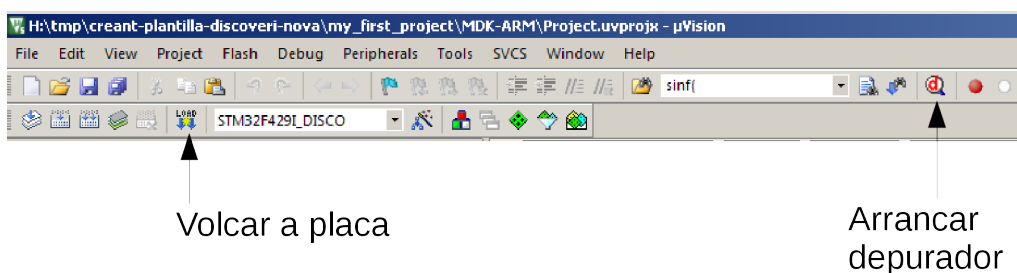


Figura 7.13: Iconos de volcado y arranque de la depuración

Gracias al depurador St-link, es posible hacer cosas más interesantes como controlar la ejecución, establecer puntos de ruptura, hacer ejecución paso a paso, monitorizar el valor de variables, etc. Para iniciar el depurador se puede emplear el icono correspondiente de la figura 7.13.

En la figura 7.14 se puede ver el aspecto por defecto de Keil en modo depuración.

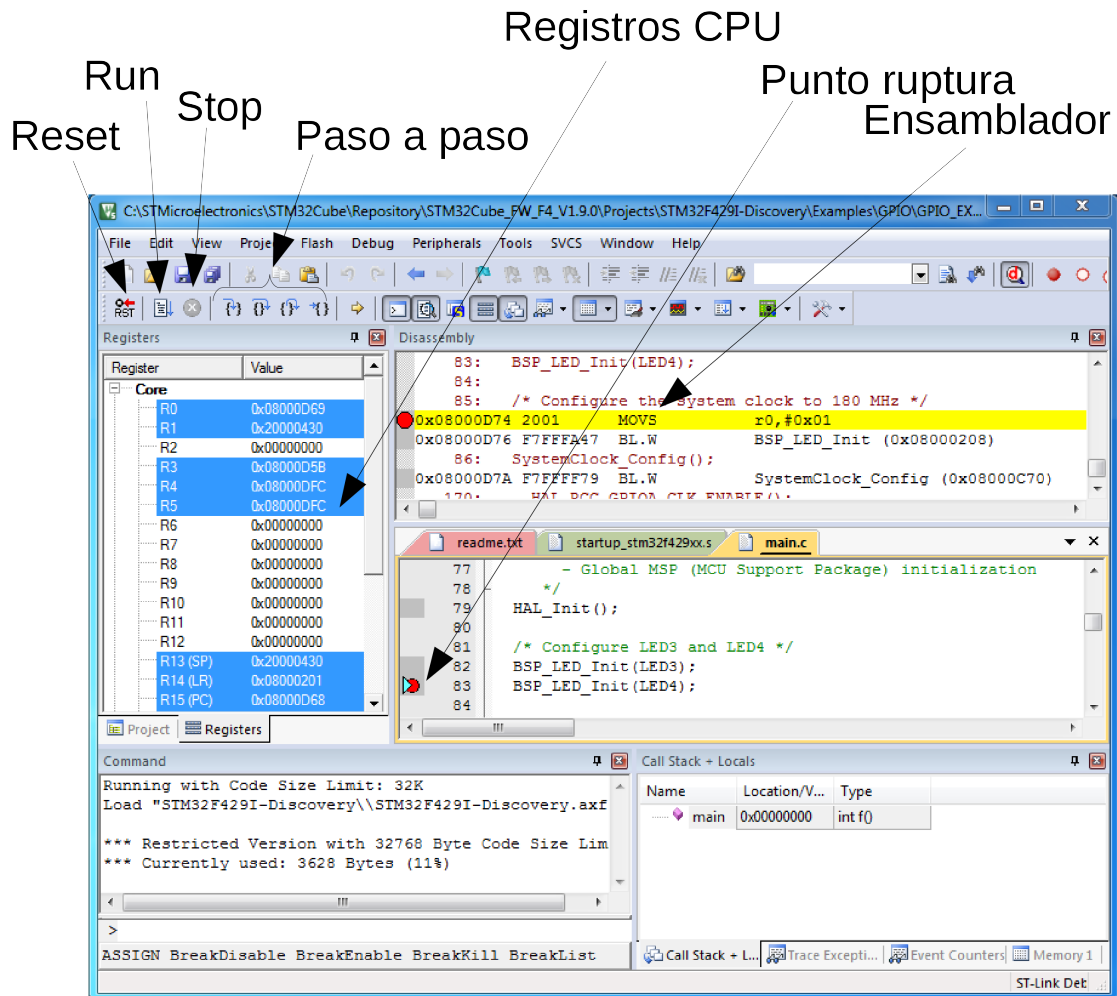


Figura 7.14: Keil en modo depuración

7.5.5 Añadir archivos a un proyecto existente

Una de las tareas más habituales a la hora de crear un proyecto es incorporar módulos creados por terceras partes. En general, estos archivos tendrán la extensión `.c`, pero pueden ser de otros tipos.

La manera más sencilla de añadir archivos en un proyecto Keil abierto consiste en usar el menú contextual de ratón sobre el grupo de archivos en el que se desea añadir el nuevo archivo según se muestra en la figura 7.15. Los “grupos” son la manera en la que Keil permite organizar los archivos de proyecto para que sean más fáciles de localizar, y los muestra engañosamente como carpetas que no existen en la realidad.

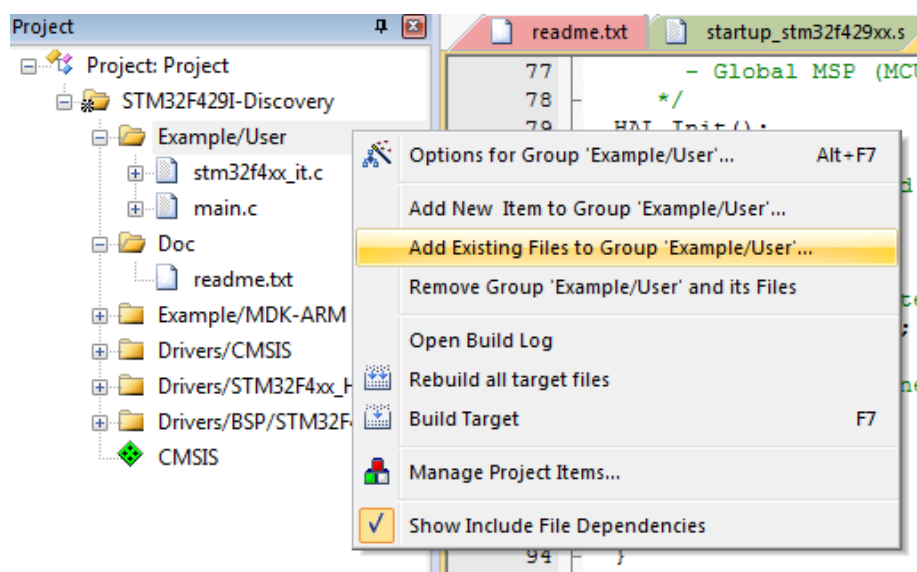


Figura 7.15: Añadir un archivo a un proyecto Keil

7.5.6 Crear archivos nuevos

Empleando el menú de *Keil File* -> *New* se crean archivos de texto que no tienen entidad en sí misma ni forman parte del proyecto.

En dichos archivos podemos colocar lenguaje C, ensamblador, texto explicativo, etc. En el momento de grabarlo será cuando se le proporcione una extensión para indicar de qué tipo es (e.g. `.c` o `.h` para indicar que es código C).

Un vez guardado, se deberá añadir al proyecto a mano tal como se ha indicado en el apartado anterior.

7.5.7 Añadir rutas de búsqueda de archivos de cabecera .h

Configurar la *ruta de búsqueda de cabeceras* en Keil accediendo al menú principal: Project ->Options for target ->C/C++ ->Include path y añadiendo la ruta de búsqueda según la figura 7.16.

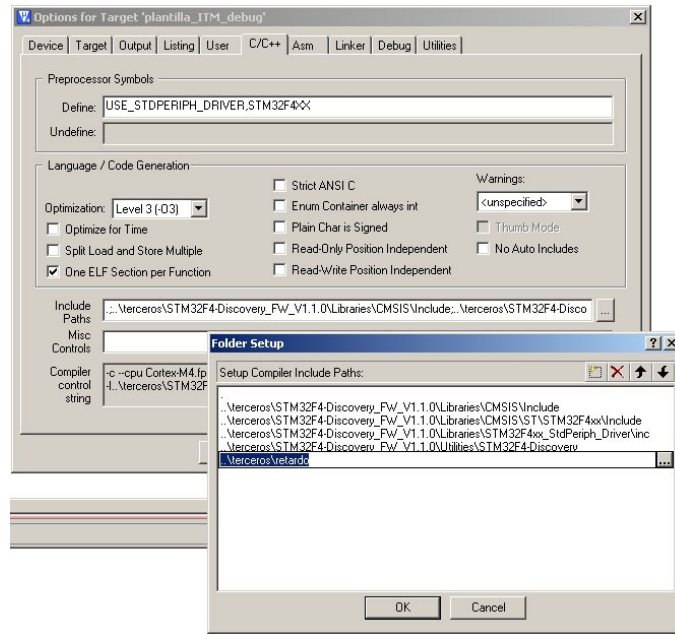


Figura 7.16: Ajuste de la ruta de búsqueda de cabeceras.

7.6 STM32CubeMX: STM32Cube initialization code generator

STM32CubeMX [**stm32cubemx**] es una herramienta software muy útil para simplificar el desarrollo de aplicaciones para microcontroladores STM32.

Básicamente, pretende generar automáticamente el código y empaquetar las bibliotecas necesarias a partir de una elección de microcontrolador y la configuración de periféricos y servicios adicionales.

Puede ser de gran ayuda a la hora de configurar aspectos farragoso como los relojes o estimar el consumo del dispositivo. Además, incluye bibliotecas preconfiguradas muy interesantes como distintos perfiles USB, FreeRTOS, gráficos y pilas TCP/IP.

De momento solo lo usaremos para facilitar la descarga, instalación y actualización de las bibliotecas para el dispositivo que elijamos.

Seguir los siguientes pasos para la instalación y configuración inicial.

- Descargar la aplicación siguiendo [**stm32cubemx**].
- Ejecutar el instalador teniendo en cuenta que puede solicitar la instalación adicional de Java. Aceptarla.
- Instalar en una carpeta raíz localizable, por ejemplo C:\STMicroelectronics\
- Configurar el lugar donde se colocan los repositorios. Menu Help->Updater settings y poner algo como “C:\STMicroelectronics \STM32Cube-Repository”

La aplicación debería tener el aspecto de la figura 7.17.

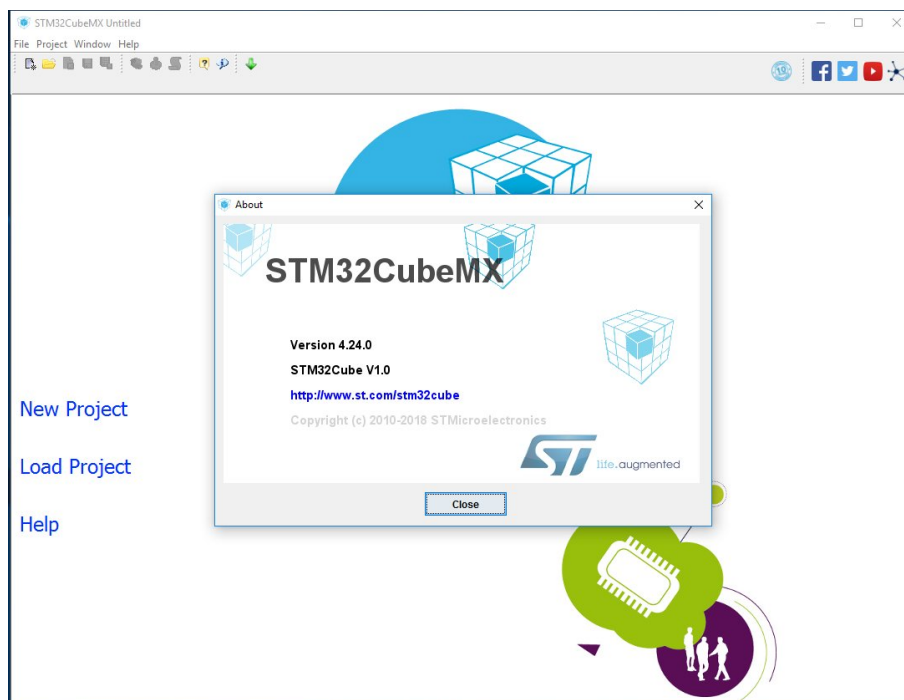


Figura 7.17: Aspecto de STM32CubeMX

7.7 Paquete de bibliotecas STM32Cube

St proporciona bibliotecas preconfiguradas para cada serie de microcontroladores. Las bibliotecas oficiales se llaman HAL (*Hardware Abstraction Layer*) y LL (*Low-level*), y son un paquete enorme que incluye las propias bibliotecas para C/C++, las bibliotecas CMSIS de ARM, bibliotecas de terceros, manuales y muchos ejemplos para las placas de desarrollo y de evaluación.

Cada serie de microcontroladores (F1, F2, ... L0, L1, ...) dispone de su propio paquete de bibliotecas que se puede descargar de la web de St. Por ejemplo, para la serie STM32F4 se llama STM32CubeF4 [**stm32cubef4**].

Otra opción es utilizar la aplicación STM32CubeMX para que gestione la carga e instalación de cada paquete. Además se tendrá la ventaja de que la propia aplicación puede comprobar si hay actualizaciones.

Para hacer la instalación de la biblioteca, se irá a "*Help -> Manage embedded software packages*" se marcará el paquete STM32CubeF4 (ver figura 7.18). El archivo que se descarga y se descomprime es enorme, así que hay que tener en cuenta si se dispone de suficiente espacio.

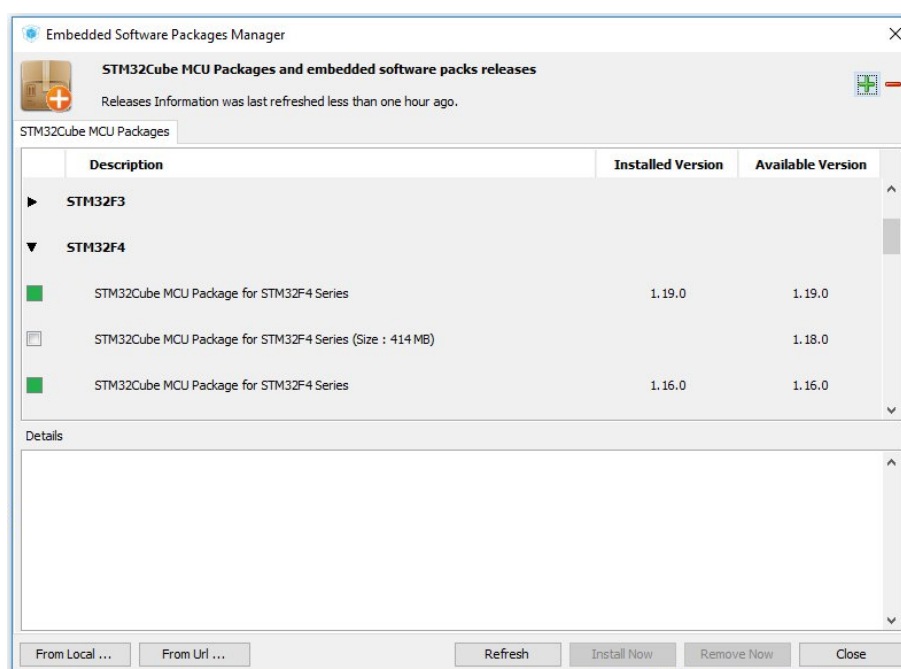


Figura 7.18: Instalar el firmware Cube F4 desde Cube Mx

De la misma manera se procederá para el paquete STM32CubeL4.

7.8 Plantilla para la placa St Discovery 429i-Disc1

Para facilitar el desarrollo de las actividades se proporciona una plantilla basada en la plantilla de St incorporada en el HAL. Las principales diferencias con dicha plantilla son:

- Requires Keil MDK-ARM 5.x
- Optimization level 0 to facilitate debugging
- Added redirection of stdout (i.e. printf) to the DEBUG SWO service.
- Added redirection of stdout (i.e. printf) to the LCD.
- STM32CubeF4 Fw not required in a relative path, but ... see file Third_Party/dependencies.txt in order to fulfill dependencies before trying to build this template

La plantilla se proporciona en un zip y, para usarla, se deben seguir antes los siguientes apartados.

7.8.1 Unidad de disco virtual O:

Lo razonable en un proyecto es que las bibliotecas externas estén en un subdirectorío del proyecto, sin embargo, el firmware STM32Cube requiere casi 1 GB de espacio, con lo que no es razonable insertarlo en cada nuevo proyecto.

Una solución es mantenerlo separado y configurar el entorno de desarrollo empleado para localizarlo. Por desgracia, Keil es uno de los entornos menos amigables en ese sentido, así que es necesario hacer una chapuza para resolverlo.

Para facilitar el uso de la plantilla desde el entorno Keil se propone un “apaño” consistente en la creación de una unidad de disco virtual que emule como contenido la raíz del firmware de St. Para lograrlo, seguir los siguientes pasos:

- En inicio->ejecutar de Windows o buscar o pulsar tecla Windows + R, escribir “shell:startup”. Se debería abrir una carpeta similar a la mostrada en la figura 7.19. Esta carpeta se usa para contener programas que se ejecutan al arrancar el SO.
- Crear un archivo por lotes con nombre **keil-hal-unit-o.bat** con Keil. Para ello, abrir Keil, crear un archivo en blanco, copiar la siguiente línea dentro reemplazando C:\... por la ruta donde esté el firmware.

```
subst 0: /D
subst 0: C:\STMicroelectronics\STM32Cube-Repository\STM32Cube_FW_F4_V1.14.0
```

- Grabar el archivo con el nombre **punyeta-keil.bat** en la carpeta de inicio según se muestra en la figura 7.20.

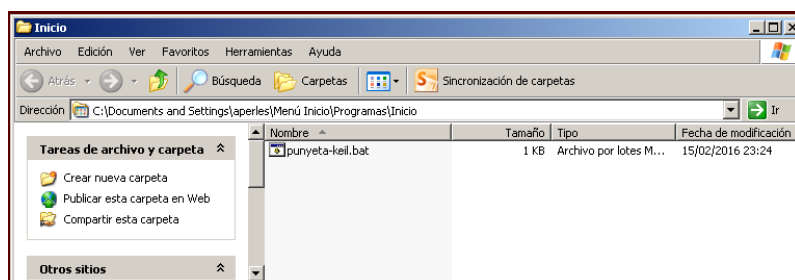


Figura 7.19: Carpeta de inicio de Microsoft Windows

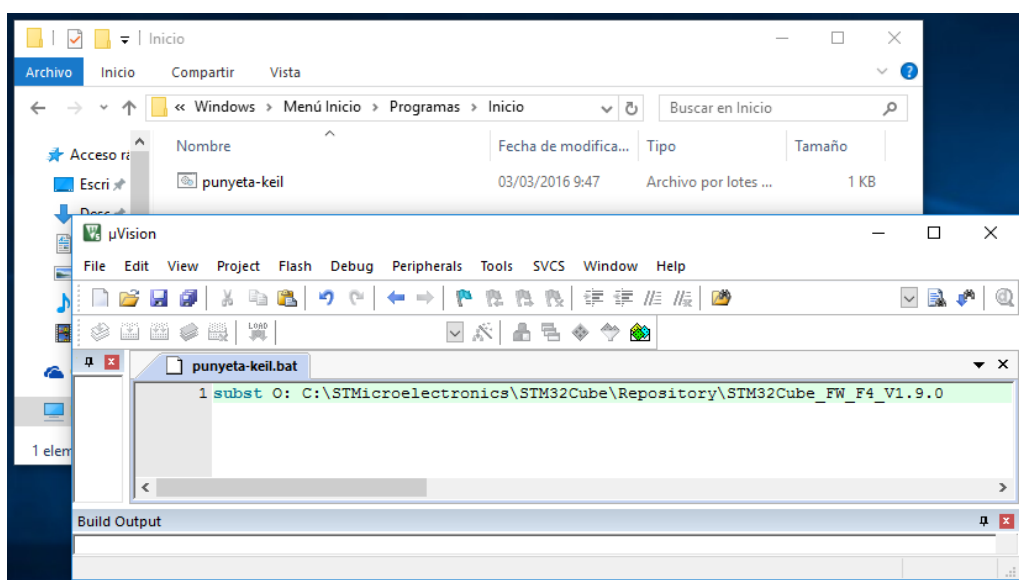


Figura 7.20: Editando el archivo de lotes para crear la unidad virtual O:.

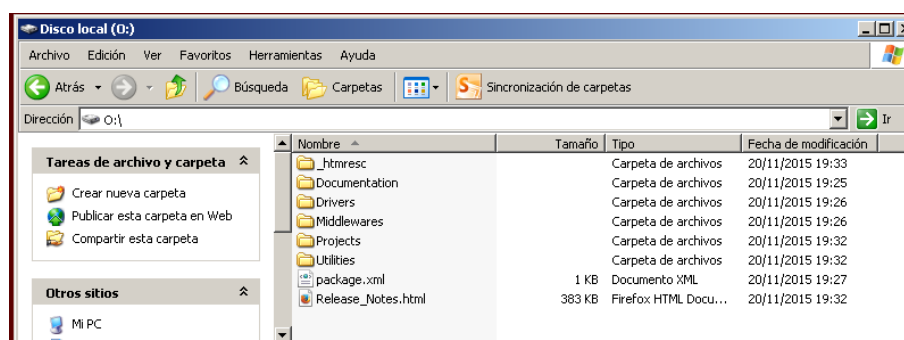


Figura 7.21: Contenido esperado de la unidad O:.

- Hacer doble click con el ratón sobre el archivo (se ejecutará) y comprobar en el explorador de windows que aparece la unidad O:. El contenido debe ser similar a la figura 7.21.
- Rearrancar Windows para comprobar que se lanza correctamente la aplicación.

7.8.2 Puente en la Discovery para habilitar

Para habilitar la redirección de printf() al servicio de depuración ITM/SWV. Cerrar el puente SB9 con un soldador y estaño según figura 7.22.

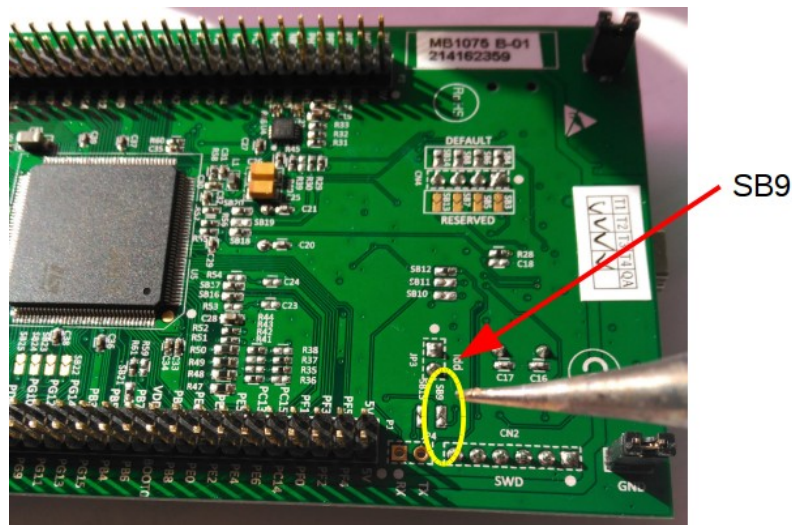


Figura 7.22: Cerrar puente SB9

7.8.3 Pendiente para la L4

Capítulo 8

Prácticas

8.1 Práctica: Instalación de St-Link y volcado de ejecutable en la placa St Discovery

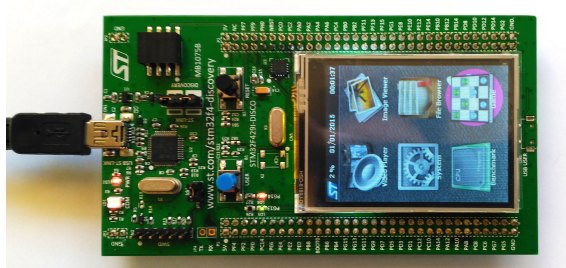


Figura 8.1: La Discovery funcionando con lo que se va a grabar.

8.1.1 Objetivos

- Instalar el software y controladores para St-link.
- Volcar un ejecutable en la placa Discovery.
- Ejecutar proyectos en la placa Discovery.

8.1.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Placa de evaluación Discovery STM32F429i-DISC1
- Cable USB tipo A a mini-B.

8.1.3 Instalación y comprobación de St-Link

NOTA: Se recomienda desactivar el antivirus.

Siguiendo la sección 7.4, instalar el software St-link que permite el aprovechamiento del sistema de depuración incorporado en la placa Discovery.

Una vez instalado el software, conectar la placa Discovery según indicaciones, esperar y comprobar que el sistema de depuración está correctamente instalado.

8.1.4 Actualización de la sonda St-Link de la Discovery

Siguiendo la sección 7.4, actualizar el firmware de la sonda. Es muy probable que sea necesario desconectar y volver a conectar la Discovery para que pase a modo DFU (Device Firmware Update).

8.1.5 Volcado de ejecutables en la Discovery

En la página web de este libro [**librostm32**], localizar el archivo ejecutable en formato IntelHEX *CubeLEDs* y descargarlo.

Mediante la aplicación *St-Link utility*, grabar el ejecutable en la placa y comprobar su funcionamiento.

Con el bloc de notas de Windows, abrir el archivo *CubeLEDs.hex* antes para comprobar su contenido. Busca información en Internet sobre el formato.

8.1.6 Volcado de la demo de la Discovery

La demo que viene de serie en la Discovery ha sido sustituida con la aplicación anterior.

El código fuente de dicha demo está disponible como un proyecto en lenguaje C, así que se ha reconstruido y el ejecutable resultante está disponible en [**librostm32**]. Descárgalo, grábalo en la placa y comprueba que funciona.

8.2 Práctica: Instalación y prueba de ARM Keil MDK

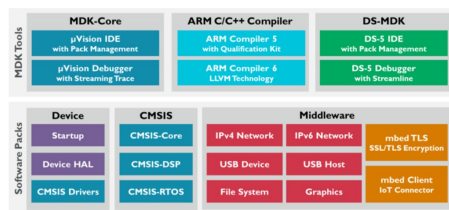


Figura 8.2: Componentes de ARM Keil MDK.

8.2.1 Objetivos

- Instalar Keil MDK-ARM 5.
- Construir proyectos para el microcontrolador.
- Volcar proyectos en el microcontrolador utilizando una sonda de depuración.

8.2.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1
- Cable USB tipo A a mini-B.

8.2.3 Instalación del software

Antes de proceder, recuérdese que es imprescindible estar conectados a la red para hacer esta instalación y que se sugiere desactivar el software antivirus.

Siguiendo la sección 7.5, instalar el entorno de desarrollo Keil.

8.2.4 Construcción del proyecto “CubeLEDs”

En la página web de este libro [[librostm32](#)], localizar el proyecto *CubeLEDs* y descargarlo. Es un archivo comprimido que se deberá desgargar en un subdirectorio y descomprimirlo. Evitar rutas de directorio con espacios y/o símbolos no anglosajones (ñ, ç, acentos, etc.).

Siguiendo la sección 7.5, abrir el proyecto entrando en el subdirectorio MDK-ARM y localizando el archivo `CubeLEDs.uvprojx` (es posible que Windows oculte la extensión, dificultando su localización).

La apertura de un proyecto puede forzar el lanzamiento del gestor de paquetes de Keil. Si es la primera vez que se hace, el gestor de paquetes requerirá la descarga de un paquete muy grande y su posterior desempaquetado, lo que puede requerir muchos minutos. La figura 8.3 muestra el aspecto de este proceso.

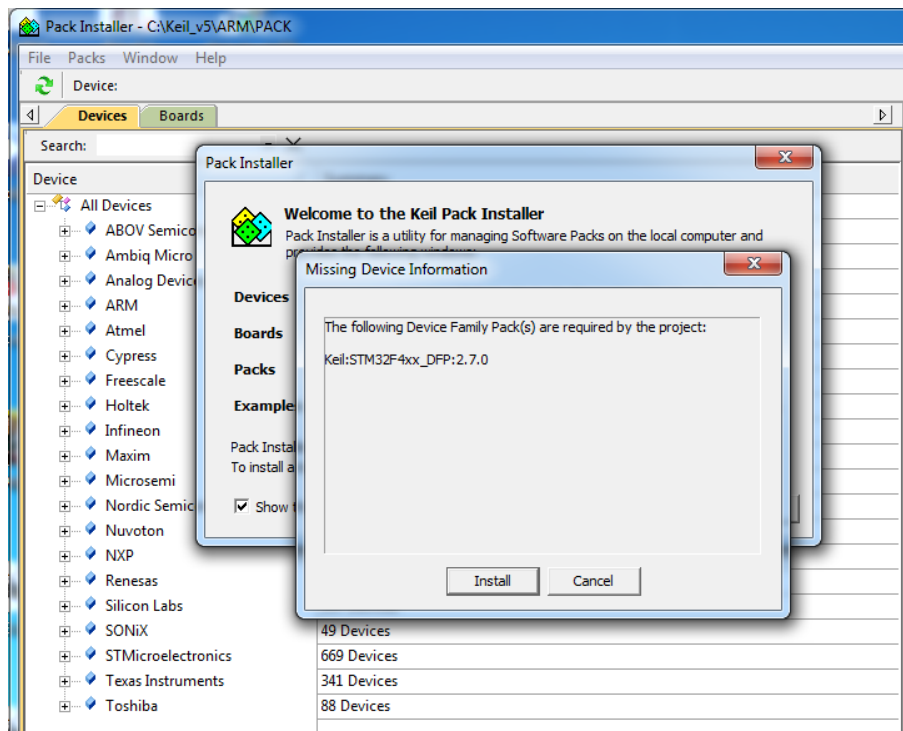


Figura 8.3: Gestor de paquetes solicitando una actualización.

Una vez descargado el paquete e instalado, se puede continuar con el uso de Keil para construir el proyecto según la instrucciones de la sección 7.5. Proceder y comprobar que el proyecto se construye correctamente.

8.2.5 Volcado del proyecto en el microcontrolador

De nuevo, siguiendo la sección 7.5, proceder al volcado y ejecución del proyecto en la placa.

Comprobar su funcionamiento, que debe ser el parpadeo de los LEDs verde y rojo.

8.2.6 Ampliación: cambio en el parpadeo

Localizar el módulo `main.c` y encontrar la función `main()`. Modificarla mediante la técnica de la intuición de manera que el LED rojo permanezca el doble de tiempo encendido que el verde.

Si la técnica de la intuición ha funcionado, significa que el código está bien desarrollado y es legible. Sigue ese camino cuando hagas programas.

8.3 Práctica: Instalación y prueba de las STM32Cube F4 HAL

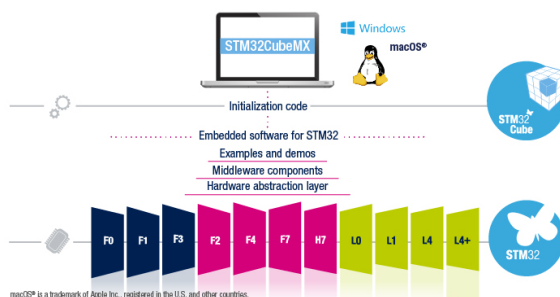


Figura 8.4: Logos del sistema STM32Cube.

8.3.1 Objetivos

- Instalar las bibliotecas HAL.
- Probar un ejemplo proporcionado con las HAL.

8.3.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package

8.3.3 Instalación y prueba de las STM32F4Cube HAL

NOTA: Desactivar el antivirus antes de proceder a la instalación de la herramienta STM32CubeMx y de las HAL. Harán falta permisos de administrador.

Instalación de STM32CubeMx

La aplicación STM32CubeMX no es un requisito imprescindible, pero facilitará el trabajo futuro de actualización de las HAL.

Siguiendo escrupulosamente la sección 7.6, instala/actualiza STM32CubeMx. Hay una opción para actualizar, pero deberás también seguir escrupulosamente los pasos

que te indica, sobre todo, arrancar la aplicación con permisos de administrador cuando se haga la actualización.

Instalación de las HAL STM32CubeF4

El siguiente paso es instalar las bibliotecas para la serie de microcontroladores STM32F4.

Siguiendo la sección 7.7, instala dichas bibliotecas.

Comprobación de un proyecto ejemplo

Localiza el directorio donde se ha colocado el firmware y explora los ejemplos disponibles.

Con el entorno de desarrollo elegido, localiza el proyecto `Projects/STM32F429I-Discovery/Examples/GPIO/GPIO_EXTI`, entra en el subdirectorio MDK-ARM y abre el archivo de proyecto. El directorio MDK-ARM es la configuración específica para Keil, siendo otros directorios para otros entornos.

Construye el proyecto y vuélcalo en la placa. Cuando esté en ejecución, este proyecto cambia el estado del LED verde con cada pulsación del pulsador de usuario azul.

8.4 Práctica: Uso de la plantilla oficial



Figura 8.5: El lcd usado como una pantalla normal.

8.4.1 Objetivos

- Poner a punto un proyecto a partir de la plantilla oficial.
- Aprovechar la redirección de la salida estándar.

8.4.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].

8.4.3 Introducción

Para facilitar el trabajo de la asignatura, se ha preparado una plantilla preconfigurada.

Para usarla, es necesario tener las HAL instaladas previamente.

8.4.4 Preparación inicial

Estos pasos se hacen solo una vez.

Crear unidad de disco O:

Esto es un poco enrevesado, pero la culpa la tiene el entorno de trabajo que usamos.

Siguiendo la sección 7.8, crea la unidad O:. Comprueba que funciona correctamente abriéndola desde el explorador.

Prepara tu entorno de trabajo

Según vaya avanzando el curso se irán desarrollando proyectos. Para tenerlos localizados se recomienda crear un subdirectorío, para ello, crear un directorío en un lugar adecuado; por ejemplo, C:\ii2\proyectos o en la unidad de red W:.

8.4.5 Usando la plantilla

Preparar la plantilla

Sigue los siguientes pasos para preparar un proyecto a partir de ella:

- Descargar la plantilla desde la página web del libro [librostm32].
- Descomprimirla en el raíz de la carpeta de proyectos.
- Renombrarla al nombre de proyecto deseado, por ejemplo prueba.

Construir la plantilla

Con el entorno de desarrollo Keil, entra en el subdirectorío MDK-ARM y abrir el archivo `Project.uvprojx`.

Probar a construir y a volcar en la placa para asegurarte de que todo está en perfectas condiciones.

8.4.6 Salida estándar con la plantilla

Una de las características de la plantilla es que incluye un módulo para que la llamada “salida estándar” funcione. Esto de la salida no es más que la capacidad de mostrar texto de las funciones tipo `printf()`.

Salida estándar por servicio de depuración

Por defecto, la plantilla se ha diseñado para que la salida estándar se redirija a un servicio especial de depuración.

Dependiendo de la placa usada, este servicio puede no estar deshabilitado. En el caso de la usada en el libro, es necesario realizar un puente siguiendo las instrucciones de la sección 7.8.2.

Asumiendo una placa con el servicio habilitado y el proyecto correctamente construido, seguir estos pasos:

- Arrancar el depurador.
- Abrir la ventana de salida del servicio de redirección al SWV-ITM debug. Ver figura 8.6.

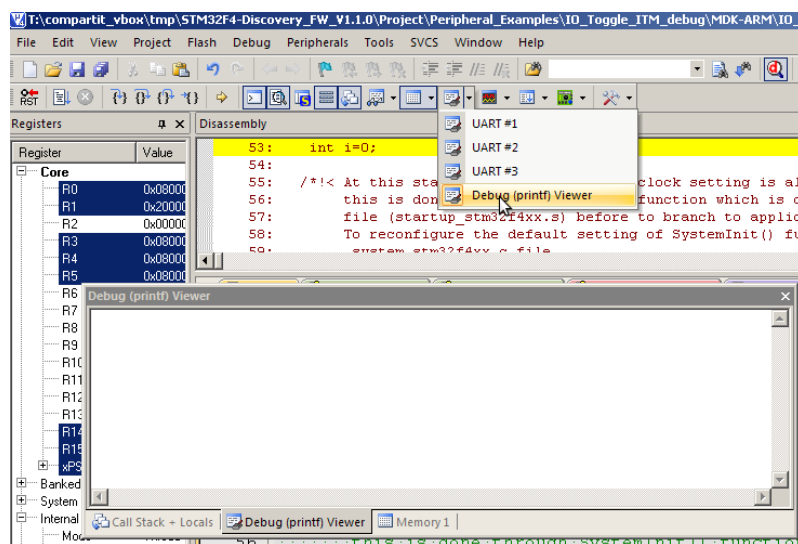


Figura 8.6: Ventana de salida de la redirección SWV.

- Ejecutar la aplicación desde el entorno de desarrollo (No usar el reset de la placa).

El mensaje “Hola” debería salir en la ventana de salida. Si no es así, seguir la explicación “STM32F4 Discovery and printf() redirection to debug viewer in Keil MDK-ARM”.

Actividad: Modificar el mensaje de salida

Editar el módulo `main.c` para cambiar el mensaje que sale por el terminal.

Crear un contador, ir incrementándolo y mostrar su valor en el terminal.

Salida estándar en el LCD

Ya que tenemos un hermoso display gráfico, parece tonto que las cosas salgan por ahí y no por la pantalla ¿no?.

NOTA: En realidad lo tonto es que salga por el LCD, pero mola. Si te dedicas a esto verás que la opción anterior es muy útil.

Se pretende pues conseguir que `printf()` funcione en la pantalla LCD y que, de paso, se comprenda de qué va el asunto y se sepa aplicar el principio a cualquier tipo de dispositivo de salida.

Para ello, elimina del proyecto el módulo `fputc_debug.c` y añade en su lugar el módulo `fputc_lcd.c` que se encuentra en la carpeta de la plantilla `Src`, que es la misma donde está el módulo anterior. En la figura 8.7. Usa el botón derecho para eliminar y añadir archivos en tu proyecto.

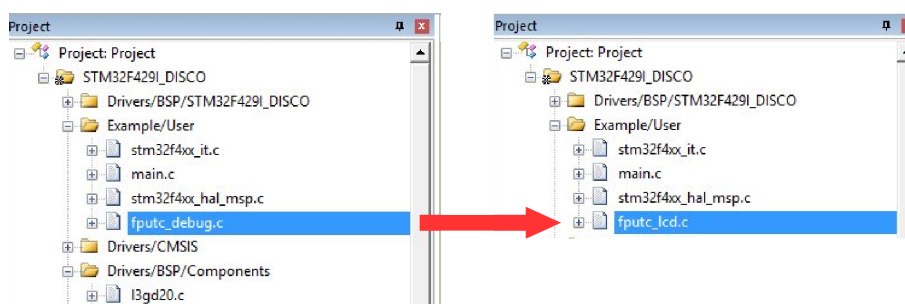


Figura 8.7: Cambiando el archivo que gestiona la salida estándar.

Prueba a construir de nuevo el proyecto, vuélcalo en la placa y tachánnnnn!. Ver figura 8.8.

Actividad: Usando el printf en el LCD

Está claro que el LCD no es la salida de depuración, así que se han añadido una serie de funciones para gestionarlo. Destacar la función `fputc_SetXY()` que permite establecer la posición en la que se escribe.

Modifica `main()` de manera que quede de la siguiente forma.

```
void fputc_SetXY(uint16_t x, uint16_t y);

int main(void)
```

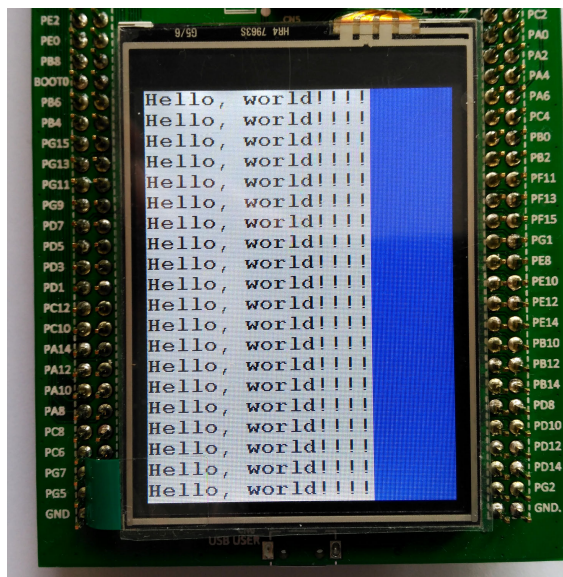


Figura 8.8: Cambiando el archivo que gestiona la salida estándar.

```
{
    int32_t contador = 0;

    HAL_Init();
    SystemClock_Config();

    printf ("Hola, mundo\n");
    printf ("feliz");

    fputc_SetXY(10,50);
    printf("Contador");

    while (1) {
        fputc_SetXY(130,50);
        printf("%6d", (int) contador);
        contador++;
        HAL_Delay(10);
    }
}
```

Y prueba, debería salir similar a la figura 8.5.

8.4.7 Qué puñetas es eso de la salida estándar

Las funciones de la biblioteca “estándar” a las que se accede a través de la cabecera `stdio` de C (`printf()`, `scanf()`, ... terminan en una función encargada de sacar/recoger caracteres por un dispositivo de salida determinado. Este dispositivo puede ser lo que sea, pero en los computadores normales suelen ser un teclado y una pantalla. Hablamos entonces de salida estándar (`stdout`), entrada estándar (`stdin`), ...

Tanto en un microcontrolador como en un ordenador normal es fácil cambiar a dónde van esos caracteres, pero en un microcontrolador es más probable que haya

que meter mano ahí. El procedimiento suele consistir en proveer una función con un nombre muy concreto y bastante universal. Para Keil, bastará con proveer la función `fputc()`.

Para hacerse una idea, ahí va el listado del módulo `fputc_lcd.c`:

```
/**
 * @file fputc.c
 * @author Angel Perles
 * @date 2016/04/10
 *
 * Redireccction of stdout to the lcd on the DiscoveryF29 board
 * Not optimal! only for going por casa
 */

#include <stdio.h>
#include "stm32f429i_discovery_lcd.h"

static uint16_t putc_x=0, putc_y=0;
static sFONT *fputc_font = &Font16; // font to use

void fputc_Init(void);
/**
 * @brief Retargets the C library printf function to the GLCD
 */

#ifdef __GNUC__ // <-- this is for gcc open compiler (gratis)
    int __io_putchar(int ch)
#else // <-- that is for Keil
    int fputc(int ch, FILE *f)
#endif /* __GNUC__ */
{
    static uint8_t first_time = 1;

    if (first_time) {
        first_time = 0;
        fputc_Init();
    }

    if (ch >= '\r') {
        BSP_LCD_DisplayChar(putc_x, putc_y, ch);
        putc_x += fputc_font->Width;
    } else if (ch == '\n') { // in this Keil case is \x0A
        putc_y += fputc_font->Height;
        putc_x = 0;
    }

    return (ch);
}

/**
 * @brief Set the coordinates bla, bla altre dia
 */
void fputc_SetXY(uint16_t x, uint16_t y) {

    putc_x = x;
    putc_y = y;
}

/**
 * @brief Prepares the display
 */
```

```
void fputc_Init(void) {
    BSP_LCD_Init(); /* Initialize the LCD */
    BSP_LCD_LayerDefaultInit(1, 0xD0000000); /* Layer2 Init */
    BSP_LCD_SelectLayer(1); /* Set Foreground Layer */
    BSP_LCD_Clear(0xFFFFFFFF); /* Clear the LCD */
    BSP_LCD_SetLayerVisible(1, DISABLE);
    BSP_LCD_LayerDefaultInit(0, 0xD0000000);
    BSP_LCD_SelectLayer(0); /* Set Background Layer */
    BSP_LCD_DisplayOn(); /* Enable The LCD */
    BSP_LCD_Clear(0xFF0000FF); /* Clear the LCD */

    BSP_LCD_SetFont(fputc_font);
    BSP_LCD_SetTextColor(0xFF000000);
}
```

No es la intención explicar de que va, solo usarlo.

8.5 Práctica: Bibliotecas



Figura 8.9: El concepto de biblioteca en C.

8.5.1 Objetivos

- Emplear y organizar adecuadamente las bibliotecas externas.

8.5.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [[librostm32](#)].
- Biblioteca `led` disponible en la web del libro [[librostm32](#)].

8.5.3 Introducción

Las bibliotecas son la base del desarrollo de sistemas embebidos en C.

En este caso, se pretende ilustrar el uso de una biblioteca muy sencilla desarrollada por una tercera parte.

Como la biblioteca usa las HAL para hacer salida digital, no está de más echarle un buen repaso.

8.5.4 Preparación inicial

Descargar y preparar la plantilla oficial. Antes de tocar nada, comprobar que se construye correctamente y se puede volcar y ejecutar en la placa.

8.5.5 Incorporar la biblioteca led

Siguiendo la sección 6.7, crear el directorio apropiado en la carpeta del proyecto y descargar y descomprimir la biblioteca LED. La biblioteca LED está formada por los archivos `led.c` y `led.h`.

Siguiendo la sección 6.7, incorpora `led.c` al proyecto. Prueba a construir el proyecto.

Incluir ahora el archivo `led.h` en la ruta de búsqueda de bibliotecas siguiendo la sección 6.7. Esto es distinto a añadir el archivo al proyecto.

Construye de nuevo el proyecto para comprobar que no has roto nada.

8.5.6 Probar el módulo LED

Editar la función `main()` para dejarla similar al siguiente listado.

```
//#include <stdio.h>
#include <led.h>    // o #include "led.h"

int main(void)
{
    // MANTENER AQUI EL CODIGO DE INICIALIZACION DEL HAL

    LED_Init();

    while(1) {
        LED_On();
        HAL_Delay(1000);
        LED_Off();
        HAL_Delay(1000);
        //printf("Hola\n");
    }

    return 0;
}
```

Pruébese a construir de nuevo. Volcar en la placa y probar que funciona adecuadamente.

8.5.7 Actividad: cambio de LED

Modificar el código fuente de la biblioteca para que sea otro el LED que se maneja.

8.6 Práctica: Salida digital con una válvula

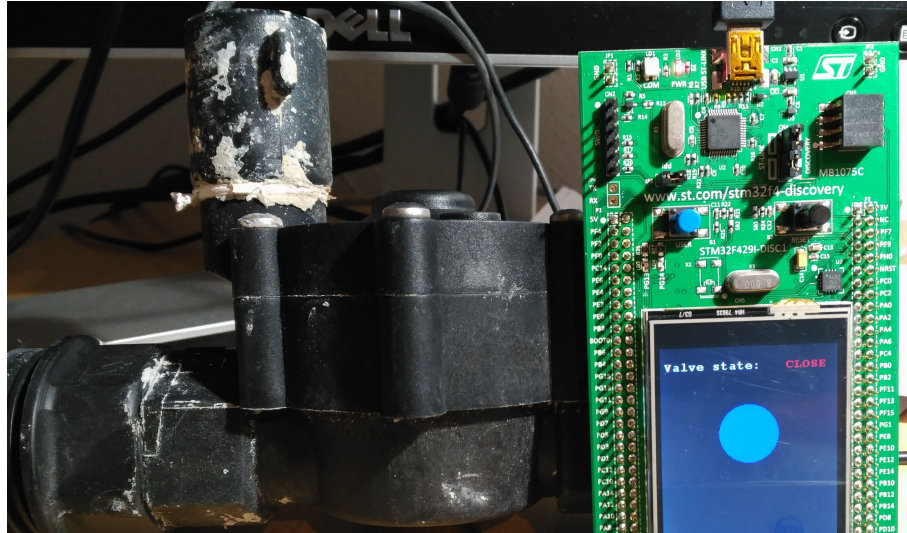


Figura 8.10: Una válvula digital

8.6.1 Objetivos

- Programar la salida digital a partir de las especificaciones electrónicas.
- Desarrollar un módulo que haga uso de la salida digital.

8.6.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].

8.6.3 Introducción

Se propone el manejo de una válvula digital para practicar como se debe crear un módulo C que gestione una salida digital básica.

Destacar que la principal lección es darse cuenta de la importancia de la abstracción que proporciona este tipo de módulos.

Se obvia la parte de diseño electrónico del driver de la válvula.

8.6.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente y se puede volcar y ejecutar en la placa.

8.6.5 Actividad: desarrollar el módulo valve

Se propone el siguiente archivo de cabecera para `valve.h`

```
/**
 * @file valve.h
 */

#ifndef VALVE_H
#define VALVE_H

typedef enum {VALVE_OPEN, VALVE_CLOSE} TValveState;

void valve_Init(void);
void valve_Set(TValveState state);

#endif
```

A partir de esta definición, realizar la implementación del módulo `valve.c` utilizando como salida el LED conectado al puerto PG14 (¿color?).

8.6.6 Actividad: comprobar el funcionamiento del módulo

Los módulos desarrollados se deben comprobar por separado antes de integrarlos en una aplicación. En general, se usa la técnica del *unit testing* para este menester.

En nuestro caso se propone desarrollar un programa principal que compruebe el funcionamiento. Se propone el siguiente listado.

```
#include <stdio.h>
#include "valve.h"

int main(void)
{
    // MANTENER AQUI EL CODIGO DE INICIALIZACION DEL HAL

    valve_Init();

    while (1)
```

```
{
    valve_Set (VALVE_OPEN);
    //printf("Open valve\n");
    HAL_Delay(1000);
    valve_Set (VALVE_CLOSE);
    //printf("Close valve\n");
    HAL_Delay(1000);
}
```

Introdúcelo y comprueba su funcionamiento.

8.6.7 Actividad opcional: mostrar mensajes por la salida de depuración

Descomenta los mensajes que informan del estado de la válvula y hazlos aparecer por la ventana de la salida de depuración.

8.6.8 Actividad opcional: mostrar información por el LCD

Adapta el proyecto para usar la pantalla y mostrar los mensajes por ella. Como ejemplo de la información a mostrar, echa un vistazo a la figura 8.10.

Las funciones para manejo del LCD están contenidas en el archivo `stm32f429i_discovery_lcd.c`.

Por ejemplo, la función `BSP_LCD_SetTextColor()` permite especificar el color del texto, donde el parámetro de entrada es un número de 32 bits, que, si se expresa en hexadecimal, tiene el formato `0xTTRRGGBB`, donde `RR` es el nivel de rojo, `GG` es el nivel de verde y `BB` es el nivel de azul.

Si se quisiese verde al máximo de brillo se haría:

```
BSP_LCD_SetTextColor(0xFF00FF00);
```


8.7 Práctica: Entrada digital con un sensor de rebose



Figura 8.11: Un sensor de nivel digital GENTECH INTERNATIONAL - LS304-51N.

8.7.1 Objetivos

- Programar la entrada digital a partir de las especificaciones electrónicas.
- Desarrollar un módulo que haga uso de la entrada digital.

8.7.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].

8.7.3 Introducción

Se propone el manejo de un sensor digital para practicar como se debe crear un módulo C que gestione una entrada digital básica, de manera que se abstraiga y oculte su implementación.

Para detectar la posibilidad de rebose de un depósito de líquidos, se propone emplear un sensor de boya típico como el mostrado figura 8.11 con referencia GENTECH

INTERNATIONAL - LS304-51N. Este sensor ofrece una salida digital en función del estado de la boya.

Para no desperdiciar tiempo montando hardware, se propone imitar su funcionamiento utilizando el pulsador de la placa Discovery cuyo esquemático se reproduce en la figura 8.14.

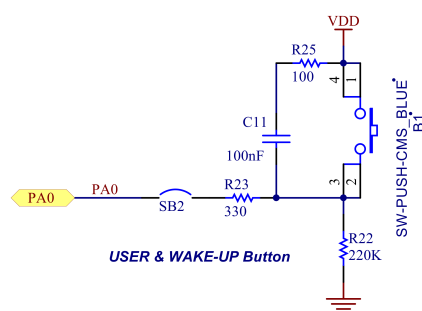


Figura 8.12: Pulsador de usuario de la STM32F429i-Discovery.

8.7.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente y se puede volcar y ejecutar en la placa.

8.7.5 Actividad: desarrollar la biblioteca overflow

Se pretende que se desarrolle un módulo C, con nombre `overflow.c`, que lea el sensor y ofrezca las funciones según el archivo de cabecera `overflow.h` listado a continuación:

```
/**
 * @file overflow.h
 * @brief Public functions for handling the overflow sensor
 */

#ifndef OVERFLOW_H
#define OVERFLOW_H

typedef enum {OVERFLOW_YES, OVERFLOW_NO} TOverflowStatus;

void overflow_Init(void);
TOverflowStatus overflow_Read(void);

#endif
```

Crear la biblioteca en el directorio apropiado de la carpeta de proyectos para la sección de terceras partes. Copiar en él la cabecera con nombre `overflow.h`.

Desarrollar a continuación el módulo con nombre `overflow.c` que implementa la funcionalidad deseada.

Añadir todos los elementos convenientemente al proyecto y construir.

8.7.6 Actividad: comprobar el módulo desarrollado

Desarrollese un programa en la función `main()` para inicializar y leer continuamente el sensor y mostrar su estado en la ventana de depuración o en el display.

NOTA: En el caso de elegir la ventana de depuración, se deberá hacer un retardo grande en cada iteración para evitar saturar el canal de depuración.

8.7.7 Actividad opcional: desarrollar una aplicación de control

Añadir el módulo `valve` desarrollado en la práctica 8.6 y modificar el bucle de manera que la válvula se abra cuando el sensor de rebose no indica rebose y se cierre en caso contrario.

8.7.8 Actividad opcional: conexión real de la válvula

Lo ideal sería desarrollar el hardware de conexión; para conocer el funcionamiento del sensor, localizar la hoja de características en Internet y hacer una propuesta de montaje.

8.8 Práctica: Entrada digital con máscaras

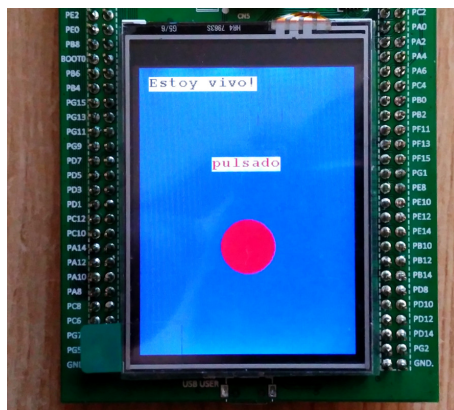


Figura 8.13: Pantalla de la Discovery con el resultado esperado.

8.8.1 Objetivos

- Programar la entrada digital a partir de las especificaciones electrónicas.
- Desarrollar un módulo que haga uso de la entrada digital mediante máscaras.

8.8.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Biblioteca STM32F4xx_AP_GPIO disponible en [librostm32].

8.8.3 Introducción

En esta práctica se propone el uso de máscaras para el análisis de los bits de un dato que se toma desde una entrada digital. El conocimiento de las operaciones basadas en máscaras son imprescindibles para la programación de sistemas embebidos.

Para practicar este concepto se empleará el pulsador de la placa Discovery cuyo esquemático se reproduce en la figura 8.14.

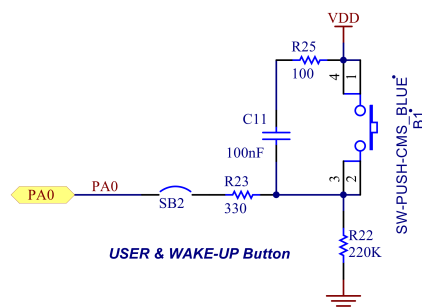


Figura 8.14: Pulsador de usuario de la STM32F429i-Discovery.

8.8.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente y se puede volcar y ejecutar en la placa.

8.8.5 Actividad: incorporar biblioteca STM32F4xx_AP_GPIO

Incorporar al proyecto la biblioteca STM32F4xx_AP_GPIO disponible en [librostm32], tanto el módulo .c como la búsqueda de las cabeceras .h.

Construir de nuevo el proyecto para comprobar que sigue siendo consistente.

8.8.6 Actividad: incorporar biblioteca de lectura del pulsador (button)

Incorporar la versión basada en máscaras de la biblioteca button vista en la sección ??.

Construir de nuevo el proyecto para comprobar que sigue siendo consistente.

8.8.7 Actividad: comprobación de la biblioteca button

Siguiendo la sección 3.6.2, crea un programa principal que demuestre que dicha biblioteca funciona. Recuerda mantener las funciones de inicialización del microcontrolador.

Para ver el efecto de la biblioteca puedes usar un LED como salida o la salida de printf().

8.8.8 Actividad opcional: mostrar estado del pulsador en el LCD

La pantalla gráfica ya se ha usado en actividades previas, pero la mayor parte de las funcionalidad estaba oculta en otro módulo.

En esta ocasión se pretende mostrar los pasos a seguir para emplearlo y, de paso, aprovecharla para mostrar el estado del pulsador.

La plantilla incorpora suficientes módulos para manejar la pantalla. Se propone comentar el código de `main()` previo y añadir el siguiente código para realizar una primera prueba de funcionamiento de la pantalla modificando `main()` de la siguiente manera:

```
BSP_LCD_Init(); /* Initialize the LCD */
BSP_LCD_LayerDefaultInit(1, 0xD0000000); /* Layer2 Init */
BSP_LCD_SelectLayer(1); /* Set Foreground Layer */
BSP_LCD_Clear(0xFFFFFFFF); /* Clear the LCD */
BSP_LCD_SetLayerVisible(1, DISABLE);
BSP_LCD_LayerDefaultInit(0, 0xD0000000);
BSP_LCD_SelectLayer(0); /* Set Background Layer */
BSP_LCD_DisplayOn(); /* Enable The LCD */
BSP_LCD_Clear(0xFF0000FF); /* Clear the LCD */

BSP_LCD_SetFont(&Font16);
BSP_LCD_SetTextColor(0xFF000000);
//BSP_LCD_SetBackColor(0x66FFFF);
BSP_LCD_DisplayStringAt(10, 10, (uint8_t *)"Estoy_vivo!", LEFT_MODE);

/* Infinite loop */

while (1)
{
    HAL_Delay(1000);
    BSP_LCD_SetTextColor(0xFFFF0000);
    BSP_LCD_DisplayStringAt(80, 100, (uint8_t *)"pulsado", LEFT_MODE);
    BSP_LCD_FillCircle(120, 200, 30);

    HAL_Delay(1000);
    BSP_LCD_SetTextColor(0xFF00FF00);
    BSP_LCD_DisplayStringAt(80, 100, (uint8_t *)"suelto", LEFT_MODE);
    BSP_LCD_FillCircle(120, 200, 30);
    //printf("Hello, world!!!!\n");
}
```

Para que el módulo compile, incluir la cabecera `stm32f429i_discovery_lcd.h`.

Comprobar que el proyecto funciona construyéndolo y volcándolo en la placa de prácticas.

A continuación reincorporar de nuevo el código del pulsador de manera que se inicialice el hardware del pulsador y, en el bucle principal, se llame a la función de lectura del pulsador y se actualice la pantalla adecuadamente.

8.9 Práctica: Optimizaciones de código para el display de 7 segmentos

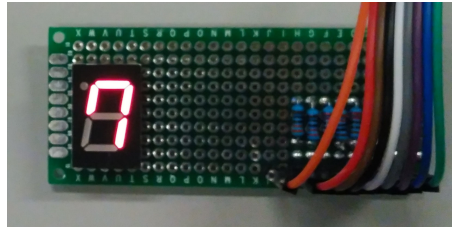


Figura 8.15: El display funcionando.

8.9.1 Objetivos

- Desarrollar un módulos que hagan uso de la salida mediante máscaras.
- Aplicar técnicas de tablas de look-up.
- Evaluar el impacto en la memoria de programa y de variables de distintas implementaciones.

8.9.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Opcional: Placa evaluación Discovery STM32F429i-DISC1.
- Opcional: Cable USB tipo A a mini-B.
- Opcional: Display 7 segmentos de cátodo común, resistencias de YYY, board, cables Dupont macho-hembra.

8.9.3 Introducción

Un aspecto crítico en los desarrollos para sistemas embebidos son las restricciones en cuanto a espacio de memoria requerido o al tiempo de ejecución.

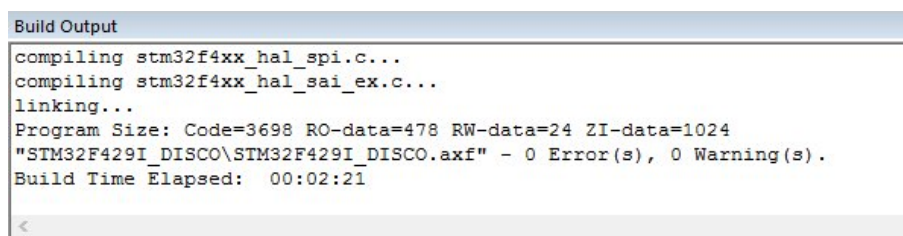
En esta práctica se pretende mostrar el efecto en cuanto a tamaño de memoria del uso de las técnicas de tablas de look-up. En particular, se probarán las optimizaciones desarrolladas para el display de 7 segmentos.

8.9.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. No es necesario volcarla en la placa.

Cuando el proyecto se construya correctamente, observar el informe de salida de la construcción, que será similar al mostrado en la figura 8.16. En la línea *Program size* se muestra el resumen de ocupación, que será:

- *Code* es el tamaño de programa (FlashROM).
- *RO-data* son los datos constantes (FlashROM).
- *RW-data* es el espacio para variables (RAM).
- *ZI-data* es el espacio de variables inicializado a 0 (RAM).



```

Build Output
compiling stm32f4xx_hal_spi.c...
compiling stm32f4xx_hal_sai_ex.c...
linking...
Program Size: Code=3698 RO-data=478 RW-data=24 ZI-data=1024
"STM32F429I_DISCO\STM32F429I_DISCO.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:02:21

```

Figura 8.16: Informe de construcción de Keil.

Anótese en una tabla estos valores.

8.9.5 Modificar main()

Incorpora ahora las funciones que hacen uso del display de 7 segmentos modificando `main()` para añadir el siguiente código:

```

uint8_t n;

// funciones inic. HAL y sistema, NO BORRARLAS

display7seg_Init();

while(1) {
    for (n=0; n <= 10; n++) {
        display7seg_SetNumber(n);
        HAL_Delay(500);
    }
}

```

Coloca el archivo de cabecera `display7seg.h` en el lugar adecuada e inclúyela en el módulo que contiene la función `main()`.

Comprueba que el módulo `main()` compila correctamente, pero que no se puede construir el proyecto.

8.9.6 Tarea: Incorporar las distintas implementaciones de display7seg.c

En el lugar apropiado, coloca las distintas versiones de la implementación del display y llámalas, por ejemplo, `display7seg-v1.c` (con switch), `display7seg-v2.c` (con tabla) y `display7seg-v3.c` (con tabla y `const`), etc.

Incorporar uno de los módulos y comprueba que la aplicación se construye correctamente. Anota los valores de consumo de memoria que te muestra el entorno de desarrollo al final de la construcción.

Repite el proceso para cada módulo. Indica qué diferencias hay y por qué.

Para que te hagas una idea, algunos resultados reales son:

Program Size: Code=3698 RO-data=478 RW-data=24 ZI-data=1024

Program Size: Code=3654 RO-data=486 RW-data=24 ZI-data=1024

Program Size: Code=3654 RO-data=478 RW-data=36 ZI-data=1028

8.9.7 Tarea: Optimizaciones del compilador

Al contrario que la plantilla de St, la plantilla oficial del curso está configurada para que el compilador no optimice el código. La razón es facilitar tareas de depuración.

Incrementa el nivel de optimización al máximo modificando la opción “Optimization” del compilador mostrada en la figura 8.17.

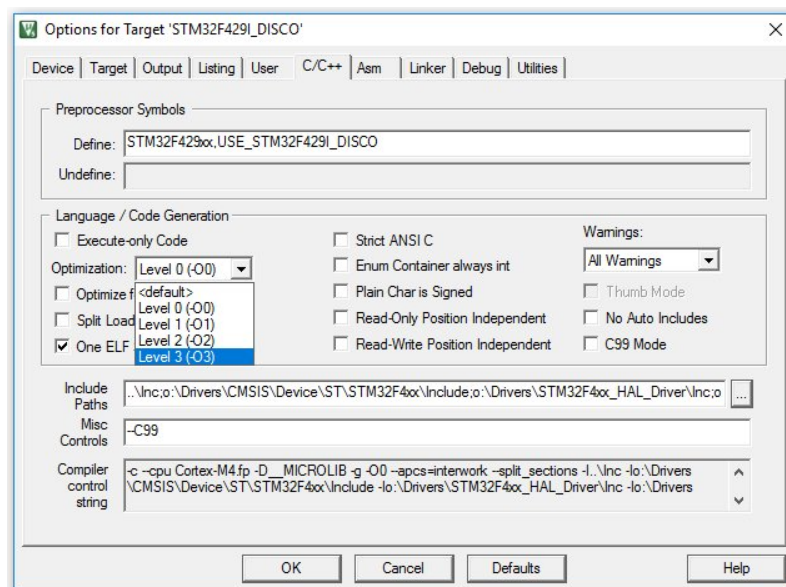


Figura 8.17: Informe de construcción de Keil.

Reconstruir completamente el código y comprobar los nuevos resultados. Anotarlo en la tabla.

8.9.8 NO HACER: Tarea: Probar el display

En una board, montar el display y las resistencias de caída.

Si se comprueba el manual de la Discovery, los puertos están muy ocupados y es imposible realizar la asignación propuesta en la parte teórica del libro. Como solución, se propone realizar la siguiente asignación de líneas a segmentos del display:

- Líneas PE6 ... PE2 a segmentos EDCBA.
- Líneas PG3 ... PG2 a segmentos GF.

El programa módulo `display7seg.c` hay que modificarlo para inicializar dichas líneas y modificar la sección que saca los datos por la líneas, por ejemplo (NO PROBADO):

```
// ...

uint32_t tmp;

/* EDCBA segments */
tmp = out;
tmp = (tmp & 0x001F) << 2;
HAL_GPIO_WritePin (GPIOE, 0x007B, GPIO_PIN_RESET);
HAL_GPIO_WritePin (GPIOE, tmp, GPIO_PIN_SET);

/* GF segments */
tmp = out;
tmp = (tmp & 0x0060) >> 3; // that is >> 5, and << 2
HAL_GPIO_WritePin (GPIOG, 0x000B, GPIO_PIN_RESET);
HAL_GPIO_WritePin (GPIOG, tmp, GPIO_PIN_SET);
```

8.10 Práctica: Usando EXTI para contar vehículos en una carretera

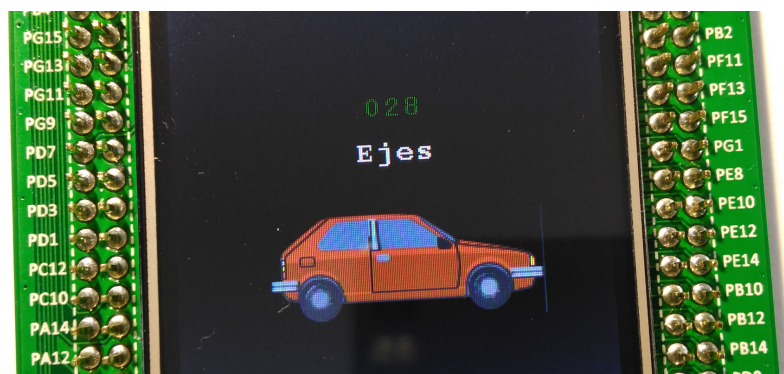


Figura 8.18: El contador de vehículos.

8.10.1 Objetivos

- Configurar el periférico EXTI para generar interrupciones.
- Desarrollar un módulo para gestionar la contabilidad de vehículos basado en interrupciones.

8.10.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.

8.10.3 Introducción

Esta práctica pretende ilustrar los beneficios de usar interrupciones con el siguiente ejemplo.

Para contabilizar vehículos que circulan por una carretera se puede utilizar un tubo de contactos de acero que genera una salida digital cada vez que un vehículo la presiona.

Para practicar con el periférico EXTI, se propone hacer la contabilización mediante la detección de flancos en el pin PA0 del microcontrolador; para ello, se propone hacer la conexión según se muestra en la figura 8.19, que coincide con el pulsador azul de la placa Discovery.

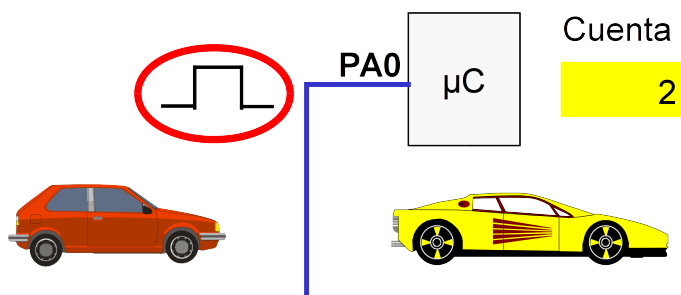


Figura 8.19: Diagrama del contador de vehículos.

8.10.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarlo en la placa para comprobar que todo está correctamente configurado.

8.10.5 Modificar main()

Modificar main() según el siguiente listado:

```
#include <stdio.h>
#include "stm32f4xx_hal.h"
#include "cars.h"

int main(void)
{
    // MANTENER AQUI EL CODIGO DE INICIALIZACION DEL HAL 1

    uint32_t count;

    cars_Init();

    while (1)
    {
        // read number of cars axis
        count = cars_GetCount();
        printf("Contabilizados %d ejes.\n", (int)count);

        // simulate other tasks
        HAL_Delay(4000);
    }
}
```

8.10.6 Tarea: Desarrollar la biblioteca cars

Desarrollar un módulo C con nombre `cars.c`, que gestione todo lo que tiene que ver con la contabilidad de vehículos y ofrezca las funciones según el archivo de cabecera `cars.h` listado a continuación:

```
/**
 * @file cars.h
 */

#ifndef CARS_H
#define CARS_H

#include <stdint.h>

void cars_Init(void);
uint32_t cars_GetCount(void);
void cars_IncrementCount(void);

#endif
```

Colocar este módulo y la cabecera en el mismo directorio que el resto de módulos principales (donde está `main.c`), pues forma parte de la aplicación y, por tanto no es necesario colocarlo en el directorio de terceros ni establecer la ruta de búsqueda de cabeceras.

Desarrolla las tres funciones dentro de `cars.c` sabiendo que se debe definir una variable global privada al módulo como sigue:

```
static volatile uint32_t cars_axis_count;
```

A continuación crea la función `cars_Init()` que pondrá a 0 la cuenta de ejes de coches y preparará el sistema de interrupciones para EXTI0 asociado al pin PA0 según se ha visto en teoría.

A continuación crea la función `cars_GetCount()` que deberá devolver el número de ejes contabilizados.

A continuación crea la función `cars_IncrementCount()` que sumará 1 al número de ejes contabilizados.

8.10.7 Tarea: Añadir el manejador

En el archivo `stm32f4_it.c` añade el manejador para EXTI0 según el siguiente listado:

```
#include "cars.h"

void EXTI0_IRQHandler(void)
{
    if(__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_0) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_0);

        cars_IncrementCount();
    }
}
```

```
}
```

8.10.8 Probar la aplicación

Comprobar la aplicación empleando el depurador y la opción de ventana de salida del canal de `printf()`.

Se puede pulsar muchas veces el botón para comprobar que se contabilizan adecuadamente las pulsaciones con independencia de la velocidad de ejecución del superbucle.

8.10.9 Ampliación: Salida por la pantalla gráfica

En una aplicación más real, la salida se debería hacer por la pantalla gráfica.

Utilizar la opción de redirección de `printf()` al display vista en la práctica ??.

Modifica `main()` para dejarlo, más o menos, así.

```
#include <stdio.h>
#include "stm32f4xx_hal.h"
#include "stm32f429i_discovery_lcd.h"
#include "cars.h"
// #include "ford.h"

// ...

int main(void)
{
    // MANTENER AQUI EL CODIGO DE INICIALIZACION DEL HAL 1

    uint32_t count;

    cars_Init();

    fputc_SetXY(100,110);
    printf("Ejes");

    BSP_LCD_SetTextColor(0x7F00FF00);
    // BSP_LCD_DrawBitmap(50,150,(uint8_t *)ford);

    while (1)
    {
        // read number of cars axis
        count = cars_GetCount();
        fputc_SetXY(105,85);
        printf("%03d", (int)count);

        // simulate other tasks
        HAL_Delay(4000);
    }
}
```

Comprueba el funcionamiento. Debería visualizarse ahora la información tal como se muestra en la figura 8.18 a excepción del Ford Fiesta rojo.

Se puede intentar añadir ahora el vehículo descargando el archivo `ford.h` de [librostm32] y colocándolo en el directorio con los fuentes de la aplicación. Este archivo no es más que una imagen BMP volcada en forma de un vector en C.

Por desgracia, el código resultante será mayor que el límite de la versión demo de Keil y no debería permitir construirlo (no probado).

8.10.10 Interrupción con método “callback”

(Añadir inlinning para el curso 2019/2020.)

En lugar de crear la función `cars_IncrementCount()`, añadir la función dentro de `cars.c` de “callback” del HAL de St. Obsérvese que tiene que ver con la ejecución de las acciones relacionadas con el paso del vehículo. Se corresponde al siguiente listado:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin == GPIO_PIN_0)
    {
        cars_axis_count++; // axis account
    }
}
```

En el archivo `stm32f4_it.c` añade el manejador para EXTI0 según el siguiente listado:

```
void EXTI0_IRQHandler(void)
{
    // St's HAL proposes to call this predefined function
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

8.10.11 Cambiar el nivel de prioridad de las interrupciones

Emplear las funciones HAL de gestión del nivel de las interrupciones para elevar el nivel de prioridad de esta interrupción al máximo y bajar la prioridad de la interrupción SYSTICK.

La modificación del agrupamiento de interrupciones se debería hacer antes de empezar a configurar la interrupción.

Intenta también revisar la plantilla para ver si St ya tiene hecha una configuración por defecto.

8.11 Haciendo pausas basadas en SysTick

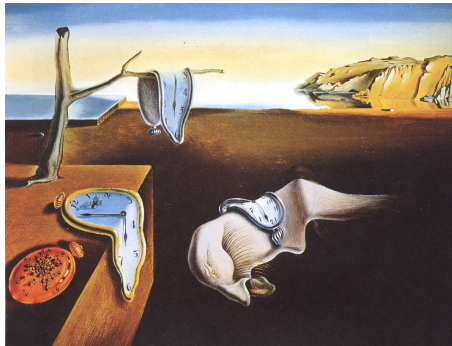


Figura 8.20: La persistencia de la memoria.

8.11.1 Objetivos

- Desarrollar código capaz de realizar pausas basada en tiempo.

8.11.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeF4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa evaluación Discovery STM32F429i-DISC1.
- Cable USB tipo A a mini-B.

8.11.3 Introducción

Una de las necesidades básicas de las aplicaciones para microcontrolador es ser capaces de medir el tiempo con muy distintos propósitos. Uno de estos propósitos es realizar pausas de precisión.

Como excusa para practicarlo, se propone la necesidad de hacer parpadear un LED a un ritmo continuo según el patrón: 300 ms encendido, 500 ms apagado, 300 ms encendido y 1 s apagado.

8.11.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarlo en la placa para comprobar que todo está correctamente configurado.

Añadir la biblioteca para encender y apagar el LED y comprobar que funciona correctamente. Úsese de momento la función `HAL_Delay()` para hacer parpadear el LED a cualquier ritmo.

8.11.5 Tarea: Implementar la biblioteca delay

Se pide crear una biblioteca específica para realizar las pausas siguiendo la propuesta trabajada en clase en la sección 5.3.3 donde se usaba `SysTick` para realizar pausas de precisión.

Para hacer el desarrollo, siguiendo la sección 6.7, crear el directorio apropiado en la carpeta de proyectos, sección “ThirdParty” y copiar en él la cabecera con nombre `delay.h`.

Desarrollar a continuación el módulo con nombre `delay.c` que implementa la funcionalidad deseada. El módulo se deberá basar en el uso de una variable global que se incrementa con las interrupciones de `SysTick`.

Comprobar que el módulo se construye correctamente antes de pasar a la siguiente sección.

8.11.6 Tarea: Efectos visuales con el LED

Modificar `main()` para incorporar el efecto visual indicado en la descripción del problema.

8.11.7 Tarea: Verificar la temporización

Modificar `main()` para que el LED se encienda durante un periodo de 20 segundos y comprobar con un cronómetro la corrección de la pausa.

8.11.8 Tarea avanzada: Tareas en segundo plano

Lograr eliminar todo el código de parpadeo de `main()` manteniendo un superbucle vacío y hacer que el parpadeo se produzca desde el manejador de interrupciones de `SysTick`.

Recuérdese que el código en el manejador ha de ser muy eficiente y no pueden realizarse pausas.

8.12 MBED: Desarrollo ARM en la nube



Figura 8.21: La nube MBED

8.12.1 Objetivos

- Conocer las posibilidades de desarrollo embebido ARM en la nube.

8.12.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior
- Placa evaluación STM32F429i-DISC1
- Cable USB tipo A a mini-B
- Conexión a Internet

8.12.3 Introducción

Hay una tendencia generalizada a que el desarrollo de aplicaciones y su gestión sea posible a través de servicios en la nube. En esta práctica se pretende ilustrar el uso de la plataforma ARM MBED para desarrollo embebido para microcontroladores.

MBED empezó como una plataforma en la nube que desarrollaron unos amiguets e incluía un compilador en la nube, un gestor de repositorios Mercurial, una plaquita basada en un NXP LPC1768 y una comunidad abierta donde te podías registrar y compartir tus proyectos.

El principio MBED está ahora presente en otras páginas, pero destacar que ARM se ha hecho con MBED y ha elevado notablemente su valor estratégico premitiendo la incorporación de más plataformas y creando un RTOS (real-time operating system) para la IoT (Internet of Things).

8.12.4 Descripción del problema

Se pretende usar MBED para lograr volcar un proyecto en la Discovery. Es necesario disponer de una Discovery MBED-enabled para poder realizar esta práctica, por ejemplo la STM32F429i-DISC1.

8.12.5 Preparación inicial

Antes de poder utilizar MBED sobre nuestra placa hay que seguir los siguientes pasos:

- Registrarse en la plataforma MBED
- Seleccionar la placa compatible con la que se desea trabajar
- Activar el compilador para la placa seleccionada

Vamos ayá.

Registro en MBED

Accedemos a <https://www.mbed.com/en/> y entraremos en la página principal del proyecto (ver figura 8.22) y seleccionaremos “MBED OS developer site” (ver figura 8.23) y procederemos a registrarnos.

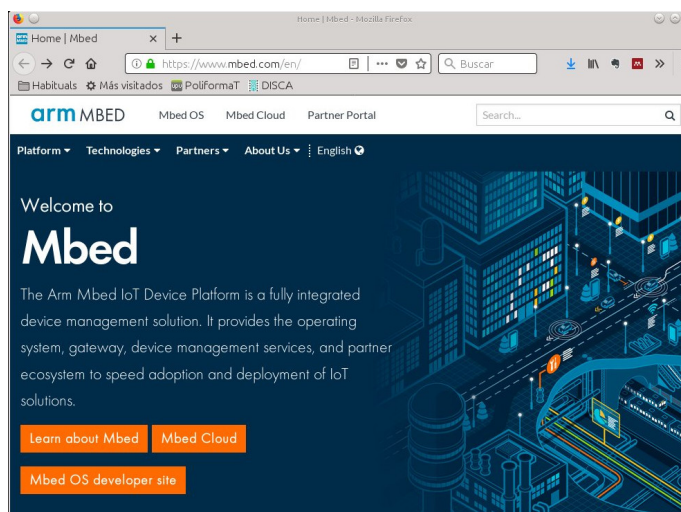


Figura 8.22: Página principal del proyecto MBED

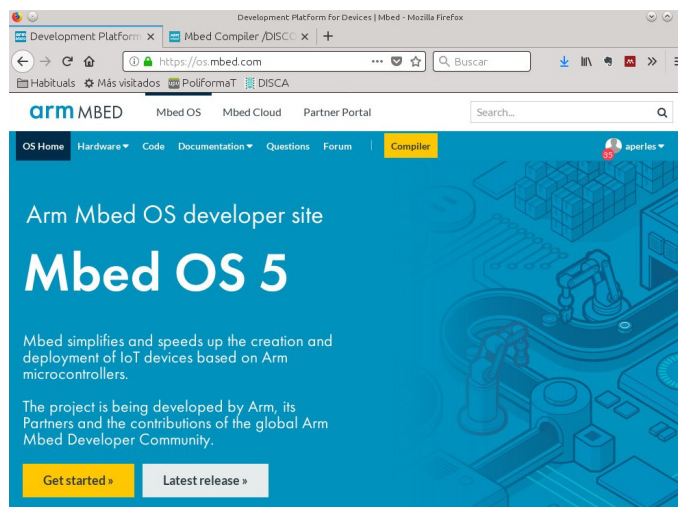


Figura 8.23: Saltamos a la página MBED clásico. Esto cambiará en pocos meses

Seleccionar plataforma: versión difícil

Un vez registrados, podemos elegir las plataforma que poseamos. Lo vamos hacer primero a la manera difícil, para ello elegimos la pestaña superior y seleccionamos “Hardware->boards”. Localizamos nuestra placa como se ve en la figura 8.24. Una vez elegida la placa, deberemos seleccionar el botón “Add to your mbed Compiler” para activar el compilador específico para la placa.

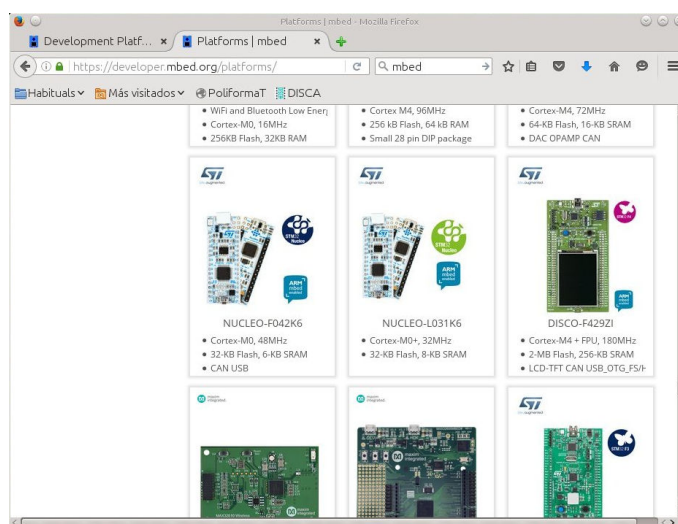


Figura 8.24: Elección de la plataforma

También es posible agregar componentes extra, como sensores, actuadores, etc. para ello seleccionar “Hardware->components” (ver figura

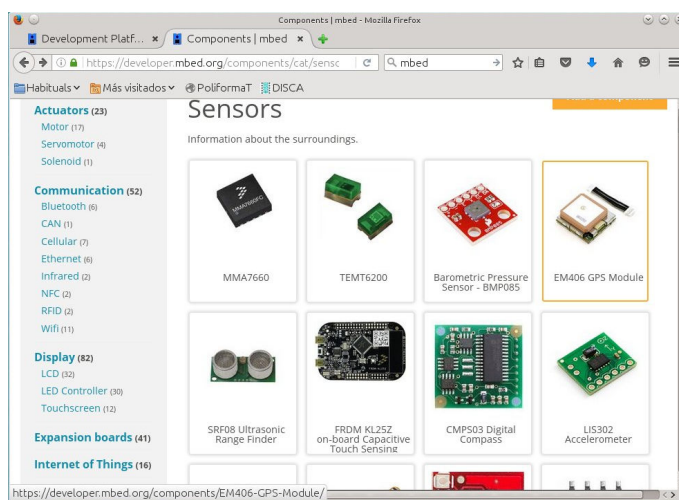


Figura 8.25: Componentes

Seleccionar plataforma: versión fácil

El apartado anterior pretende que se entiendan los pasos. La manera rápida es la siguiente.

- Pincha la placa al USB del ordenador
- Abre la unidad de disco que aparece
- Abre el documento MBED.HTM
- Introduce tus datos de registro

Plataforma identificada y lista para trabajar.

8.12.6 Los proyectos

Los proyectos de código pueden estar en el ordenador local, pero lo común es que se almacene en un repositorio profesional tipo “Mercurial” en el propio sistema MBED (el código fuente de este libro está en un repositorio Mercurial).

Para nuestra placa hay unos cuantos proyecto demo para probar que se pueden ver al elegir la plataforma. Se propone acceder al proyecto

https://developer.mbed.org/teams/ST/code/DISCO-F429ZI_LCD_demo/ cuya página se muestra en la figura y echar un vistazo rápido. No se pretende aprender a usarlos.

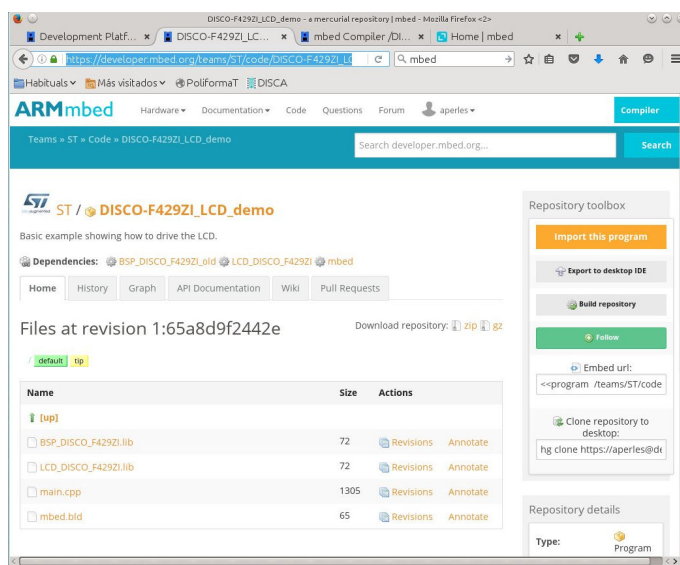


Figura 8.26: Repositorio de un proyecto MBED

8.12.7 El compilador

En la esquina superior-derecha de tu navegador se puede ver el botón de acceso al compilador. Púlsalo para abrir una nueva ventana en la que aparece un típico IDE para desarrollo como el mostrado en la figura 8.27. Echa un vistazo a las opciones.

Importar un proyecto

Para ir al grano, dar al botón “Import”, y en buscar escribimos “DISCO-F429Z” y damos a Buscar. Debería aparecer algo similar a lo mostrado en la figura 8.28.

Elegir “Import” del proyecto visto en el apartado anterior (es el mismo). Dará un error al actualizar bibliotecas, pero seguir adelante.

Construir el proyecto

Le damos a compilar (se dice construir) y esperamos un poco. Si todo ha ido bien, se abrirá un diálogo para descargar el resultado de la compilación.

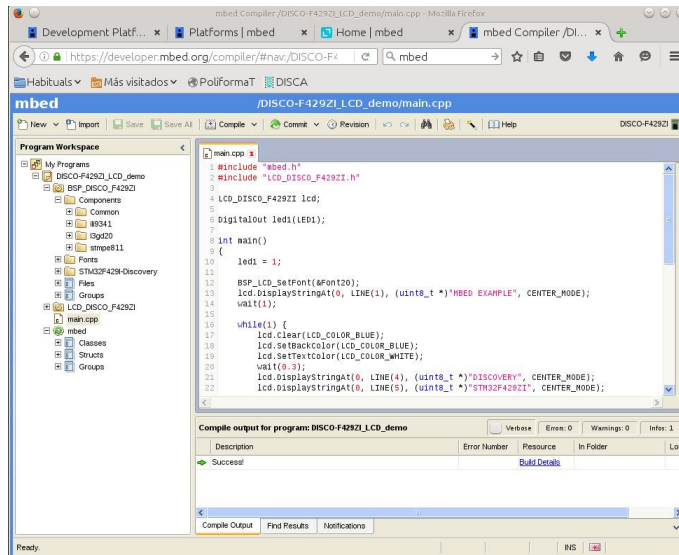


Figura 8.27: El compilador

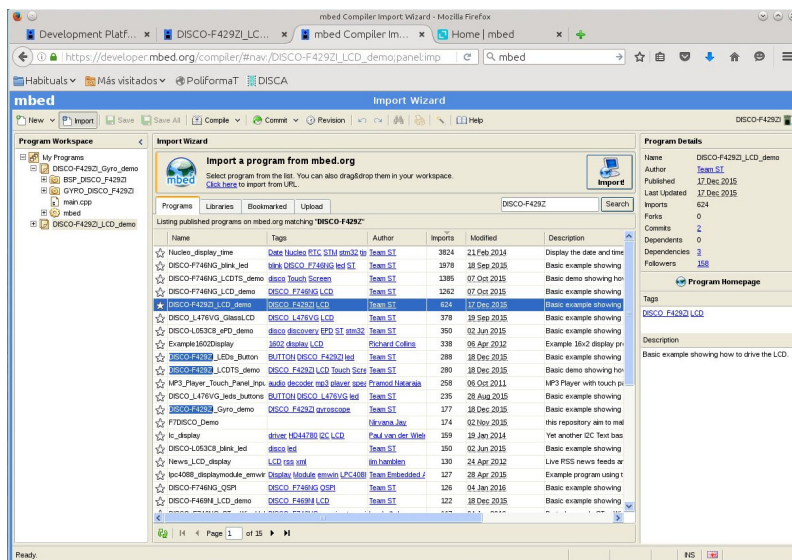


Figura 8.28: Buscando proyectos

Volcar a la placa

El archivo que se descarga lo sueltas en la unidad de la placa y a funcionar.

Aunque esta manera de trabajar tiene sus ventajas, también tiene sus desventajas. La principal es la dificultad para depurar el proyecto.

8.13 Manipulando osciladores y relojes (STM32L476)



Figura 8.29: Un cristal de cuarzo de 8 MHz.

8.13.1 Objetivos

- Manipular los osciladores y relojes del microcontrolador.

8.13.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeL4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa Nucleo-64 STM32L476RG.
- Cable USB tipo A a mini-B.

8.13.3 Introducción

El control de los relojes es primordial para poder controlar el ritmo al que se ejecutan los programas, la cadencia de los temporizadores, la velocidad de las transmisiones, etc. Además del reloj en sí, esto implica conocer y controlar distintos tipos de osciladores.

En esta práctica se propone probar el efecto del uso de distintos modos de reloj y oscilador en un microcontrolador de ultra-bajo consumo (el año que viene estaría bien medir corriente consumida).

8.13.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarlo en la placa para comprobar que todo está correctamente configurado.

Añadir la biblioteca para encender y apagar el LED y adaptarla a esta placa consultando en el manual de la placa la configuración hardware. Úsese de momento la función `HAL_Delay()` para hacer parpadear el LED a un ritmo de 500 ms.

Revisar a continuación la plantilla para ver cómo se ha configurado el reloj.

8.13.5 Tarea: Implementar la biblioteca `clocks`

Se pide crear una biblioteca específica para realizar las configuraciones del reloj. Como es parte de nuestra aplicación, esta biblioteca deberá estar en el mismo directorio que `main()`. Se pide que la biblioteca (cabecera e implementación) se escriban con el mejor estilo posible.

Desarrollar la función `clock_HSI16()` que configure el reloj del sistema (y alrededores) para usar el oscilador HSI interno de 16 MHz. Comprobar que funciona haciendo que la placa parpadee 5 veces con el reloj original y después continúe con el nuevo.

Desarrollar la función `clock_MSI4()` que configure el reloj del sistema (y alrededores) para usar el oscilador MSI interno a 4 MHz. Comprobar que funciona haciendo que la placa parpadee 5 veces con el reloj original y después continúe con el nuevo. Se puede usar STM32CubeMX para ayudar en la configuración.

Desarrollar la función `clock_MSIPLL80()` que configure el reloj del sistema (y alrededores) para usar el oscilador MSI interno y el sintetizador PLL y lograr funcionar a 80 MHz. Comprobar que funciona haciendo que la placa parpadee 5 veces con el reloj original y después continúe con el nuevo. Se puede usar STM32CubeMX para ayudar en la configuración.

Desarrollar la función `clock_HSEPLL80()` que configure el reloj del sistema (y alrededores) para usar el oscilador HSE externo de 8 MHz y el sintetizador PLL y lograr funcionar a 80 MHz. Comprobar que funciona haciendo que la placa parpadee 5 veces con el reloj original y después continúe con el nuevo. Se puede usar STM32CubeMX para ayudar en la configuración.

8.13.6 Tarea: Mejora de la biblioteca

Las funciones anteriores dejan encendidos o en un estado errático a los osciladores. Eso hay que hacerlo mejor.

Crea la función `clock_Default()` que configure el reloj del sistema a la configuración por defecto del micro al hacer reset y apague el resto de osciladores. Porbárla como se ha hecho en el caso anterior.

Modificar los ejemplos anteriores de manera que, antes de hacer un cambio, llamen a esta nueva función y después hagan la función solicitada.

8.14 Retardos de precisión (STM32L476)



Figura 8.30: Un cristal de cuarzo de 8 MHz.

8.14.1 Objetivos

- Practicar timers haciendo una retardo de precisión.

8.14.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeL4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa Nucleo-64 STM32L476RG.
- Cable USB tipo A a mini-B.

8.14.3 Introducción

Una de las aplicaciones básicas de los timers es realizar retardos de precisión. El procedimiento no es evidente a primera vista, por eso se aha dedicado una sesión de teoría a entender el enfoque.

En esta práctica se propone probar la función `delayus()` planteada en clase.

8.14.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarlo en la placa para comprobar que todo está correctamente configurado.

Añadir la biblioteca para encender y apagar el LED adaptada a la placa o emplear las funciones BSP que los manejan. Úsese la función `HAL_Delay()` para hacer parpadear el LED a un ritmo de 1 s.

8.14.5 Tarea: Implementar la biblioteca `delayus`

Se pide crear la biblioteca `delayus.c` propuesta en la sesión de teoría para poder hacer pausas de precisión basadas en las funcionalidad básica de un timer. Se pide que la biblioteca (cabecera e implementación) se escriban con el mejor estilo posible.

Atención, en clase metí la pata al no recordar poner en marcha el timer. Antes de toquetearlo, hacer:

```
__HAL_RCC_TIM6_CLK_ENABLE();
```

Esta se puede considerar como una biblioteca externa, por tanto deberá colocarse en el directorio de terceros.

Probar la biblioteca sustituyendo en el bucle anterior la función `HAL_Delay()` por la función `delayus()`. Podría quedar algo como:

```
while (1)
{
    BSP_LED_Toggle(LED2);
    //HAL_Delay(1000);
    delay_us(1000000);
}
```

Y comprobar que, a ojo, la pausa sigue teniendo la misma duración.

8.14.6 Tarea: CutrePWM

Modificar el superbucle para conseguir controlar el brillo del LED y lograr un efecto desvanecimiento, para ello ir encendiendo y apagando el LED controlando el tiempo a ON y el tiempo a OFF partiendo de fragmentar 1 ms en dos periodos.

Pregunta ¿Por qué es un cutrePWM?

8.15 Probando PWM sobre un LED (STM32L476)

Figura 8.31: Poner aquí la Peltier/o el motor y el MOSPower.

8.15.1 Objetivos

- Practicar timers generando una señal PWM.

8.15.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeL4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa Nucleo-64 STM32L476RG.
- Cable USB tipo A a mini-B.

8.15.3 Introducción

Una aplicación fundamental de los timers es la generación de señales PWM. En esta práctica se propone ponerla en práctica controlando el brillo del LED de la placa de prácticas.

8.15.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarlo en la placa para comprobar que todo está correctamente configurado.

8.15.5 Tarea: Configurar una señal PWM para atacar el LED de la placa

El LED está conectado en el pin PA5. Con esta información, decidir:

- Timer a emplear.
- Canal PWM a emplear.
- Configuración de la GPIO necesaria.

Una vez decididos estos parámetros, desarrollar el código de generación de PWM para logra:

- Una frecuencia PWM de 1 kHz.
- Un duty-cycle del 10 %.

No hace falta desarrollar código con buen estilo, con que funcione es suficiente. Comprobar que funciona volcándolo en la placa.

8.15.6 Tarea: Cútre-efecto desvanecimiento

En el superbucle, implementar una variación del duty-cycle para crear un efecto desvanecimiento-envanecimiento cíclico.

8.15.7 Reto

Lo digo a mitad de práctica.

8.16 Práctica: Un módulo para manejar servos

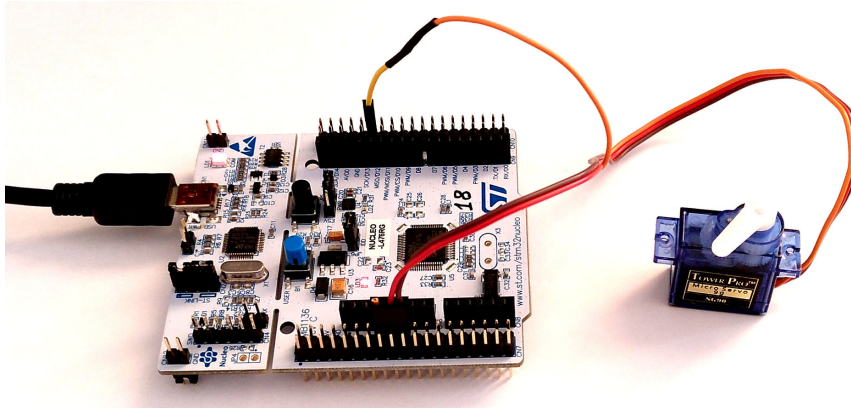


Figura 8.32: ¡Funciona! Ja, ja.

8.16.1 Objetivos

- Construir un módulo básico para gestionar un periférico.

8.16.2 Material necesario

- Ordenador personal con sistema operativo Microsoft Windows 7 o superior.
- Entorno de desarrollo Keil MDK-ARM 5.xx.
- STM32CubeL4 firmware package.
- Plantilla oficial del libro disponible en [librostm32].
- Placa Nucleo-64 STM32L476RG.
- Cable USB tipo A a mini-B.
- Servo Tower Pro SG-90.

IMPORTANTE: El montaje hardware que se propone en esta práctica es muy deficiente electrónicamente hablando. No es nada adecuado alimentar un servo con la tensión de USB ni atacar su entrada con una salidad de 3,3 V. No tomar como referencia. El servo también deja bastante que desear en cuanto a precisión.

8.16.3 Introducción

Se propone crear una aplicación test para manejar un servo analógico como excusa para practicar el desarrollo de módulos C enfocados a manejar un hardware externo.

Si se ha trabajado con Arduino, se sabrá que utilizar un servo es sencillísimo, pues basta emplear la biblioteca correspondiente y a correr. Pero, alguien habrá hecho esa biblioteca ¿no?

La diferencia entre un profesional y un aficionado está en la capacidad de hacer ese tipo de cosas.

8.16.4 Preparación inicial

Descargar y preparar la plantilla oficial y comprobar que se construye correctamente. Volcarla en la placa para comprobar que todo está correctamente configurado.

8.16.5 Actividad: desarrollar el módulo servo

Se propone el siguiente archivo de cabecera para `servo.h`

```
/**
    @file servo.h
    @brief Servo handling
 */

#ifndef SERVO_H
#define SERVO_H

void servo_Init(void);
void servo_SetPosition(uint16_t tenth_degree);

#endif
/**** End of file ****/
```

A partir de esta definición, realizar la implementación del módulo `servo.c` partiendo del siguiente esqueleto.

```
/**
    @file servo.cpp
    @brief Servo handling using bl, bla
 */

#include <stdint.h>
#include "servo.h"

static const uint16_t TENTH_DEGREES_MIN    = -900;
static const uint16_t TIMMING_MIN          = 1000;

static const uint16_t TENTH_DEGREES_MAX    = 900;
static const uint16_t TIMMING_MAX          = 2000;

static const uint16_t TIMMING_TOTAL        = 20000;
```

```
/**
 * *****
 */
/**
 * @brief Set-ups the underlaying hardware, bla, bla ...
 * @param none
 * Example:
 * @verbatim
 *     servo_Init(); // the position will be 45.6 degrees
 * @endverbatim
 */

void servo_Init(void)
{

}

/**
 * *****
 */
/**
 * @brief Sets the position of the servo
 * @param tenth_degree tenth of degree for the position
 * Example:
 * @verbatim
 *     servo_SetPosition(456); // the position will be 45.6 degrees
 * @endverbatim
 */

void servo_SetPosition(uint16_t tenth_degree)
{

}

/** End of file *****
```

Y usando el datasheet del servo.

Para la implementación del timer, se puede partir de las siguientes ideas:

- Para la salida de control, emplear la misma que el LED (PA5), el mismo timer, sin pull-ups y modo push-pull.
- Fijar el preescaler del timer de manera que proporcione una señal de 1 MHz (1 uS).
- Fijar el periodo del timer en 20.000 unidades para tener 20 ms de ciclo.
- Lanzar el funcionamiento del PWM en el valor central del servo (0°).
- Las actualizaciones del ciclo PWM hacerlas accediendo directamente al registro CC. Por ejemplo `htim2.Instance->CCR1 = 2000;`.

8.16.6 Actividad: comprobar el funcionamiento del módulo

Para hacer una primera comprobación del módulo, generar la señal para la posición central del servo y emplear un osciloscopio para verificar que es correcta. Por ejemplo, haciendo:

```
servo_Init();  
servo_SetPosition(0);
```

8.16.7 Actividad: comprobar funcionamiento con el servo

Desconectar la placa Nucle y montar el servo con mucha atención conectando GND al pin 20 del conector CN7 y alimentación al pin 18 del conector CN7 (ver trasera blister Nucleo). Conectar señal de control al pin 11 de CN10.

Comprobar que el programa anterior lleva el servo a la posición central (a ojo).

Modificar el programa anterior para lograr un efecto de movimiento. Por ejemplo, se propone el siguiente listado:

```
servo_Init();  
  
for(;;)  
{  
    servo_SetPosition(-90);  
    HAL_Delay(3000);  
  
    servo_SetPosition(0);  
    HAL_Delay(3000);  
  
    servo_SetPosition(90);  
    HAL_Delay(3000);  
  
    HAL_Delay(3000);  
}
```

Glosario

AHB (Advanced High-performance Bus)

APB (Advanced Peripheral Bus)

